

DeepSeek-V4 :

迈向高效的百万 Token 上下文智能

DeepSeek-AI research@deepseek.com

摘要

我们发布了 DeepSeek-V4 系列的预览版本，其中包括两款强大的混合专家 (MoE) 语言模型：拥有 1.6T 参数 (49B 激活) 的 DeepSeek-V4-Pro，以及拥有 284B 参数 (13B 激活) 的 DeepSeek-V4-Flash；二者都支持一百万 Token 的上下文长度。DeepSeek-V4 系列在架构与优化方面包含若干关键升级：(1) 将压缩稀疏注意力 (CSA) 与重度压缩注意力 (HCA) 结合的混合注意力架构，以提升长上下文效率；(2) 增强传统残差连接的流形约束超连接 (mHC)；(3) 用于更快收敛和更高训练稳定性的 Muon 优化器。我们在超过 32T 的多样化高质量 Token 上对两个模型进行了预训练，随后采用全面的后训练流水线来释放并进一步增强其能力。DeepSeek-V4-Pro-Max 作为 DeepSeek-V4-Pro 的最高推理力度模式，重新定义了开源模型的最新水平，在核心任务上超越了其前代。同时，DeepSeek-V4 系列在长上下文场景下也极为高效。在一百万 Token 的上下文设定中，相比 DeepSeek-V3.2，DeepSeek-V4-Pro 的单 Token 推理 FLOPs 仅为 27%，KV 缓存仅为 10%。这使我们能够常态化支持一百万 Token 上下文，从而让长时程任务和进一步的测试时扩展更加可行。模型检查点可在 <https://huggingface.co/collections/deepseek-ai/deepseek-v4> 获取。

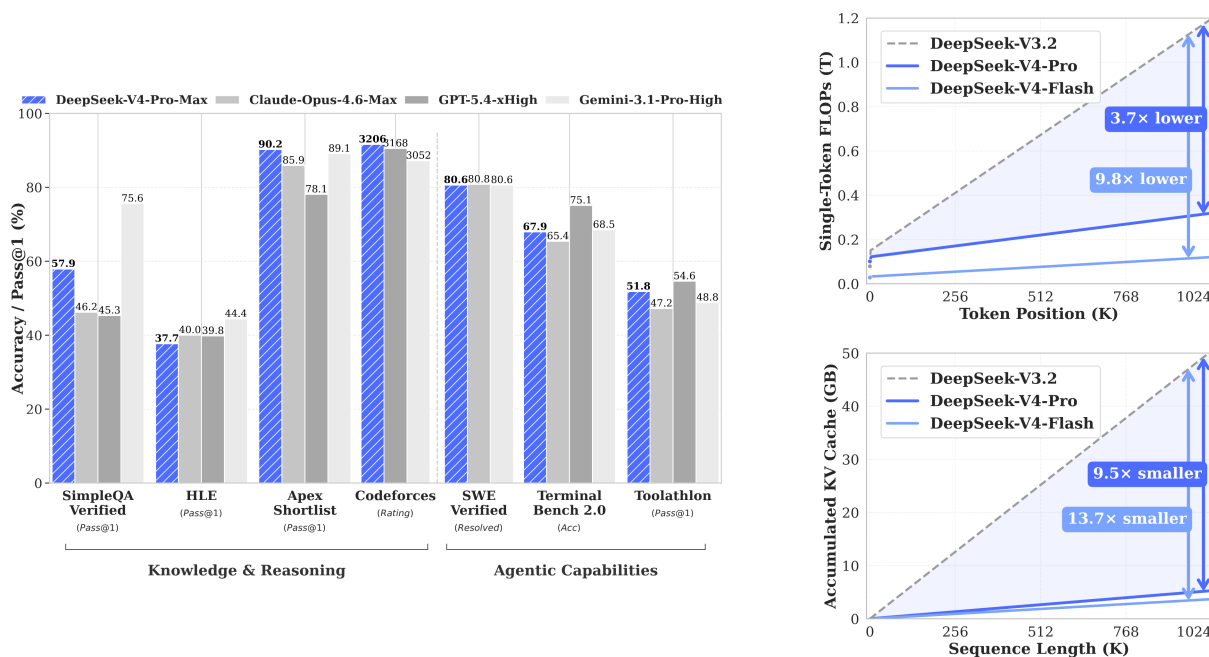


图 1 | 左：DeepSeek-V4-Pro-Max 与其对比模型的基准性能。右：DeepSeek-V4 系列与 DeepSeek-V3.2 的推理 FLOPs 与 KV 缓存大小。

Contents

摘要	1
1 引言	5
2 架构	8
2.1 继承自 DeepSeek-V3 的设计	8
2.2 流形约束超连接	8
2.3 结合 CSA 与 HCA 的混合注意力	10
2.3.1 压缩稀疏注意力	11
2.3.2 重度压缩注意力	13
2.3.3 其他细节	14
2.3.4 效率讨论	15
2.4 Muon 优化器	16
3 通用基础设施	16
3.1 专家并行中细粒度的通信-计算重叠	16
3.2 使用 TileLang 进行灵活高效的 Kernel 开发	18
3.3 高性能、批不变且确定性的 Kernel 库	19
3.4 FP4 量化感知训练	20
3.5 训练框架	20
3.5.1 Muon 的高效实现	20
3.5.2 mHC 的低成本且内存高效实现	21
3.5.3 面向长上下文注意力的上下文并行	21
3.5.4 面向灵活激活检查点的扩展自动微分	22
3.6 推理框架	22
3.6.1 KV Cache 结构与管理	22
3.6.2 磁盘上的 KV Cache 存储	25
4 预训练	25
4.1 数据构建	25
4.2 预训练设置	26
4.2.1 模型设置	26
4.2.2 训练设置	26
4.2.3 缓解训练不稳定性	27
4.3 评测	28
4.3.1 评测基准	28
4.3.2 评测结果	29
5 后训练	29
5.1 后训练流程	29

5.1.1	专家训练	29
5.1.2	在策略蒸馏	33
5.2	RL 与 OPD 基础设施	34
5.2.1	FP4 量化集成	34
5.2.2	面向全词表 OPD 的高效教师调度	34
5.2.3	可抢占且容错的 Rollout 服务	34
5.2.4	面向百万 Token 上下文的 RL 框架扩展	35
5.2.5	面向智能体 AI 的沙箱基础设施	35
5.3	标准基准评估	36
5.3.1	评估设置	36
5.3.2	评估结果	37
5.4	真实世界任务上的表现	42
5.4.1	中文写作	43
5.4.2	搜索	43
5.4.3	白领任务	43
5.4.4	代码智能体	48
6	结论、局限性与未来方向	48
A	作者名单与致谢	57
A.1	Author List	57
A.2	致谢	58
B	评测细节	58

1 引言

推理模型 (DeepSeek-AI, 2025; OpenAI, 2024c) 的出现, 重新确立了测试时扩展这一新范式, 并为大语言模型 (LLM) 带来了显著的性能提升。然而, 这一扩展范式从根本上受限于原始注意力机制 (Vaswani et al., 2017) 的二次计算复杂度, 这为超长上下文与推理过程带来了难以承受的瓶颈。与此同时, 长时程场景与任务 — 从复杂的智能体工作流到大规模跨文档分析 — 的出现, 也使得对超长上下文的高效支持成为未来进展的关键。尽管近期的开源工作 (Bai et al., 2025a; DeepSeek-AI, 2024; MiniMax, 2025; Qwen, 2025) 推动了通用能力的发展, 但在处理超长序列时, 这一核心架构低效性仍然是主要障碍, 既限制了测试时扩展的进一步收益, 也阻碍了对长时程场景与任务的进一步探索。

为了突破超长上下文中的效率瓶颈, 我们提出了 DeepSeek-V4 系列, 其中包括 DeepSeek-V4-Pro 的预览版 (1.6T 参数, 49B 激活) 以及 DeepSeek-V4-Flash 的预览版 (284B 参数, 13B 激活)。借助架构创新, DeepSeek-V4 系列在处理超长序列时实现了计算效率的巨大飞跃。这一突破使模型能够高效支持一百万 token 的上下文长度, 开启了下一代 LLM 的百万长度上下文时代。我们相信, 高效处理超长序列的能力将开启测试时扩展的下一前沿, 为长时程任务的深入研究铺平道路, 并为探索诸如在线学习等未来范式奠定必要基础。

与 DeepSeek-V3 架构 (DeepSeek-AI, 2024) 相比, DeepSeek-V4 系列保留了 DeepSeekMoE 框架 (Dai et al., 2024) 和多 Token 预测 (MTP) 策略, 同时在架构和优化上引入了若干关键创新。为提升长上下文效率, 我们设计了一种结合压缩稀疏注意力 (Compressed Sparse Attention, CSA) 和重度压缩注意力 (Heavily Compressed Attention, HCA) 的混合注意力机制。CSA 沿序列维度压缩 KV cache, 然后执行 DeepSeek Sparse Attention (DSA) (DeepSeek-AI, 2025); 而 HCA 则对 KV cache 施加更激进的压缩, 但保留稠密注意力。为增强建模能力, 我们引入了流形约束超连接 (Manifold-Constrained Hyper-Connections, mHC) (Xie et al., 2026), 以升级传统残差连接。此外, 我们还将 Muon (Jordan et al., 2024; Liu et al., 2025) 优化器用于 DeepSeek-V4 系列的训练, 从而实现更快收敛和更好的训练稳定性。

为了让 DeepSeek-V4 系列实现高效训练与推理, 并提升开发效率, 我们还引入了若干基础设施优化。首先, 我们为 MoE 模块设计并实现了单一融合 kernel, 使计算、通信和内存访问能够完全重叠。其次, 我们采用 TileLang (Wang et al., 2026) 这一领域专用语言 (DSL), 以在开发效率与运行效率之间取得平衡。第三, 我们提供了高效的 batch-invariant 和 deterministic kernel 库, 以确保训练与推理过程中的按位可复现性。第四, 我们在 MoE 专家权重以及索引器 QK 路径上引入了 FP4 量化感知训练, 以降低内存与计算开销。第五, 在训练框架方面, 我们通过张量级 checkpointing 扩展了 autograd 框架, 以实现细粒度的重计算控制; 同时, 还通过针对 Muon 优化器的混合 ZeRO 策略、借助重计算与融合 kernel 的高性价比 mHC 实现, 以及用于管理压缩注意力的两阶段上下文并行, 来提升训练效率。最后, 在推理框架方面, 我们设计了具有磁盘存储策略的异构 KV cache 结构, 以实现高效的共享前缀复用。

通过采用混合 CSA 和 HCA, 并结合在计算与存储上的精度优化, DeepSeek-V4 系列相较 DeepSeek-V3.2 实现了显著更低的推理 FLOPs 和大幅更小的 KV cache 尺寸, 尤其是在长上下文场景下更是如此。图 1 右侧展示了 DeepSeek-V3.2 与 DeepSeek-V4 系列的估算单 token 推理 FLOPs 和累计 KV cache 大小。在 1M token 上下文场景中, 即便是激活参数更多的 DeepSeek-V4-Pro, 其单 token FLOPs (按等效 FP8 FLOPs 计) 也仅为 DeepSeek-V3.2 的 27%, KV cache 大小仅为 10%。此外, 激活参数更少的 DeepSeek-V4-Flash 进一步提升了效率: 在 1M token 上下文设置下, 它的单 token FLOPs 仅为 DeepSeek-V3.2 的 10%, KV cache 大小仅为 7%。另外, 对于 DeepSeek-V4 系列, 路由专家参数采用 FP4 精度。尽管在现有硬件上, $FP4 \times FP8$ 运算的峰值 FLOPs 目前

与 $\text{FP8} \times \text{FP8}$ 相同，但从理论上讲，在未来硬件上它可以实现高出 1/3 的效率，这将进一步提升 DeepSeek-V4 系列的效率。

在预训练阶段，我们分别用 32T token 训练 DeepSeek-V4-Flash，用 33T token 训练 DeepSeek-V4-Pro。完成预训练后，这两个模型都可以原生且高效地支持 1M 长度上下文。在我们的内部评测中，凭借更高的参数效率设计，DeepSeek-V4-Flash-Base 已在大多数基准上超越 DeepSeek-V3.2-Base。DeepSeek-V4-Pro-Base 则进一步扩大了这一优势，为 DeepSeek 基础模型树立了新的性能标准，在推理、代码、长上下文和世界知识任务上实现了全面领先。

DeepSeek-V4 系列的后训练流程采用两阶段范式：先独立培养领域专长专家，再通过 on-policy distillation (Lu and Lab, 2025) 完成统一模型整合。首先，对于每个目标领域——如数学、编程、智能体以及指令跟随——都会独立训练一个专门的专家模型。基础模型先在高质量的领域数据上进行监督微调 (SFT)，以建立基础能力。随后，再使用 Group Relative Policy Optimization (GRPO) (DeepSeek-AI, 2025) 进行强化学习 (RL)，在面向特定成功标准设计的奖励模型引导下，进一步优化模型的领域对齐行为。该阶段会产出一组多样化的专长专家，每个专家都在各自领域表现出色。最后，为了整合这些不同的能力，我们通过 on-policy distillation 训练一个统一模型，其中统一模型作为学生，学习在教师模型指导下优化 reverse KL loss。

核心评测结果摘要

- 知识：在广泛世界知识评测中，DeepSeek-V4-Pro 的最大推理努力模式 DeepSeek-V4-Pro-Max 在 SimpleQA (OpenAI, 2024d) 和 Chinese-SimpleQA (He et al., 2024) 基准上显著优于领先的开源模型。在教育知识方面——通过 MMLU-Pro (Wang et al., 2024b)、HLE (Phan et al., 2025) 和 GPQA (Rein et al., 2023) 评测——DeepSeek-V4-Pro-Max 也较开源同类模型略有领先。尽管在这些知识类评测上仍落后于领先的闭源模型 Gemini-3.1-Pro，但 DeepSeek-V4-Pro-Max 已显著缩小了与其之间的差距。
- 推理：通过扩展推理 token，DeepSeek-V4-Pro-Max 在标准推理基准上展现出相对于 GPT-5.2 和 Gemini-3.0-Pro 的更强性能。不过，其表现仍略逊于 GPT-5.4 和 Gemini-3.1-Pro，这表明其发展进度大约落后于最前沿模型 3 到 6 个月。此外，DeepSeek-V4-Flash-Max 也达到了与 GPT-5.2 和 Gemini-3.0-Pro 相当的性能，证明其在复杂推理任务上是一种极具性价比的架构。

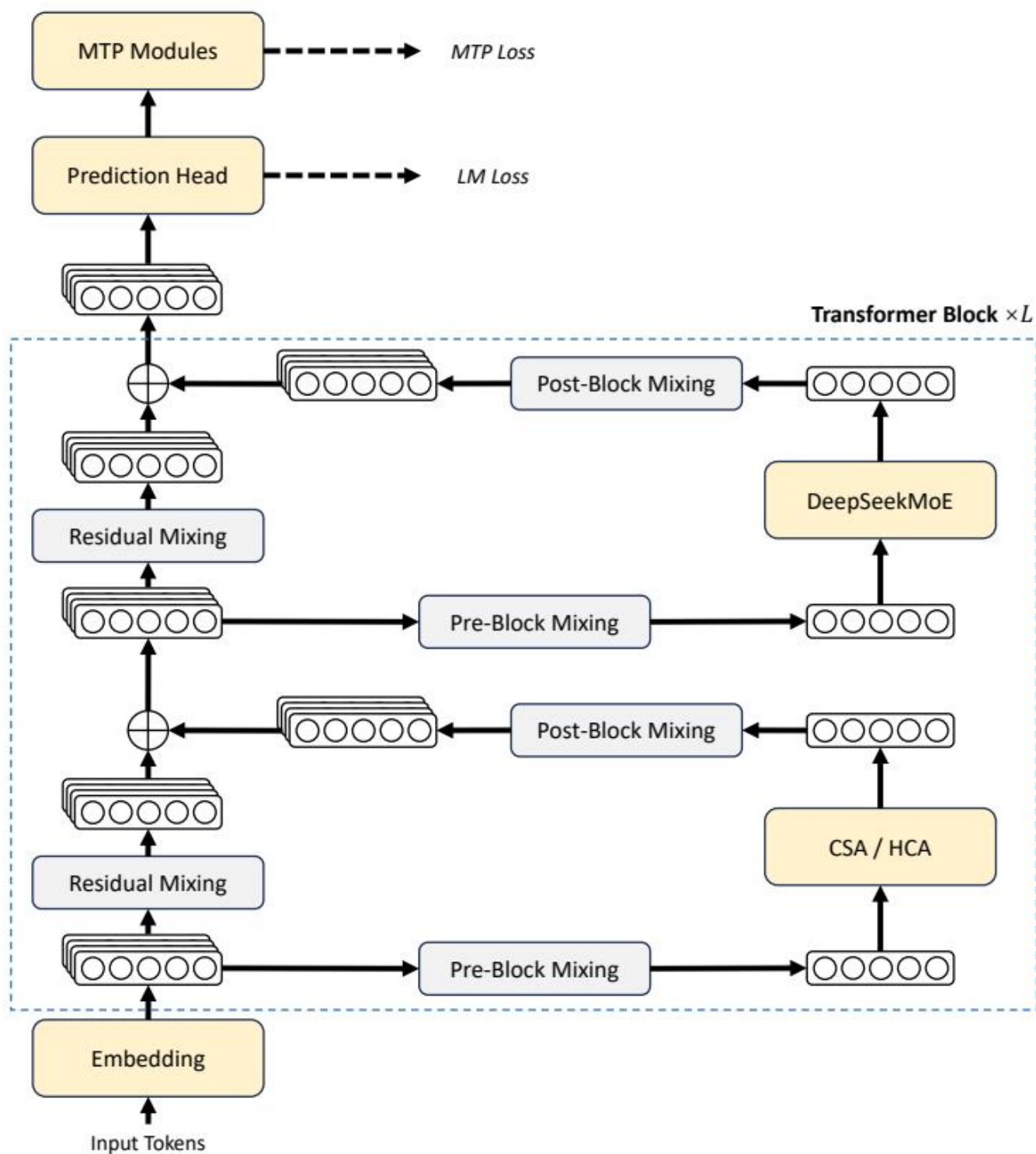


图 2 | DeepSeek-V4 系列的整体架构。我们在注意力层使用混合 CSA (Compressed Sparse Attention) 和 HCA (Heavily Compressed Attention) , 在前馈层使用 DeepSeekMoE , 并使用 mHC 强化传统残差连接。

- 智能体：在公开基准上，DeepSeek-V4-Pro-Max 与 Kimi-K2.6 和 GLM-5.1 等领先开源模型表现相当，但略逊于前沿闭源模型。在我们的内部评测中，DeepSeek-V4-Pro-Max 优于 Claude Sonnet 4.5，并接近 Opus 4.5 的水平。
- 长上下文：DeepSeek-V4-Pro-Max 在 100 万 token 上下文窗口下的合成与真实用例中都表现强劲，甚至在学术基准上超过了 Gemini-3.1-Pro。
- DeepSeek-V4-Pro 与 DeepSeek-V4-Flash：由于参数规模更小，DeepSeek-V4-Flash-Max 在知识类评测中的表现较低。不过，当分配更大的思考预算时，它在推理任务上可以取得可比结果。在智能体评测中，尽管 DeepSeek-V4-Flash-Max 在若干基准上与 DeepSeek-V4-Pro-Max 表现相当，但在更复杂、更高难度的任务上仍落后于更大的版本。

2 架构

总体而言，DeepSeek-V4 系列保留了 Transformer (Vaswani et al., 2017) 架构和多 Token 预测 (MTP) 模块 (DeepSeek-AI, 2024; Gloeckle et al., 2024)，同时相较 DeepSeek-V3 引入了若干关键升级：(1) 首先，我们引入流形约束超连接 (mHC) (Xie et al., 2026) 以强化传统残差连接；

- (2) 其次，我们设计了一种混合注意力架构，通过压缩稀疏注意力和重度压缩注意力大幅提升长上下文效率。(3) 第三，我们采用 Muon (Jordan et al., 2024; Liu et al., 2025) 作为优化器。对于混合专家 (MoE) 组件，我们仍然采用 DeepSeekMoE (Dai et al., 2024) 架构，仅对 DeepSeek-V3 做了少量调整。多 Token 预测 (MTP) (DeepSeek-AI, 2024; Gloeckle et al., 2024; Li et al., 2024; Qi et al., 2020) 的配置与 DeepSeek-V3 完全一致。其余未特别说明的细节均遵循 DeepSeek-V3 (DeepSeek-AI, 2024) 中已建立的设定。图 2 展示了 DeepSeek-V4 的整体架构，具体细节如下所述。

2.1 继承自 DeepSeek-V3 的设计

混合专家。与以往的 DeepSeek 系列模型 (DeepSeek-AI, 2024; DeepSeek-AI, 2024) 一致，DeepSeek-V4 系列在前馈网络 (FFN) 中同样采用 DeepSeekMoE 范式 (Dai et al., 2024)，其中设置了细粒度路由专家和共享专家。与 DeepSeek-V3 不同的是，我们将用于计算亲和分数的激活函数从 Sigmoid($\cdot$) 改为 Sqrt(Softplus($\cdot$))。在负载均衡方面，我们也采用了无辅助损失策略 (DeepSeek-AI, 2024; Wang et al., 2024a)，并辅以轻量的按序列均衡损失，以防止单个序列内部出现极端失衡。对于 DeepSeek-V4，我们移除了对路由目标节点数量的约束，并精心重新设计了并行策略，以维持训练效率。此外，相比 DeepSeek-V3，我们将最前面若干个 Transformer block 中的稠密 FFN 层替换为采用 Hash routing (Roller et al., 2021) 的 MoE 层。Hash routing 策略会根据输入 token ID 对应的预定义哈希函数，为每个 token 确定目标专家。

多 Token 预测。与 DeepSeek-V3 一样，DeepSeek-V4 系列同样设置了 MTP 模块和目标。鉴于 MTP 策略已经在 DeepSeek-V3 中得到验证，我们在 DeepSeek-V4 系列中不加修改地采用同样的策略。

2.2 流形约束超连接

如图 2 所示，DeepSeek-V4 系列引入了流形约束超连接 (Manifold-Constrained Hyper-Connections, mHC) (Xie et al., 2026)，以强化相邻 Transformer block 之间的传统残差连接。与朴素超连接 (Hyper-Connections,

HC) (Zhu et al., 2025) 相比, mHC 的核心思想是将残差映射约束到特定流形上, 从而在保留模型表达能力的同时, 增强跨层信号传播的稳定性。本小节将简要介绍标准 HC, 并说明我们如何设计 mHC 以实现稳定训练。

标准超连接。标准 HC 将残差流的宽度按 n_{hc} 倍扩展。具体来说, 残差流的形状会从 $\mathbb{R}^d$ 扩展为 $\mathbb{R}^{n_{\text{hc}} \times d}$, 其中 d 是实际层输入的隐藏维度。设 $X_l = [\mathbf{x}_{l,1}; \dots; \mathbf{x}_{l,n_{\text{hc}}}]^T \in \mathbb{R}^{n_{\text{hc}} \times d}$ 为第 l 层之前的残差状态。HC 引入了三个线性映射: 输入映射 $A_l \in \mathbb{R}^{1 \times n_{\text{hc}}}$, 残差变换 $B_l \in \mathbb{R}^{n_{\text{hc}} \times n_{\text{hc}}}$, 以及输出映射 $\bar{C}_l \in \mathbb{R}^{n_{\text{hc}} \times 1}$ 。残差状态的更新形式如下:

$$X_{l+1} = B_l X_l + C_l \mathcal{F}_l(A_l X_l), \quad (1)$$

其中 $\mathcal{F}_l$ 表示第 l 层 (例如某个 MoE 层), 其输入和输出形状都为 $\mathbb{R}^d$ 。需要注意的是, 实际层输入 $A_l X_l \in \mathbb{R}^d$ 也是 d 维的, 因此扩展后的残差宽度不会影响内部层的设计。HC 将残差宽度与实际隐藏维度解耦, 以极小的计算开销提供了一个互补的扩展维度, 因为 n_{hc} 通常远小于隐藏维度 d 。然而, 尽管 HC 已表现出提升模型性能的潜力, 我们发现当堆叠多层时, 训练过程经常会出现数值不稳定, 这阻碍了 HC 的进一步扩展。

流形约束残差映射。mHC 的核心创新在于, 将残差映射矩阵 B_l 约束到双随机矩阵流形 (即 Birkhoff 多面体) $M_{\odot}$ 上, 从而增强跨层信号传播的稳定性:

$$B_l \in \mathcal{M} := \{M \in \mathbb{R}^{n \times n} \mid M \mathbf{1}_n = \mathbf{1}_n, \mathbf{1}_n^T M = \mathbf{1}_n^T, M \geq 0\}. \quad (2)$$

这一约束确保映射矩阵的谱范数 $\|B_l\|_2$ 被限制在 1 以内, 因此残差变换是非扩张的, 从而提升前向传播和反向传播过程中的数值稳定性。此外, 集合 $\mathcal{M}$ 对乘法封闭, 这保证了在深层 mHC 堆叠场景下的稳定性。除此之外, 输入变换 A_l 和输出变换 C_l 也通过 Sigmoid 函数被约束为非负且有界, 以避免信号抵消的风险。

动态参数化。三个线性映射的参数采用动态生成方式, 并被分解为动态 (与输入相关) 部分和静态 (与输入无关) 部分。给定输入 $\bar{X}_l \in \mathbb{R}^{n_{\text{hc}} \times d}$, 我们首先将其展平并归一化: $\hat{X}_l = \text{RMSNorm}(\text{vec}(X_l)) \in \mathbb{R}^{1 \times n_{\text{hc}} d}$ 。随后, 我们遵循常规 HC 生成无约束原始参数 $\tilde{A}_l \in \mathbb{R}^{1 \times n_{\text{hc}}}$ 、 $\tilde{B}_l \in \mathbb{R}^{n_{\text{hc}} \times n_{\text{hc}}}$ 和 $\tilde{C}_l \in \mathbb{R}^{n_{\text{hc}} \times 1}$:

$$\tilde{A}_l = \alpha_l^{\text{pre}} \cdot (\hat{X}_l W_l^{\text{pre}}) + S_l^{\text{pre}}, \quad (3)$$

$$\tilde{B}_l = \alpha_l^{\text{res}} \cdot \text{Mat}(\hat{X}_l W_l^{\text{res}}) + S_l^{\text{res}}, \quad (4)$$

$$\tilde{C}_l = \alpha_l^{\text{post}} \cdot (\hat{X}_l W_l^{\text{post}})^T + S_l^{\text{post}}, \quad (5)$$

其中 $W_l^{\text{pre}}, W_l^{\text{post}} \in \mathbb{R}^{n_{\text{hc}} d \times n_{\text{hc}}}$ 和 $W_l^{\text{res}} \in \mathbb{R}^{n_{\text{hc}} d \times n_{\text{hc}}^2}$ 是用于生成动态部分的可学习参数; $\text{Mat}(\cdot)$ 将一个大小为 $1 \times n_{\text{hc}}^2$ 的向量重塑为大小为 $n_{\text{hc}} \times n_{\text{hc}}$ 的矩阵; $S_l^{\text{pre}} \in \mathbb{R}^{1 \times n_{\text{hc}}}$, $S_l^{\text{post}} \in \mathbb{R}^{n_{\text{hc}} \times 1}$, 以及 $S_l^{\text{res}} \in \mathbb{R}^{n_{\text{hc}} \times n_{\text{hc}}}$ 是可学习的静态偏置; 而 $\alpha_l^{\text{pre}}, \alpha_l^{\text{res}}, \alpha_l^{\text{post}} \in \mathbb{R}$ 是初始化为较小数值的可学习门控因子。

施加参数约束。在得到无约束原始参数 $\tilde{A}_l, \tilde{B}_l, \tilde{C}_l$ 后，我们再对其施加前述约束，以增强数值稳定性。具体而言，对于输入映射和输出映射，我们使用 Sigmoid 函数 $\sigma(\cdot)$ 来确保它们的非负性与有界性：

$$A_l = \sigma(\tilde{A}_l), \quad (6)$$

$$C_l = 2\sigma(\tilde{C}_l). \quad (7)$$

至于残差映射 $\tilde{B}_l$ ，我们将其投影到双随机矩阵流形 $\mathcal{M}$ 上。这一过程通过 Sinkhorn-Knopp 算法实现：首先对 $\tilde{B}_l$ 应用指数函数以确保其为正，得到 $M^{(0)} = \exp(\tilde{B}_l)$ ，随后迭代执行列归一化和行归一化：

$$M^{(t)} = \mathcal{T}_r(\mathcal{T}_c(M^{(t-1)})), \quad (8)$$

其中 $\mathcal{T}_r$ 和 $\mathcal{T}_c$ 分别表示行归一化与列归一化。该迭代会收敛到一个受约束的双随机矩阵 $B_l = M^{(t_{\max})}$ 。我们选择 $t_{\max} = 20$ 作为一个实用取值。

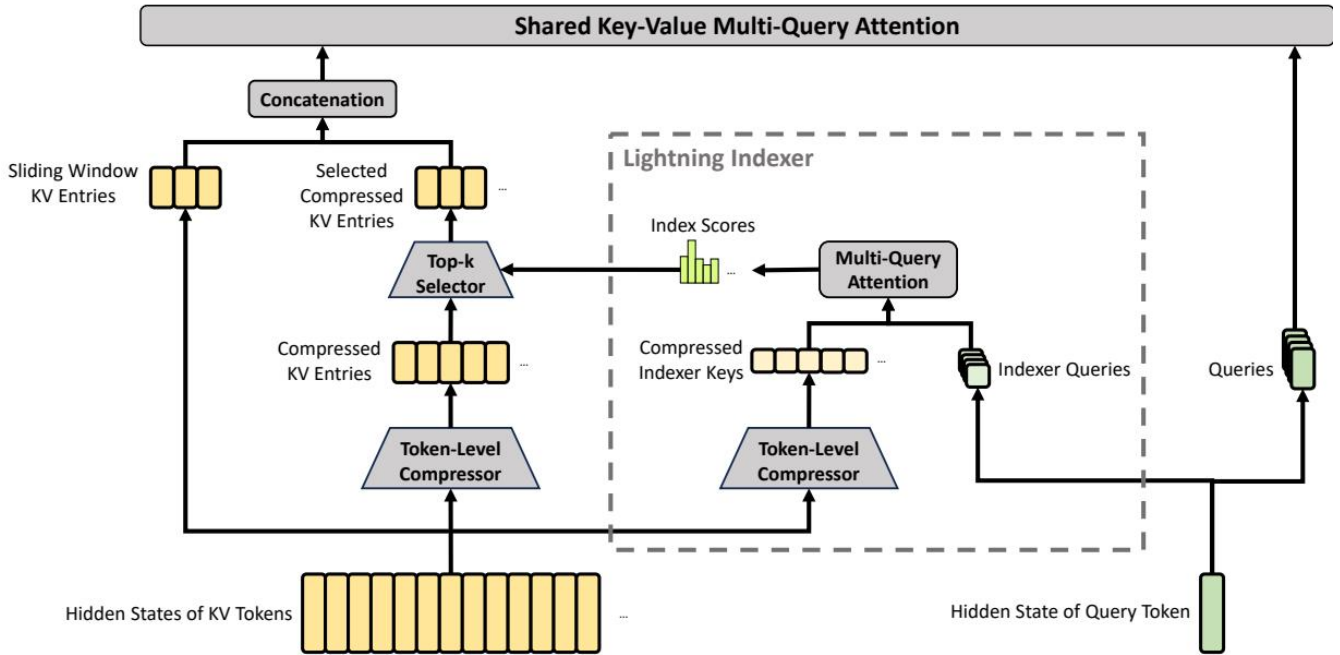


图 3 | CSA 的核心架构。它将 KV 条目的数量压缩为 $\frac{1}{m}$ 倍，然后应用 DeepSeek Sparse Attention 以进一步加速。此外，一小组滑动窗口 KV 条目会与选中的压缩 KV 条目结合，以增强局部细粒度依赖。

2.3 结合 CSA 与 HCA 的混合注意力

当上下文长度达到极端规模时，注意力机制会成为模型中的主导计算瓶颈。针对 DeepSeek-V4，我们设计了两种高效注意力架构——压缩稀疏注意力 (CSA) 和重度压缩注意力 (HCA)——并采用它们交错的混合配置，

从而显著降低长文本场景中的注意力计算开销。CSA 同时结合压缩和稀疏注意力策略：它首先将每 m 个 token 的 Key-Value (KV) cache 压缩为一个条目，然后应用 DeepSeek Sparse Attention (DSA) (DeepSeek-AI, 2025)，其中每个查询 token 仅关注 k 个压缩 KV 条目。HCA 则追求极致压缩，将每 $m' (\gg m)$ 个 token 的 KV cache 合并为单个条目。CSA 与 HCA 的混合架构显著提升了 DeepSeek-V4 系列的长上下文效率，使一百万 token 上下文在实践中成为可能。本小节将介绍我们混合注意力架构的核心技术；此外，我们还提供了开源实现¹，以无歧义地说明更多细节。

2.3.1 压缩稀疏注意力

图 3 展示了 CSA 的核心架构：它首先将每 m 个 token 的 KV cache 压缩为一个条目，然后应用 DeepSeek Sparse Attention 以进一步加速。

压缩 Key-Value 条目。设 $H \in \mathbb{R}^{n \times d}$ 为输入隐藏状态序列，其中 n 是序列长度， d 是隐藏维度。CSA 首先计算两组 KV 条目 $C^a, C^b \in \mathbb{R}^{n \times c}$ 及其对应的压缩权重 $Z^a, Z^b \in \mathbb{R}^{n \times c}$ ，其中 c 为头维度：

$$C^a = H \cdot W^{aKV}, \quad C^b = H \cdot W^{bKV}, \quad (9)$$

$$Z^a = H \cdot W^{aZ}, \quad Z^b = H \cdot W^{bZ}, \quad (10)$$

其中 $W^{aKV}, W^{bKV}, W^{aZ}, W^{bZ} \in \mathbb{R}^{d \times c}$ 是可训练参数。接下来， C^a 和 C^b 中每 m 个 KV 条目会依据其压缩权重以及可学习的位置偏置 $B^a \square^a, B^b \in \mathbb{R}^{m \times c}$ 被压缩为一个条目，生成 $C^{\text{Comp}} \in \mathbb{R}^{\frac{n}{m} \times c}$ 。每个压缩条目 $C_i^{\text{Comp}} \in \mathbb{R}^c$ 的计算方式为

$$[S_{mi:m(i+1)-1}^a; S_{m(i-1):mi-1}^b] = \text{Softmax}_{\text{row}}([Z_{mi:m(i+1)-1}^a + B^a; Z_{m(i-1):mi-1}^b + B^b]), \quad (11)$$

$$C_i^{\text{Comp}} = \sum_{j=mi}^{m(i+1)-1} S_j^a \odot C_j^a + \sum_{j=m(i-1)}^{mi-1} S_j^b \odot C_j^b, \quad (12)$$

其中 $\odot$ 表示 Hadamard 积； $\text{Softmax}_{\text{row}}(\cdot)$ 表示沿行维度进行 softmax 操作，即在来自 Z^a 和 Z^b 的共 $2m$ 个元素上进行归一化。当 $i = 0$ 时， $Z_{m(i-1):mi-1}^b$ 用负无穷填充， $C_{m(i-1):mi-1}^b$ 用零填充。注意，每个 C_i^{Comp} 都由 $2m$ 个 KV 条目导出，但用于 C_i^{Comp} 的 C^b 索引与用于前一个压缩条目 C_{i-1}^{Comp} 的 C^a 索引存在重叠。因此，CSA 实际上将序列长度压缩到了 $\frac{1}{m}$ 倍。

用于稀疏选择的 Lightning 索引器。在得到压缩 KV 条目 C^{Comp} 后，CSA 应用 DSA 策略来为核心注意力选择 top-k 个压缩 KV 条目。首先，CSA 使用与 C^{Comp} 相同的压缩操作，得到压缩索引器 key $K^{\text{IComp}} \in \mathbb{R}^{\frac{n}{m} \times c^I}$ ，其中 c^I 是索引器头维度。随后，对于查询 token t ，我们以低秩方式生成索引器 query $\{\mathbf{q}_{t,1}^I; \mathbf{q}_{t,2}^I; \dots; \mathbf{q}_{t,n_h}^I\}$ ：

$$\mathbf{c}_t^Q = \mathbf{h}_t \cdot W^{DQ}, \quad (13)$$

$$[\mathbf{q}_{t,1}^I; \mathbf{q}_{t,2}^I; \dots; \mathbf{q}_{t,n_h^I}^I] = \mathbf{q}_t^I = \mathbf{c}_t^Q \cdot W^{IUQ}, \quad (14)$$

其中 $\mathbf{h}_t \in \mathbb{R}^d$ 是查询 token t 的输入隐藏状态； $\mathbf{c}_t^Q \in \mathbb{R}^{d_c}$ 是查询的压缩潜向量； d_c 表示查询压缩维度； n_h^I 表示索引器查询头的数量； $W^{DQ} \in \mathbb{R}^{d \times d_c}$ 和 $W^{IUQ} \in \mathbb{R}^{d_c \times c^I n_h^I}$ 分别是索引器 query 的下投影与上投影矩阵。接下来，查询 token t 与前序压缩块 ($s < \text{Floor}(\frac{t}{m})$) 之间的索引分数 $I_{t,s} \in \mathbb{R}$ 按如下方式计算：

$$[w_{t,1}^I; w_{t,2}^I; \dots; w_{t,n_h^I}^I] = \mathbf{w}_t^I = \mathbf{h}_t \cdot W^w, \quad (15)$$

$$I_{t,s} = \sum_{h=1}^{n_h^I} w_{t,h}^I \cdot \text{ReLU}(\mathbf{q}_{t,h}^I \cdot K_s^{\text{IComp}}), \quad (16)$$

其中 $W^w \in \mathbb{R}^{d \times n_h^I}$ 是可学习矩阵； $w_{t,h}^I \in \mathbb{R}$ 是第 h 个索引器头的权重。对于查询 token t ，给定其索引分数 $I_{t,:}$ ，我们采用一个 $\text{top-}\nabla \cdot \mathbf{k}$ 选择器，有选择地保留一部分压缩 KV 条目 C_t^{SprsComp} 用于后续核心注意力：

$$C_t^{\text{SprsComp}} = \{C_s^{\text{Comp}} \mid I_{t,s} \in \text{Top-k}(I_{t,:})\}. \quad (17)$$

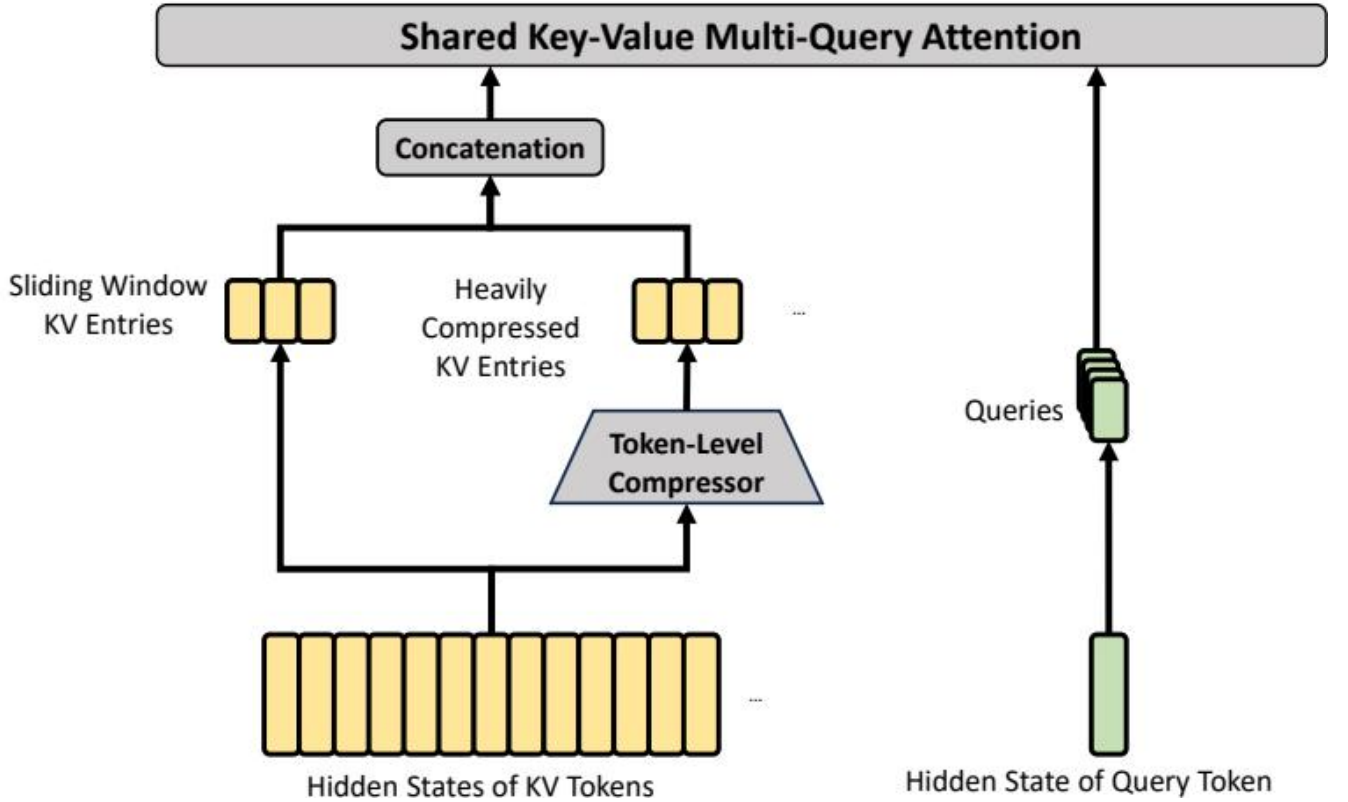


图 4 | HCA 的核心架构。它执行更强的压缩，其中 $m' (\gg m)$ 个 token 的 KV 条目会被合并为一个。此外，我们还额外引入了一小组滑动窗口 KV 条目，以增强局部细粒度依赖。

共享 Key-Value MQA。在选出稀疏 KV 条目之后，CSA 以 Multi-Query Attention (MQA) (Shazeer, 2019) 的方式执行核心注意力，其中 C_t^{SprsComp} 中的每个压缩 KV 条目都同时充当注意力 key 和 value。具体而言，对于查询 token t ，我们首先从压缩潜向量 $\mathbf{c}_t^Q$ 生成注意力 query $\{\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; \dots; \mathbf{q}_{t,n_h}\}$ ：

$$[\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; \dots; \mathbf{q}_{t,n_h}] = \mathbf{q}_t = \mathbf{c}_t^Q \cdot W^{UQ}, \quad (18)$$

其中 n_h 表示查询头数量； $W^{UQ} \in \mathbb{R}^{d_c \times cn_h}$ 是 query 的上投影矩阵。需要注意的是，潜在查询向量 $\mathbf{c}_t^Q$ 与索引器 query 所使用的潜在查询向量是共享的。接下来，我们在 $\{\mathbf{q}_{t,i}\}$ 与 C_t^{SprsComp} 上执行 MQA：

$$\mathbf{o}_{t,i} = \text{CoreAttn}(\text{query} = \mathbf{q}_{t,i}, \text{key} = C_t^{\text{SprsComp}}, \text{value} = C_t^{\text{SprsComp}}), \quad (19)$$

其中 $\mathbf{o}_{t,i} \in \mathbb{R}^c$ 是第 i 个头在第 t 个 token 处的核心注意力输出； $\text{CoreAttn}(\cdot)$ 表示核心注意力操作。

分组输出投影。在 DeepSeek-V4 的配置中， cn_h 相当大。因此，若将核心注意力操作的输出 $[\mathbf{o}_{t,1}; \mathbf{o}_{t,2}; \dots; \mathbf{o}_{t,n_h}] = \mathbf{o}_t \in \mathbb{R}^{cn_h}$ 直接投影到 d 维隐藏状态，将带来很大的计算负担。为降低这一成本，我们设计了分组输出投影策略。具体而言，我们先将 n_h 个输出划分为 g 组，然后对每组输出 $\mathbf{o}_{t,i}^G \in \mathbb{R}^{c \frac{n_h}{g}}$ 投影得到一个 d_g 维中间输出 $\mathbf{o}_{t,i}^{G'} \in \mathbb{R}^{d_g}$ ，其中 $d_g < c \frac{n_h}{g}$ 。最后，我们再将中间输出 $[\mathbf{o}_{t,1}^{G'}; \mathbf{o}_{t,2}^{G'}; \dots; \mathbf{o}_{t,g}^{G'}] \in \mathbb{R}^{d_g g}$ 投影为最终注意力输出 $\hat{\mathbf{o}}_t \in \mathbb{R}^d$ 。

2.3.2 重度压缩注意力

图 4 展示了 HCA 的核心架构，它以更激进的方式压缩 KV cache，但不使用稀疏注意力。

压缩 Key-Value 条目。总体而言，HCA 的压缩策略与 CSA 相似，但采用了更大的压缩率 $m' (\gg m)$ ，并且不进行重叠压缩。设 $H \in \mathbb{R}^{n \times d}$ 为输入隐藏状态序列，HCA 首先计算原始 KV 条目 $C \in \mathbb{R}^{n \times c}$ 及其对应的压缩权重 $Z \in \mathbb{R}^{n \times c}$ ：

$$C = H \cdot W^{KV}, \quad (20)$$

$$Z = H \cdot W^Z, \quad (21)$$

其中 W^{KV} 和 $W^Z \in \mathbb{R}^{d \times c}$ 是可训练参数。接下来， C 中每 m' 个 KV 条目会依据压缩权重及可学习的位置偏置 $B \in \mathbb{R}^{m' \times c}$ 被压缩成一个条目，生成 $C^{\text{Comp}} \in \mathbb{R}^{\frac{n}{m'} \times c}$ 。每个压缩条目 $C_i^{\text{Comp}} \in \mathbb{R}^c$ 按如下方式计算：

$$S_{m'i:m'(i+1)-1} = \text{Softmax}_{\text{row}}(Z_{m'i:m'(i+1)-1} + B), \quad (22)$$

$$C_i^{\text{Comp}} = \sum_{j=m'i}^{m'(i+1)-1} S_j \odot C_j. \quad (23)$$

通过这一压缩操作，HCA 将序列长度压缩到 $\frac{1}{m'}$ 倍。

共享 Key-Value MQA 与分组输出投影。HCA 同样采用与 CSA 相同的共享 KV MQA 与分组输出投影策略。在 KV 压缩之后，对于查询 token t ，HCA 首先以低秩方式生成注意力 query $\{\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; \dots; \mathbf{q}_{t,n_h}\}$ ：

$$\mathbf{c}_t^Q = \mathbf{h}_t \cdot W^{DQ}, \quad (24)$$

$$[\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; \dots; \mathbf{q}_{t,n_h}] = \mathbf{q}_t = \mathbf{c}_t^Q \cdot W^{UQ}, \quad (25)$$

其中 $\mathbf{h}_t \in \mathbb{R}^d$ 是查询 token t 的输入隐藏状态； n_h 表示查询头数量； $W^{DQ} \in \mathbb{R}^{d \times d_c}$ 和 $W^{UQ} \in \mathbb{R}^{d_c \times cn_h}$ 分别是 query 的下投影和上投影矩阵。接下来，我们在 $\{\mathbf{q}_{t,i}\}$ 和 C^{Comp} 上执行 MQA：

$$\mathbf{o}_{t,i} = \text{CoreAttn}(\text{query} = \mathbf{q}_{t,i}, \text{key} = C^{\text{Comp}}, \text{value} = C^{\text{Comp}}), \quad (26)$$

其中 $\mathbf{o}_{t,i} \in \mathbb{R}^c$ 是第 i 个头在第 t 个 token 处的核心注意力输出。接下来，与 CSA 一样，HCA 将 n_h 个输出划分为 g 组，并对每组输出 $\mathbf{o}_{t,i}^G \in \mathbb{R}^{c \frac{n_h}{g}}$ 投影得到一个 d_g 维中间输出 $\mathbf{o}_{t,i}^{G'} \in \mathbb{R}^{d_g}$ ，其中 $d_g < c \frac{n_h}{g}$ 。最后，HCA 将中间输出 $[\mathbf{o}_{t,1}^{G'}; \mathbf{o}_{t,2}^{G'}; \dots; \mathbf{o}_{t,g}^{G'}] \in \mathbb{R}^{d_g g}$ 投影为最终注意力输出 $\hat{\mathbf{o}}_t \in \mathbb{R}^d$ 。

2.3.3 其他细节

除了上述 CSA 和 HCA 的核心架构外，我们的混合注意力还融合了若干其他技术。为保证表述清晰，我们在前面的介绍中省略了这些附加技术，并将在本小节中作简要说明。此外，本小节只关注这些技术的核心思想，为了简洁起见可能会省略一些细小细节。我们鼓励读者参考我们的开源实现，以获得无歧义的细节说明。

查询与 Key-Value 条目归一化。对于 CSA 和 HCA，我们都会在核心注意力操作之前，对每个头上的 query 以及压缩 KV 条目唯一的头额外执行一次 RMSNorm 操作。这种归一化可以避免注意力 logit 爆炸，并可能提升训练稳定性。

部分旋转位置编码。对于 CSA 和 HCA，我们在注意力 query、KV 条目以及核心注意力输出上部分采用 Rotary Positional Embedding (RoPE) (Su et al., 2024)。具体而言，对于 CSA 和 HCA 中使用的每个 query 向量和 KV 条目向量，我们会对其最后 64 个维度应用 RoPE。由于 KV 条目同时充当注意力 key 和 value，朴素的核心注意力输出 $\{\mathbf{o}_{t,i}\}$ 会携带由 KV 条目加权求和导出的绝对位置编码。作为应对措施，我们还会在每个 $\mathbf{o}_{t,i}$ 的最后 64 个维度上，以位置 $-i$ 再应用一次 RoPE。这样一来，核心注意力的输出也会携带相对位置编码——每个 KV 条目对核心注意力输出的贡献也会与查询和该 KV 条目之间的距离相关。

额外的滑动窗口注意力分支。为了在 CSA 和 HCA 中严格保持因果性，每个 query 只会关注其之前的压缩 KV 块。因此，一个 query 无法访问与其处于同一压缩块中的其他 token 信息。与此同时，在语言建模中，最近的

token 往往与当前 query token 更为相关。基于这些原因，我们为 CSA 和 HCA 都引入了一个额外的滑动窗口式注意力分支，以更好地建模局部依赖。具体来说，对于每个 query token，我们还会额外生成 n_{win} 个未压缩的 KV 条目，对应最近的 n_{win} 个 token。在 CSA 和 HCA 的核心注意力中，这些滑动窗口内的 KV 条目将与压缩 KV 条目一起使用。

Attention Sink。在 CSA 和 HCA 的核心注意力中，我们采用了 attention sink 技巧 (OpenAI, 2025; Xiao et al., 2024)。具体而言，我们设置了一系列可学习的 sink logits $\{z'_1, z'_2, \dots, z'_{n_h}\}$ 。对于第 h 个注意力头， $\text{Exp}(z'_h)$ 会被加入到注意力分数分母中：

$$s_{h,i,j} = \frac{\text{Exp}(z_{h,i,j})}{\sum_k \text{Exp}(z_{h,i,k}) + \text{Exp}(z'_h)}, \quad (27)$$

其中 $s_{h,i,j}, z_{h,i,j} \in \mathbb{R}$ 表示第 h 个注意力头在第 i 个查询 token 与第 j 个前序 token 或压缩块之间的注意力分数和注意力 logit。该技术使得每个 query 头都可以将其总注意力分数调节为不必等于 1，甚至可以接近 0。

2.3.4 效率讨论

由于采用了混合 CSA 和 HCA，并结合低精度计算与存储，DeepSeek-V4 系列的注意力模块在注意力 FLOPs 和 KV cache 大小两方面都实现了显著效率提升，尤其是在长上下文场景中。首先，我们为 KV 条目采用混合存储格式：RoPE 维度使用 BF16 精度，其余维度使用 FP8 精度。与纯 BF16 存储相比，这种混合表示将 KV cache 大小几乎减半。其次，lightning indexer 内部的注意力计算采用 FP4 精度，这在极长上下文下加速了注意力操作。第三，相较 DeepSeek-V3.2，DeepSeek-V4 系列采用了更小的 attention top-k，从而提升了模型在短文本和中等长度文本上的效率。最后，也是最重要的一点，压缩注意力与混合注意力技术大幅降低了 KV cache 大小和计算 FLOPs。

以头维度为 128 的 BF16 GQA8 (Ainslie et al., 2023) 作为基线 — 这是 LLM 注意力的一种常见配置 — 在 1M 上下文设置下，DeepSeek-V4 系列的 KV cache 大小可显著降低至约为该基线的 2%。

算法 1 DeepSeek-V4 的 Muon 优化器

Require: Learning rate η , momentum μ , weight decay λ , update rescaling factor γ

```

1: for each training step  $t$  do
2:   for each logically independent weight  $W \in \mathbb{R}^{n \times m}$  do
3:      $G_t = \nabla_W \mathcal{L}_t(W_{t-1})$ 
4:      $M_t = \mu M_{t-1} + G_t$ 
5:      $O'_t = \text{Hybrid NewtonSchulz}(\mu M_t + G_t)$ 
6:      $O_t = O'_t \cdot \sqrt{\max(n, m)} \cdot \gamma$ 
7:      $W_t = W_{t-1} \cdot (1 - \eta\lambda) - \eta O_t$ 
8:   end for
9: end for

```

此外，即便与 DeepSeek-V3.2 (DeepSeek-AI, 2025) — 一个本已高效的基线 — 相比，DeepSeek-V4 系列依然展现出显著的效率优势。它们的推理 FLOPs 与 KV cache 大小对比见图 1 右侧。

2.4 Muon 优化器

由于收敛更快、训练稳定性更好，我们在 DeepSeek-V4 系列的大多数模块中采用 Muon (Jordan et al., 2024; Liu et al., 2025) 优化器。我们的 Muon 优化完整算法总结见算法 1。

基本配置。对于 embedding 模块、prediction head 模块、mHC 模块中的静态偏置与门控因子，以及所有 RMSNorm 模块的权重，我们继续使用 AdamW (Loshchilov and Hutter, 2017) 优化器。其余所有模块都使用 Muon 更新。遵循 Liu et al. (2025)，我们也会对 Muon 参数施加 weight decay，使用 Nesterov (Jordan et al., 2024; Nesterov, 1983) 技巧，并对更新矩阵的 Root Mean Square (RMS) 进行重缩放，以复用我们的 AdamW 超参数。与他们不同的是，我们采用 hybrid Newton-Schulz iterations 来进行正交化。

Hybrid Newton-Schulz 迭代。对于给定矩阵 M ，设其奇异值分解 (SVD) 为 $M = U\Sigma V^T$ 。Newton-Schulz 迭代的目标是将 M 近似正交化为 UV^T 。通常， M 会先被归一化为 $M_0 = M/\|M\|_F$ ，以确保其最大奇异值不超过 1。随后，每一步 Newton-Schulz 迭代执行如下操作：

$$M_k = aM_{k-1} + b(M_{k-1}M_{k-1}^T)M_{k-1} + c(M_{k-1}M_{k-1}^T)^2M_{k-1}. \quad (28)$$

我们的 hybrid Newton-Schulz 分两个阶段共执行 10 次迭代。在前 8 步中，我们使用系数 $(a, b, c) = (3.4445, -4.7750, 2.0315)$ 来推动快速收敛，使奇异值接近 1。在最后 2 步中，我们切换到系数 $(a, b, c) = (2, -1.5, 0.5)$ ，从而将奇异值精确稳定在 1。

避免注意力 logit 爆炸。DeepSeek-V4 系列的注意力架构使我们能够直接在注意力 query 和 KV 条目上应用 RMSNorm，这可以有效防止注意力 logit 爆炸。因此，我们在 Muon 优化器中不使用 QK-Clip 技术 (Liu et al., 2025)。

3 通用基础设施

3.1 专家并行中细粒度的通信-计算重叠

专家混合 (MoE) 可以通过专家并行 (EP) 加速。然而，EP 需要复杂的节点间通信，并对互连带宽和延迟提出了很高要求。为了缓解 EP 中的通信瓶颈，并在更低互连带宽要求下实现更高的端到端性能，我们提出了一种细粒度 EP 方案，将通信与计算融合进单个流水化 kernel 中，以实现通信-计算重叠。

通信延迟可以被隐藏。我们 EP 方案的关键洞见在于：在 MoE 层中，通信延迟可以有效隐藏在计算之下。如图 5 所示，在 DeepSeek-V4 系列中，每个 MoE 层主要可分解为四个阶段：两个受通信约束的阶段 Dispatch 和 Combine，以及两个受计算约束的阶段 Linear-1 和 Linear-2。我们的性能分析表明，在单个 MoE 层内，通信总时长小于计算总时长。因此，在将通信和计算融合为统一流水线之后，计算仍然是主要瓶颈，这意味着系统可以在不降低端到端性能的情况下容忍更低的互连带宽。

(a) 朴素方案



(b) Comet



(c) 我们的方法

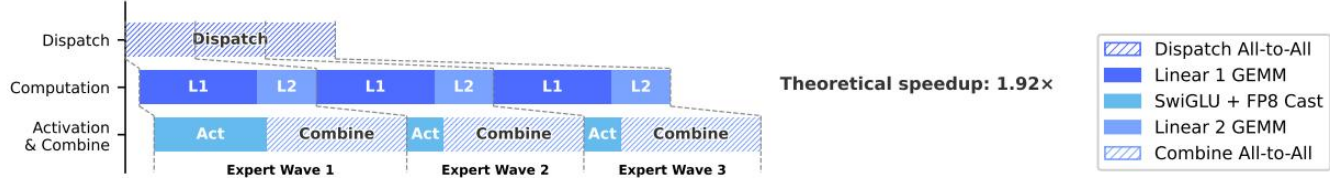


图 5 | 我们的 EP 方案及相关工作的示意图。Comet (Zhang et al., 2025b) 分别将 Dispatch 与 Linear-1、以及 Linear-2 与 Combine 进行重叠。我们的 EP 方案通过将专家切分并调度为多个 wave，实现了更细粒度的重叠。理论加速比是在 DeepSeek-V4-Flash 架构配置下评估的。

细粒度 EP 方案。为了进一步降低互连带宽需求并放大重叠带来的收益，我们引入了更细粒度的专家划分方案。受多项相关工作启发 (Aimuyo et al., 2025; Zhang et al., 2025b)，我们将专家切分并调度为多个 wave。每个 wave 由一小部分专家组成。一旦某个 wave 中的所有专家完成通信，计算就可以立刻开始，而无需等待其他专家。在稳态下，当前 wave 的计算、下一 wave 的 token 传输以及已完成专家的结果发送会并发进行，如图 5 所示。这在专家之间形成了细粒度流水线，使得计算和通信在整个 wave 期间都能持续进行。基于 wave 的调度也提升了极端场景下的性能，例如强化学习 (RL) rollout，这类场景通常会遇到长尾小批次。

性能与开源 Mega-Kernel。我们在 NVIDIA GPU 和 HUAWEI Ascend NPU 平台上验证了这一细粒度 EP 方案。与强大的非融合基线相比，它在一般推理工作负载上实现了 $1.50 \sim 1.73\times$ 的加速，在 RL rollout 和高速 agent 服务等延迟敏感场景下最高可达 $1.96\times$ 。我们已将这一基于 CUDA 的 mega-kernel 实现开源，名称为 MegaMoE2，并作为 DeepGEMM 的一个组件发布。

观察与建议。我们分享 kernel 开发中的观察与经验，并向硬件厂商提出一些建议，希望有助于高效硬件设计，并实现更好的软硬件协同设计：

- 计算-通信比。完全的通信-计算重叠取决于计算-通信比，而不只是带宽本身。记峰值计算吞吐为 C ，互连带宽为 B ，当 $C/B \leq V_{\text{comp}}/V_{\text{comm}}$ 时，通信可以被完全隐藏，其中 V_{comp} 表示计算量， V_{comm} 表示通信量。对于 DeepSeek-V4-Pro，每个 token-expert 对需要 $6hd$ FLOPs (SwiGLU 的 gate、up 和 down 投影)，但通信只需 $3h$ 字节 (FP8 Dispatch + BF16 Combine)，因此可简化为：

$$\frac{C}{B} \leq 2d = 6144\text{FLOPs/Byte}.$$

也就是说，每 1 GBps 的互连带宽足以隐藏 6.1 TFLOP/s 计算所对应的通信。一旦带宽达到这一阈值，它就不再是瓶颈，而继续投入更多硅面积来提升带宽的收益将迅速递减。我们鼓励未来硬件设计瞄准这样的平衡点，而不是无条件地扩展带宽。

- 功耗预算。极致的 kernel 融合会同时让计算、内存和网络处于高负载状态，因此功耗限频会成为关键性能限制因素。我们建议未来的硬件设计为这种完全并发的 workload 提供充足的功耗余量。
- 通信原语。我们采用了 pull-based 方法，即每个 GPU 主动从远端 GPU 读取数据，从而避免细粒度 push 所带来的高通知延迟。若未来硬件能提供更低延迟的跨 GPU 信号机制，push 将变得可行，并支持更自然的通信模式。
- 激活函数。我们建议用一种低成本的逐元素激活函数替换 SwiGLU，该函数不涉及指数或除法运算。这会直接减轻 GEMM 后处理开销；并且在相同参数预算下，移除 gate 投影还能扩大中间维度 d ，从而进一步放宽带宽要求。

3.2 使用 TileLang 进行灵活高效的 Kernel 开发

在实践中，我们精心设计的模型架构本会产生数百个细粒度的 Torch ATen 算子。我们采用 TileLang (Wang et al., 2026) 开发了一组融合 kernel，替代其中绝大多数算子，以最小开发成本获得最优性能。它也使我们在验证阶段快速原型化诸如注意力变体之类的算子。这些 kernel 在模型架构开发、大规模训练，以及最终推理服务的生产部署中都发挥着关键作用。作为一种领域专用语言 (DSL)，TileLang 在开发效率与运行时性能之间取得了平衡，既支持快速开发，也支持在同一代码库中进行深入、迭代式优化。此外，我们还与 TileLang 社区紧密协作，以推动更加敏捷、高效且稳定的 kernel 开发生态。

通过 Host Codegen 降低调用开销。随着加速器性能持续提升，CPU 侧编排开销变得愈发突出。对于小型但高度优化的 kernel，这类固定的 host 开销很容易限制利用率和吞吐。此类开销的一个常见来源是 host 端逻辑，例如运行时契约检查，通常为了灵活性而以 Python 编写，因此会引入固定的每次调用开销。

我们通过 Host Codegen 缓解这一问题，它将大部分 host 侧逻辑迁移到生成式 host 代码中。具体而言，我们首先在 IR (Intermediate Representation) 层面协同生成 device kernel 和轻量级 host launcher，并嵌入从语言前端解析出的必要元数据，例如数据类型、rank/shape 约束以及 stride/layout 假设。随后，该 launcher 会被 lowering 到基于 TVM-FFI (Chen et al., 2018) 框架构建的 host 源码中，其紧凑的调用约定和零拷贝 tensor 互操作共同将 host 侧开销降到最低。在运行时，这些生成出来的 host 代码负责执行校验与参数封送，从而把每次调用的检查逻辑完全移出 Python 执行路径。我们的测量表明，CPU 侧校验开销已从数十到数百微秒降至每次调用不足一微秒。

借助 SMT 求解器的形式化整数分析。TileLang kernel 涉及复杂的张量索引整数运算，这需要强大的形式化整数分析能力。在布局推断、内存 hazard 检测和边界分析等编译过程里，编译器必须验证整数表达式是否满足特定性质，才能启用相应优化。因此，更强的形式化分析能力能够释放更高级、更复杂的优化机会。

为此，我们将 Z3 SMT 求解器 (De Moura and Bjørner, 2008) 集成到 TileLang 的代数系统中，为张量程序中的大多数整数表达式提供形式化分析能力。我们通过将 TileLang 的整数表达式翻译为 Z3 的无量词非线性整数算术 (QF_NIA)，在计算开销与形式化表达能力之间取得平衡。基于整数线性规划 (ILP) 求解器，QF_NIA 可以无缝处理 kernel 中常见的标准线性整数表达式。此外，其内在的非线性推理能力也能有效应对诸如可变张量形状上的向量化等高级挑战。在合理资源限制下，Z3 能显著提升整体优化效果，同时将编译时间开销限制在几秒之内。其影响覆盖多个编译过程，包括向量化、barrier 插入和代码简化。

数值精度与按位可复现性。在生产环境中，数值正确性和可复现性与原始吞吐同样重要。因此，我们默认优先

保证精度：在编译器层面禁用 fast-math 优化，而影响精度的近似仅通过显式、可选启用的前端算子提供（例如 `T.__exp`、`T.__log` 和 `T.__sin`）。相反，当需要严格的 IEEE-754 语义时，TileLang 提供带显式舍入模式的 IEEE 兼容 intrinsic（例如 `T.ieee_fsqrt`、`T.ieee_fdiv` 和 `T.ieee_add`），使开发者能够精确指定数值行为。

我们还追求按位可复现，以便将 kernel 与手写 CUDA 基线进行校验。我们让 TileLang 的代数化简和 lowering 规则与主流 CUDA 工具链（例如 NVCC）保持一致，避免引入非预期的位级差异变换。布局标注（例如 `T.annotate_layout`）还允许用户固定依赖布局的 lowering 决策，使求值和累加顺序与参考 CUDA 实现保持一致，从而在需要时实现按位完全一致的输出。

我们的评估表明，这些面向精度和可复现性的设计选择并未牺牲性能：在保守默认设置下，TileLang kernel 仍然具有竞争力，同时也提供了可按需放宽数值约束以换取更高速度的调节开关。

3.3 高性能、批不变且确定性的 Kernel 库

为了实现高效训练与推理，我们开发了一套完整的高性能计算 kernel。除了提供基础功能和最大化硬件利用率之外，另一个关键设计目标是确保预训练、后训练与推理流程之间的训练可复现性与按位对齐。因此，我们以尽可能小的性能开销实现了端到端、按位批不变且确定性的 kernel。这些 kernel 有助于调试、稳定性分析以及保证后训练行为的一致性。

批不变性。批不变性保证任意给定词元的输出都在比特级保持一致，而不受其在一个 batch 中位置的影响。为了实现批不变性，主要挑战如下：

- 注意力。为了实现批不变性，我们不能使用 split-KV 方法（Dao et al., 2023）。该方法会将单个序列的注意力计算分配到多个流式多处理器（SM）上，以平衡各 SM 的负载。然而，放弃这一技术会导致严重的 wave-quantization 问题³，并可能对 GPU 利用率造成不利影响。为了解决这一问题，我们开发了面向批不变解码的双 kernel 策略。第一个 kernel 在单个 SM 内计算整个序列的注意力输出，以保证在 wave 被完全占满时具有高吞吐。第二个 kernel 则为了最小化最后一个部分填充 wave 的延迟、从而缓解 wave-quantization，使用多个 SM 共同处理单个序列。为了让这两个 kernel 在比特级上保持一致，我们仔细设计了第二个 kernel 的计算路径，确保其累加顺序与第一个 kernel 相同。此外，第二个 kernel 利用了 thread-block cluster 内的 distributed shared memory⁴，从而支持 SM 之间的高速数据交换。这种双 kernel 方法将批不变解码带来的开销有效控制到可以忽略不计。

- 矩阵乘法。传统的 cuBLAS 库（NVIDIA Corporation, 2024）无法实现批不变性。因此，我们用 DeepGEMM（Zhao et al., 2025）在端到端范围内替换了它。此外，在极小 batch size 下，常规实现通常会采用 split-k（Osama et al., 2023）技术以提升性能。不幸的是，split-k 技术无法保证批不变性，而这正是 DeepSeek-V4 的关键特性。

因此，我们在大多数场景下放弃了 split-k，但这可能带来性能下降。为此，我们引入了一系列优化，使我们的矩阵乘法实现能够在大多数主要场景中达到甚至超过标准 split-k 的性能。

确定性。确定性训练对于调试硬件或软件问题非常有帮助。此外，当训练中出现 loss spike 等异常时，确定性能帮助研究者更容易定位数值原因，并进一步改进模型设计。训练中的非确定性通常源于非确定性的累加顺序，而这往往是由 atomic add 指令的使用导致的。这个问题主要出现在反向传播阶段，尤其体现在以下部分：

- 注意力反向传播。在稀疏注意力反向传播的常规实现中，我们使用 `atomicAdd` 来为 KV 词元累加梯度。由于浮点加法不满足结合律，这会引入非确定性。为了解决这个问题，我们为每个 SM 分配独立的累加缓冲区，

随后再在所有缓冲区之上执行全局确定性求和。

- MoE 反向传播。当来自不同 rank 的多个 SM 并发向接收 rank 上的同一个缓冲区写数据时，写入位置的协商也会引入非确定性。为了解决这一问题，我们设计了单个 rank 内的词元顺序预处理机制，并结合跨多个 rank 的缓冲区隔离策略。这一策略确保了专家并行发送结果的确定性，以及 MoE 反向传播中累加顺序的确定性。
- mHC 中的矩阵乘法。mHC 包含一个输出维度仅为 24 的矩阵乘法。在极小 batch size 下，我们不得不使用 split-k (Osama et al., 2023) 算法，而其朴素实现会导致非确定性。为了解决这一问题，我们分别输出每个 split 部分，并在后续 kernel 中执行确定性归约，从而同时保持性能与确定性。

3.4 FP4 量化感知训练

为了在部署时实现推理加速和内存节省，我们在后训练阶段引入量化感知训练 (QAT) (Jacob et al., 2018)，使模型能够适应量化带来的精度退化。我们将 FP4 (MXFP4) 量化 (Rouhani et al., 2023) 应用到两个部分：(1) MoE 专家权重，这是 GPU 内存占用的主要来源之一 (OpenAI, 2025)；(2) CSA 中索引器的 Query-Key (QK) 路径，其中 QK 激活会完全以 FP4 进行缓存、加载和乘法运算，从而加速长上下文场景中的注意力分数计算。此外，在这一 QAT 过程中，我们还将索引分数 $I_{:,i}$ 从 FP32 进一步量化到 BF16。该优化使 top-k 选择器实现了 $2\times$ 加速，同时保持了 99.7% 的 KV 条目召回率。

对于 MoE 专家权重，遵循 QAT 的常见做法，优化器维护的 FP32 主权重会先量化到 FP4，再反量化回 FP8 进行计算。值得注意的是，我们的 FP4 到 FP8 反量化是无损的。这是因为，相比 FP4 (E2M1)，FP8 (E4M3) 额外拥有 2 个指数位，因此具有更大的动态范围。于是，只要每个 FP8 量化块 (128×128 tiles) 内 FP4 子块 (1×32 tiles) 的最大与最小缩放因子之比不超过某个阈值，细粒度的缩放信息就可以被 FP8 扩展后的动态范围完全吸收。我们通过实验验证了当前权重满足这一条件。这使得整个 QAT 流程能够在不做任何修改的情况下，完全复用现有的 FP8 训练框架。在反向传播中，梯度是相对于前向传播中使用的同一份 FP8 权重计算的，并直接回传给 FP32 主权重，这等价于在量化操作上应用直通估计器 (STE)。这也避免了对转置权重重新量化的需求。

在 RL 训练的推理与 rollout 阶段，由于不涉及反向传播，我们直接使用真实的 FP4 量化权重，而不是模拟量化。这确保了采样阶段的模型行为与在线部署完全一致，同时也减少了 kernel 的内存加载，从而带来真实加速，并显著降低内存消耗。我们对 CSA 索引器中的 QK 路径也采用了类似处理。

3.5 训练框架

我们的训练框架建立在为 DeepSeek-V3 (DeepSeek-AI, 2024) 开发的可扩展且高效的基础设施之上。在训练 DeepSeek-V4 时，我们继承了这一坚实基础，同时引入了若干关键创新，以适配其新的架构组件——具体包括 Muon 优化器、mHC 和混合注意力机制——同时保持高训练效率与稳定性。

3.5.1 Muon 的高效实现

Muon 优化器需要完整的梯度矩阵来计算参数更新，这在与零冗余优化器 (ZeRO) (Rajbhandari et al., 2020) 结合时带来了挑战。传统 ZeRO 是为 AdamW 这类逐元素优化器设计的，在这类优化器中，单个参数矩阵可以在多个 rank 之间切分并更新。为了解决这一冲突，我们为 Muon 设计了一种混合式 ZeRO bucket 分配策略。

对于稠密参数，我们限制 ZeRO 并行的最大规模，并采用背包算法将参数矩阵分配到这些 rank 上，以确保每个 rank 管理的负载大致均衡。每个 rank 上的 bucket 都会被填充到与各 rank 中最大 bucket 相同的大小，以便高效执行 reduce-scatter 操作。在我们的设置中，每个 rank 管理的参数矩阵不超过五个，因此这种填充通常带来的内存开销不到 10%。当数据并行的整体规模超过 ZeRO 的限制时，我们会在额外的数据并行组上冗余计算 Muon 更新，用计算换取更低的总 bucket 内存占用。

对于 MoE 参数，我们独立优化每个专家。我们首先将所有层中所有专家在 SwiGLU (Shazeer, 2020) 里的 down projection 矩阵全部展平，接着展平 up projection 矩阵和 gate 矩阵。随后，我们对展平后的向量进行填充，以确保可以在不切分任何逻辑独立矩阵的前提下，将该向量均匀分配到所有 rank 上。由于专家数量很多，我们不对 MoE 参数施加 ZeRO 并行规模限制，因此填充开销也可以忽略不计。

此外，在每个 rank 上，形状相同且相邻的参数会被自动合并，从而可以批量执行 Newton-Schulz 迭代，以获得更好的硬件利用率。进一步地，我们观察到 Muon 中的 Newton-Schulz 迭代在使用 BF16 矩阵乘法计算时依然稳定。基于这一点，我们进一步以随机舍入方式，将需要在数据并行 rank 间同步的 MoE 梯度量化到 BF16 精度，从而将通信量减半。为避免低精度加法器引入的累加误差，我们用两阶段方法替代传统的基于树或环的 reduce-scatter collective。首先，通过一次 all-to-all 操作在各 rank 间交换本地梯度；然后，每个 rank 在本地以 FP32 执行求和。这一设计保持了数值鲁棒性。

3.5.2 mHC 的低成本且内存高效实现

mHC 的引入相比传统残差连接增加了激活内存消耗以及流水线阶段之间的通信量。为了缓解这些成本，我们实现了若干优化策略。

首先，我们为训练和推理都精心设计并实现了 mHC 的融合 kernel。其次，我们引入了一种重计算策略，对中间张量进行选择性的 checkpoint。具体而言，我们会重算层间的大多数隐状态以及所有归一化后的层输入，同时避免重算计算密集型操作。这在内存节省与计算开销之间取得了平衡。第三，我们调整了 DualPipe1F1B 重叠方案，以适配增加后的流水线通信，并支持 mHC 中部分操作的并发执行。

综合来看，这些优化将 mHC 的墙钟时间开销限制到了重叠 1F1B 流水线阶段的 6.7%。更多工程优化细节可参见专门的 mHC 论文 (Xie et al., 2026)。

3.5.3 面向长上下文注意力的上下文并行

传统上下文并行 (CP) 会沿序列维度进行切分，每个 rank 维护连续的 s 个词元。这给我们的压缩注意力机制 (即 CSA 和 HCA) 带来了两个挑战。一方面，训练样本由多个序列打包而成，每个序列都会以 m (或 m') 为因子独立压缩，而末尾少于 m 的词元会被丢弃。因此，压缩后的 KV 长度通常小于 $\frac{s}{m}$ ，并且在不同 rank 之间会有所不同。另一方面，压缩过程需要连续的 m 个 KV 条目，而这可能会跨越相邻两个 CP rank 的边界。

为了解决这些挑战，我们设计了一种两阶段通信方法。在第一阶段，每个 rank i 将其最后 m 个未压缩 KV 条目发送给 rank $i + 1$ 。然后，rank $i + 1$ 将收到的部分条目与其本地的 s 个未压缩 KV 条目一起压缩，生成固定长度为 $\frac{s}{m} + 1$ 的压缩条目，其中会存在一些 padding 条目。在第二阶段，跨所有 CP rank 的 all-gather 操作会收集各自本地压缩得到的 KV 条目。随后，一个融合的 select-and-pad 算子会将它们重新组织成完整的压缩 KV 条目集合，总长度为 $\text{cp_size} \cdot \frac{s}{m}$ 。所有 padding 条目都会被放置在尾部。对于 HCA 和 CSA 中的索引器，每个

查询词元可见的压缩 KV 条目范围都可以通过规则预先计算。对于 CSA 中的稀疏注意力，top- k 选择器则会显式指定每个查询可见压缩 KV 条目的索引。

3.5.4 面向灵活激活检查点的扩展自动微分

传统的激活检查点实现以整个模块为粒度，决定是否在反向传播时保留或重算其输出激活。这种粗粒度方式往往会在重计算成本与激活内存占用之间产生次优折中。另一种做法是手动实现整层的前向和反向逻辑，并显式管理张量 checkpoint 状态。虽然这种方法能提供细粒度控制，但它失去了自动微分框架的便利性，从而显著增加了开发复杂度。

为了在不牺牲编程效率的前提下实现细粒度控制，我们实现了一种支持自动微分的张量级激活检查点机制。借助这一机制，开发者只需实现前向传播，并对单个张量选择性地添加标注，以便自动执行 checkpoint 与重计算。我们的框架利用 TorchFX (Reed et al., 2022) 追踪完整计算图。对于每个被标注的张量，它都会执行一次反向遍历，以识别其重计算所需的最小子图。我们将这些最小子图定义为重计算图，并在对应梯度计算之前将其插入到反向逻辑中。

与手工实现相比，这一设计在训练期间不会引入额外开销。该框架中的重计算是通过直接释放被标注张量的 GPU 内存，并复用重计算后张量的存储指针来实现的，无需任何 GPU 内存拷贝。此外，由于图追踪会具体执行模型，我们能够跟踪每个张量底层的存储指针，这使得对共享存储的张量（例如 reshape 操作的输入和输出）能够自动去重其重计算。这让开发者在标注重计算时无需再推理底层内存细节。

3.6 推理框架

我们的推理框架大体继承自 DeepSeek-V3，但在 KV Cache 管理上存在一些差异。

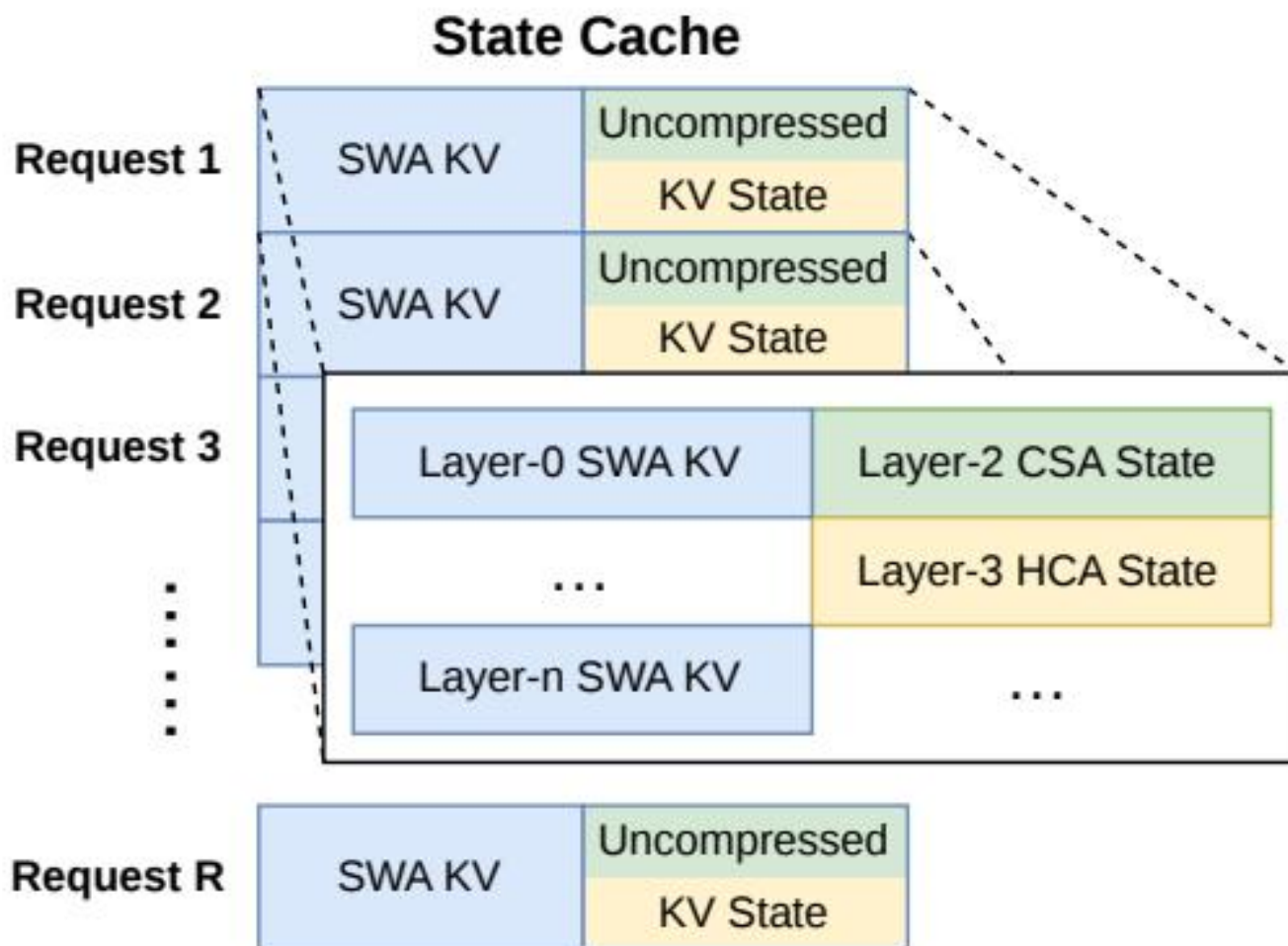
3.6.1 KV Cache 结构与管理

为了高效管理 DeepSeek-V4 中由混合注意力机制产生的异构 KV cache，我们设计了一种定制化的 KV cache 布局。该布局如图 6 所示，下面我们将详细介绍。

DeepSeek-V4 中的异构 KV 条目。DeepSeek-V4 系列中的混合注意力机制引入了多种类型的 KV 条目，它们具有不同的 Key-Value (KV) cache 大小和更新规则。用于稀疏选择的闪电索引器为 KV cache 引入了额外维度，其嵌入大小与主注意力中的嵌入大小不同。CSA 和 HCA 中采用的压缩技术会分别将序列长度缩减为原来的 $\frac{1}{m}$ 和 $\frac{1}{m'}$ ，从而减少整体 KV cache 大小。因此，不同层之间的 KV cache 大小并不相同。此外，滑动窗口注意力 (SWA) 层也具有不同的 KV cache 大小，并采用独立的 cache 命中与淘汰策略。在压缩分支中，每 m 个词元会生成一个 KV 条目。当剩余词元数量不足以执行压缩时，所有待处理词元及其对应的隐状态都必须暂存在缓冲区中，直到可以执行压缩操作为止。这些缓冲中的词元表示一种由位置上下文决定的序列状态，也被纳入 KV cache 框架进行管理。

混合注意力 KV Cache 管理的挑战。混合注意力机制违背了 PagedAttention 及其变体背后的一些基本假设。尽管近期的混合 KV cache 管理算法（例如 Jenga (Zhang et al., 2025a) 和 Hymba (Dong et al., 2025)）面向通用混合注意力模型或特定结构，但在 PagedAttention 框架下，将所有层的 KV cache 统一整合仍面临两个主要障碍：

- 多样化的 cache 策略，例如滑动窗口注意力中使用的策略。
- 高性能注意力 kernel 所施加的约束，包括对齐要求。



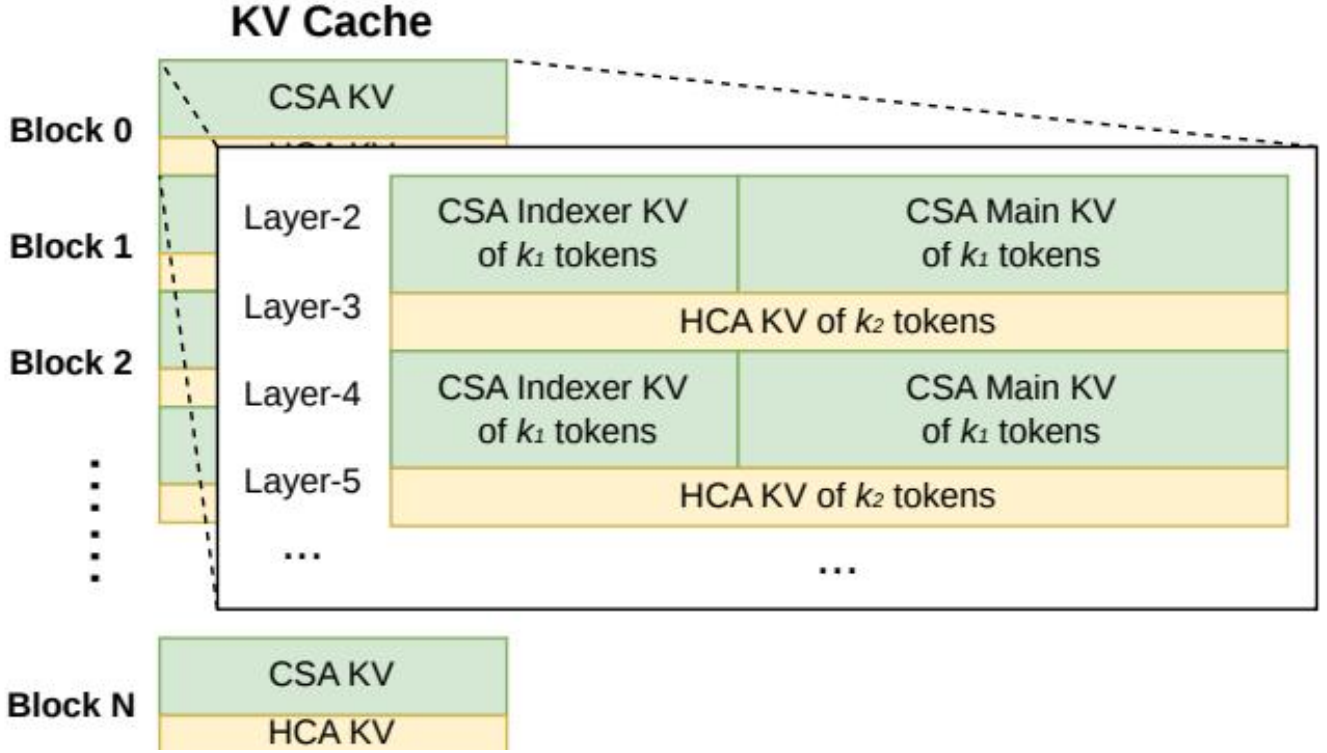


图 6 | DeepSeek-V4 的 KV cache 布局示意图。KV cache 被组织为两个主要部分：用于 CSA/HCA 的经典 KV cache，以及用于 SWA 和 CSA/HCA 中尚未满足压缩条件词元的状态 cache。在状态 cache 中，每个请求都会被分配一个固定大小的 cache block。在该 block 内，SWA 段存储最近 n_{win} 个词元对应的 KV 条目，而 CSA/HCA 段则存储尚未满足压缩条件的未压缩尾部状态。在经典 KV cache 中，我们为每个请求分配多个 block。每个 cache block 覆盖 $\text{lcm}(m, m')$ 个原始词元，并产生 $k_1 = \frac{\text{lcm}(m, m')}{m}$ 个 CSA 压缩词元，以及 $k_2 = \frac{\text{lcm}(m, m')}{m'}$ 个 HCA 压缩词元。

为了高效管理 DeepSeek-V4 的 KV cache，我们设计了相应策略来克服上述两个挑战。

用于 SWA 和未压缩尾部词元的状态 Cache。为了解决第一个障碍，我们采用了一种替代性的 cache 管理机制。由于 SWA 的设计目标是在受限 KV cache 大小下增强性能，因此，将它与压缩分支中未压缩的尾部词元一并视为一种状态空间模型是合理的。相应的 KV cache 因而可以被看作仅依赖当前位置的序列特定状态。据此，我们预先分配一个固定且有限大小的状态 cache 池，并将其动态分配给每条序列。

稀疏注意力 Kernel 协同设计。对于第二个障碍，传统高性能注意力 kernel 通常假设每个 block 具有固定数量 B 个词元以优化性能，这在 CSA 中对应 $B \cdot m$ 个原始词元，在 HCA 中对应 $B \cdot m'$ 个原始词元。通过使用高性能稀疏注意力 kernel，不同层能够在不损失性能的情况下支持可变的每块词元数。实现这一点需要对 KV cache 布局 and 稀疏注意力 kernel 进行协同设计。例如，将 block 填充到与 cache line 对齐可以提升性能。因此，对于压缩率为 m 的 CSA 和压缩率为 m' 的 HCA，每个 block 所含的原始词元数都可以是 $\text{lcm}(m, m')$ 的任意整数倍，也就是这两个压缩率的最小公倍数。

3.6.2 磁盘上的 KV Cache 存储

在服务 DeepSeek-V4 时，我们利用一种基于磁盘的 KV cache 存储机制，以消除共享前缀请求中的重复 prefilling。对于 CSA/HCA 中的压缩 KV 条目，以及滑动窗口注意力 (SWA) 中的未压缩 KV 条目，我们分别设计了不同的存储管理方案。

对于 CSA 和 HCA，我们直接将所有压缩 KV 条目存储到磁盘中。当某个请求命中已存储前缀时，我们会读取并复用该前缀对应的压缩 KV 条目，直到最后一个完整压缩 block。特别地，对于尾部不完整 block 中属于前缀的词元，我们仍然需要重新计算，以恢复未压缩 KV 条目，因为 CSA 和 HCA 中的未压缩 KV 条目并不会被存储。

对于 SWA 的 KV 条目，由于它们未被压缩且存在于每一层，其体积大约是压缩后 CSA 和 HCA KV 条目的 8 倍。为了高效处理这些庞大的 SWA KV 条目，我们提出并实现了三种不同的磁盘 SWA KV 条目管理策略，它们分别在存储开销与计算冗余之间提供不同的权衡：

- 完整 SWA 缓存。该策略为所有词元存储完整的 SWA KV 条目，从而实现计算零冗余。在这种策略下，命中前缀对应的 SWA KV 条目只需读取该前缀中最后 n_{win} 个词元的磁盘 cache 即可重建。尽管计算零冗余，这种策略对于现代基于 SSD 的存储系统并不高效——对于每个命中请求，只会访问所存储 SWA KV cache 的一小部分，从而形成一种不均衡、偏写密集访问模式。
- 周期性 Checkpoint。该策略每隔 p 个词元，对最近 n_{win} 个词元的 SWA KV 条目进行一次 checkpoint，其中 p 是可调参数。对于一个命中前缀，我们加载最近一次 checkpoint 的状态，然后重算剩余的尾部词元。通过调节 p ，该策略能够按需在存储与计算之间进行权衡。
- 零 SWA 缓存。该策略不存储任何 SWA KV 条目。对于命中前缀，我们需要执行更多重计算来恢复 SWA KV 条目。具体来说，在每个注意力层中，每个词元的 SWA KV 条目只依赖前一层中最近 n_{win} 个词元的 SWA KV 条目。因此，借助已缓存的 CSA 和 HCA KV 条目，只需重算最后 $n_{\text{win}} \cdot L$ 个词元，就足以为一个 L 层模型恢复最后 n_{win} 个 SWA KV 条目。

我们会根据具体部署场景选择最合适的策略，以实现期望的存储-计算权衡。

4 预训练

4.1 数据构建

在 DeepSeek-V3 的预训练数据基础上，我们致力于构建一个更加多样化、质量更高且具有更长有效上下文的训练语料。我们持续改进数据构建流水线。对于网页来源的数据，我们实现了过滤策略，以去除批量自动生成内容和模板化内容，从而降低模型塌缩的风险 (Zhu et al., 2024)。数学和编程语料仍然是我们训练数据的核心组成部分，我们还在中期训练阶段引入智能体式数据，以进一步增强 DeepSeek-V4 系列的代码能力。对于多语言数据，我们为 DeepSeek-V4 构建了更大的语料库，以提升其对不同文化中长尾知识的捕捉能力。对于 DeepSeek-V4，我们特别强调长文档数据整理，优先纳入科学论文、技术报告以及其他体现独特学术价值的材料。综合以上各类数据，我们的预训练语料包含超过 32T 个词元，涵盖数学内容、代码、网页、长文档以及其他高质量类别。

对于预训练数据，我们基本沿用了 DeepSeek-V3 的同类预处理策略。在分词方面，我们在 DeepSeek-V3 分词器

的基础上引入了少量用于上下文构造的特殊词元，同时仍将词表大小保持为 128K。我们还继承了 DeepSeek-V3 的 token-splitting (DeepSeek-AI, 2024) 和 Fill-in-Middle (FIM) (DeepSeek-AI, 2024) 策略。受 Ding et al. (2024) 启发，我们将来自不同来源的文档打包成合适的序列，以尽量减少样本截断。不同于 DeepSeek-V3，我们在预训练期间采用样本级 attention masking。

4.2 预训练设置

4.2.1 模型设置

DeepSeek-V4-Flash. 我们将 Transformer 层数设为 43，隐藏维度 d 设为 4096。前两层使用纯滑动窗口注意力。对于后续层，CSA 和 HCA 交替使用。对于 CSA，我们将压缩率 m 设为 4，将 indexer 查询头数 n_h^I 设为 64，将 indexer 头维度 c^I 设为 128，并将为稀疏注意力选取的 KV 条目数（即 attention top-k）设为 512。对于 HCA，我们将压缩率 m' 设为 128。对于 CSA 和 HCA，我们都将查询头数 n_h 设为 64，头维度 c 设为 512，查询压缩维度 d_c 设为 1024。输出投影分组数 g 设为 8，每个中间注意力输出的维度 d_g 设为 1024。对于滑动窗口注意力的附加分支，窗口大小 n_{win} 设为 128。我们在所有 Transformer 块中都采用 MoE 层，但前 3 个 MoE 层使用哈希路由策略。每个 MoE 层由 1 个共享专家和 256 个路由专家构成，其中每个专家的中间隐藏维度为 2048。在这些路由专家中，每个词元会激活 6 个专家。多词元预测深度设为 1。对于 mHC，扩展因子 n_{hc} 设为 4，Sinkhorn-Knopp 迭代次数 t_{max} 设为 20。在该配置下，DeepSeek-V4-Flash 总参数量为 284B，其中每个词元激活 13B 参数。

DeepSeek-V4-Pro. 我们将 Transformer 层数设为 61，隐藏维度 d 设为 7168。前两层使用 HCA。对于后续层，CSA 和 HCA 交替使用。对于 CSA，我们将压缩率 m 设为 4，将 indexer 查询头数 n_h^I 设为 64，将 indexer 头维度 c^I 设为 128，并将为稀疏注意力选取的 KV 条目数（即 attention top-k）设为 1024。对于 HCA，我们将压缩率 m' 设为 128。对于 CSA 和 HCA，我们都将查询头数 n_h 设为 128，头维度 c 设为 512，查询压缩维度 d_c 设为 1536。输出投影分组数 g 设为 16，每个中间注意力输出的维度 d_g 设为 1024。对于滑动窗口注意力的附加分支，窗口大小 n_{win} 设为 128。我们在所有 Transformer 块中都采用 MoE 层，但前 3 个 MoE 层使用哈希路由策略。每个 MoE 层由 1 个共享专家和 384 个路由专家构成，其中每个专家的中间隐藏维度为 3072。在这些路由专家中，每个词元会激活 6 个专家。多词元预测深度设为 1。对于 mHC，扩展因子 n_{hc} 设为 4，Sinkhorn-Knopp 迭代次数 t_{max} 设为 20。在该配置下，DeepSeek-V4-Pro 总参数量为 1.6T，其中每个词元激活 49B 参数。

4.2.2 训练设置

DeepSeek-V4-Flash. 我们对大多数参数使用 Muon 优化器 (Jordan et al., 2024; Liu et al., 2025)，但对嵌入模块、预测头模块和所有 RMSNorm 模块的权重使用 AdamW 优化器 (Loshchilov and Hutter, 2017)。对于 AdamW，我们将其超参数设为 $\beta_1 = 0.9$ 、 $\beta_2 = 0.95$ 、 $\varepsilon = 10^{-20}$ ，以及 weight_decay = 0.1。对于 Muon，我们将 momentum 设为 0.95，将 weight decay 设为 0.1，并将每个更新矩阵的 RMS 重缩放为 0.18，以复用 AdamW 的学习率。我们在 32T 个词元上训练 DeepSeek-V4-Flash，并且和 DeepSeek-V3 一样，也采用 batch size 调度策略：将 batch size（按词元计）从较小值逐步增加到 75.5M，随后在训练的大部分阶段维持在 75.5M。学习率在前 2000 步线性预热，并在训练的大部分阶段保持在 2.7×10^{-4} 。在训练接近尾声时，我们最终按照 cosine 调度将学习率衰减到 2.7×10^{-5} 。训练从 4K 的序列长度开始，并逐步将训练序列长度扩展到 16K、64K 和 1M。对于稀疏注意力的设置，我们首先在前 1T 个词元上使用稠密注意力对模型进行预热，并在序列长度达到 64K 时引入稀疏注意力，随后在训练的其余阶段持续使用稀疏注意力。在引入 attention sparsity 时，我们先设置一个较短阶段

来预热 CSA 中的 lightning indexer，然后在训练的大部分阶段使用稀疏注意力训练模型。对于无辅助损失的负载均衡，我们将 bias update speed 设为 0.001。对于平衡损失，我们将其损失权重设为 0.0001，以避免单个序列内部出现极端不均衡。MTP 损失权重在训练的大部分阶段设为 0.3，并在学习率衰减开始时调整为 0.1。

DeepSeek-V4-Pro. 除了少数超参数取值不同外，DeepSeek-V4-Pro 的训练设置与 DeepSeek-V4-Flash 基本一致。我们对大多数参数使用 Muon 优化器，但对嵌入模块、预测头模块和所有 RMSNorm 模块的权重使用 AdamW 优化器。AdamW 和 Muon 的超参数与 DeepSeek-V4-Flash 相同。我们在 33T 个词元上训练 DeepSeek-V4-Pro，并同样采用 batch size 调度策略，其中最大 batch size 为 94.4M 个词元。学习率调度策略与 DeepSeek-V4-Flash 基本相同，但峰值学习率设为 2.0×10^{-4} ，末尾学习率设为 2.0×10^{-5} 。训练同样从 4K 的序列长度开始，并逐步扩展到 16K、64K 和 1M。与 DeepSeek-V4-Flash 相比，DeepSeek-V4-Pro 的稠密注意力阶段更长，而引入稀疏注意力的策略与 DeepSeek-V4-Flash 相同，遵循两阶段训练方法。对于无辅助损失的负载均衡，我们将 bias update speed 设为 0.001。对于平衡损失，我们将其损失权重设为 0.0001，以避免单个序列内部出现极端不均衡。MTP 损失权重在训练的大部分阶段设为 0.3，并在学习率衰减开始时调整为 0.1。

4.2.3 缓解训练不稳定性

训练万亿参数级的 MoE 模型会带来显著的稳定性挑战，DeepSeek-V4 系列也不例外。我们在训练过程中遇到了明显的不稳定性问题。尽管简单的回滚可以暂时恢复训练状态，但它并不足以作为长期解决方案，因为它无法阻止 loss spike 再次发生。根据经验，我们发现 spike 的出现始终与 MoE 层中的离群值有关，而路由机制本身似乎还会加剧这些离群值的产生。因此，我们尝试从两个维度来解决这一问题：一是打破由路由诱发的恶性循环，二是直接抑制异常值。幸运的是，我们发现了两种在实践中非常有效的技术，能够维持训练稳定性。尽管我们目前仍未完全从理论上理解其底层机制，但我们公开分享这两种方法，以促进社区的进一步探索。

前瞻式路由. 我们发现，将骨干网络和路由网络的同步更新解耦，能够显著提升训练稳定性。因此，在 step t 时，我们使用当前网络参数 θ_t 进行特征计算，但路由索引则使用历史网络参数 $\theta_{t-\Delta t}$ 计算并应用。在实践中，为了避免两次加载模型参数的开销，我们会在 step $t - \Delta t$ 时提前获取 step t 的数据。我们会“前瞻式地”计算并缓存稍后在 step t 使用的路由索引，这也是我们将该方法命名为 Anticipatory Routing 的原因。我们还在基础设施层面对其进行了大量优化。首先，考虑到预计算路由索引只需要对数据做一次前向传播，我们精心编排了流水线执行，并对计算与 Expert Parallelism (EP) 通信进行了重叠，从而成功将 Anticipatory Routing 带来的额外实际运行时间开销限制在大约 20%。其次，我们引入了一种自动检测机制：仅当出现 loss spike 时，系统才会触发一次短暂回滚并启用 Anticipatory Routing；在该模式运行一段时间后，系统会恢复到标准训练。最终，这种动态应用方式使我们能够以几乎可忽略的额外总体训练开销避免 loss spike，同时不损害模型性能。

SwiGLU 截断. 在以往文献中 (Bello et al., 2017; Riviere et al., 2024)，clamping 已被明确用于约束数值范围，从而增强训练稳定性。在我们的实际训练中，我们通过经验发现，应用 SwiGLU clamping (OpenAI, 2025) 能够有效消除离群值，并在不损失性能的前提下显著帮助稳定训练过程。在 DeepSeek-V4-Flash 和 DeepSeek-V4-Pro 的整个训练过程中，我们将 SwiGLU 线性分量截断在 $[-10, 10]$ 范围内，同时将 gate 分量的上界限为 10。

4.3 评测

4.3.1 评测基准

在基座模型的评测中，我们考虑了四个关键维度的基准：世界知识、语言理解与推理、代码与数学，以及长上下文处理。

世界知识类基准包括 AGIEval (Zhong et al., 2023)、C-Eval (Huang et al., 2023)、CMMLU (Li et al., 2023)、MMLU (Hendrycks et al., 2020)、MMLU-Redux (Gema et al., 2024)、MMLU-Pro (Wang et al., 2024b)、MMMLU (OpenAI, 2024a)、MultiLoKo (Hupkes and Bogoychev, 2025)、Simple-QA verified (Haas et al., 2025)、SuperGPQA (Du et al., 2025)、FACTS Parametric (Cheng et al., 2025)，以及 TriviaQA (Joshi et al., 2017)。

语言理解与推理类基准包括 BigBench Hard (BBH) (Suzgun et al., 2022)、DROP (Dua et al., 2019)、HellaSwag (Zellers et al., 2019)、CLUEWSC (Xu et al., 2020)，以及 WinoGrande (Sakaguchi et al., 2019)。

代码与数学类基准包括 BigCodeBench (Zhuo et al., 2025)、HumanEval (Chen et al., 2021)、GSM8K (Cobbe et al., 2021)、MATH (Hendrycks et al., 2021)、MGSM (Shi et al., 2023)，以及 CMath (Wei et al., 2023)。

长上下文类基准包括 LongBench-V2 (Bai et al., 2025b)。

表 1 | DeepSeek-V3.2-Base、DeepSeek-V4-Flash-Base 和 DeepSeek-V4-Pro-Base 的对比。所有模型均在我们的内部框架下评测，并共享相同的评测设置。分差不超过 0.3 的分数被视为处于同一水平。每行最高分以粗体标出，第二高分以下划线标出。

	基准 (指标)	# 示例	DeepSeek-V3.2 Base	DeepSeek-V4-Flash Base	DeepSeek-V4-Pro Base
	架构	-	MoE	MoE	MoE
	# 激活参数量	-	37B	13B	49B
	# 总参数量	-	671B	284B	1.6T
世界知识	AGIEval (EM)	0-shot	80.1	82.6	83.1
	MMLU (EM)	5-shot	87.8	88.7	90.1
	MMLU-Redux (EM)	5-shot	87.5	89.4	90.8
	MMLU-Pro (EM)	5-shot	65.5	68.3	73.5
	MMMLU (EM)	5-shot	87.9	88.8	90.3
	C-Eval (EM)	5-shot	90.4	92.1	93.1
	CMMLU (EM)	5-shot	88.9	90.4	90.8
	MultiLoKo (EM)	5-shot	38.7	42.2	51.1
	Simple-QA verified (EM)	25-shot	28.3	30.1	55.2
	SuperGPQA (EM)	5-shot	45.0	46.5	53.9
	FACTS Parametric (EM)	25-shot	27.1	33.9	62.6
	TriviaQA (EM)	5-shot	83.3	82.8	85.6
	BBH (EM)	3-shot	87.6	86.9	87.5
	DROP (F1)	1-shot	88.2	88.6	88.7
	HellaSwag (EM)	0-shot	86.4	85.7	88.0
语言与推理	WinoGrande (EM)	0-shot	78.9	79.5	81.5

代码与数学	CLUEWSC (EM)	5-shot	83.5	82.2	85.2
	BigCodeBench (Pass@1)	3-shot	63.9	56.8	59.2
	HumanEval (Pass@1)	0-shot	62.8	69.5	76.8
	GSM8K (EM)	8-shot	91.1	90.8	92.6
	MATH (EM)	4-shot	60.5	57.4	64.5
	MGSM (EM)	8-shot	81.3	85.7	84.4
	CMath (EM)	3-shot	92.6	93.6	90.9
长上下文	LongBench-V2 (EM)	1-shot	40.2	44.7	51.5

4.3.2 评测结果

在表 1 中，我们给出了 DeepSeek-V3.2、DeepSeek-V4-Flash 和 DeepSeek-V4-Pro 三个基座模型的详细对比；所有结果都在统一的内部框架下、采用严格一致的设置进行评测。

将 DeepSeek-V4-Flash-Base 与 DeepSeek-V3.2-Base 对比，可以看到一个非常有说服力的效率提升案例。尽管其激活参数量和总参数量都显著更小，DeepSeek-V4-Flash-Base 仍在广泛的基准上超越了 DeepSeek-V3.2-Base。这一优势在世界知识任务和具有挑战性的长上下文场景中尤为明显。这些结果表明，DeepSeek-V4-Flash-Base 在架构改进、数据质量提升和训练优化上的进步，使其即便在更紧凑的参数预算下也能取得更优性能，并在大多数评测中有效超过规模更大的 DeepSeek-V3.2-Base。

此外，DeepSeek-V4-Pro-Base 还展示了进一步且决定性的能力跃升，几乎在所有方面都压过了 DeepSeek-V3.2-Base 和 DeepSeek-V4-Flash-Base。随着几乎所有类别上的提升，DeepSeek-V4-Pro-Base 在最具挑战性的基准上达到了 DeepSeek 基座模型中的新性能高点。在知识密集型评测上，它取得了显著增益，同时也大幅推进了长上下文理解能力。在大多数推理和代码基准上，DeepSeek-V4-Pro-Base 同样超过了前两个模型。这种全方位提升表明，DeepSeek-V4-Pro-Base 是 DeepSeek 系列中最强的基础模型，在知识、推理、代码和长上下文能力等各个维度上都优于其前代模型。

5 后训练

5.1 后训练流程

在预训练之后，我们开展了后训练阶段，以得到 DeepSeek-V4 系列的最终模型。尽管训练流程在很大程度上沿用了 DeepSeek-V3.2 的方案，但其中发生了一项关键的方法学替换：混合强化学习 (RL) 阶段被完全替换为在策略蒸馏 (On-Policy Distillation, OPD)。

5.1.1 专家训练

领域专家的开发是通过适配 DeepSeek-V3.2 的训练流程来完成的。具体而言，每个模型都依次经过初始微调阶段，以及在领域特定提示和奖励信号引导下进行的后续强化学习 (RL) 阶段。对于 RL 阶段，我们实现了 Group Relative Policy Optimization (GRPO) 算法，并保持超参数与我们此前研究中的设置高度一致 (DeepSeek-AI,2025; DeepSeek-AI, 2025)。

推理强度。众所周知，模型在推理任务上的性能从根本上受所投入计算量的支配。因此，我们在不同的 RL 配置下训练了不同的专家模型，以推动面向不同推理能力优化的模型开发。如表 2 所示，DeepSeek-V4-Pro 和 DeepSeek-V4-Flash 都支持三种特定的推理强度模式。对于每一种模式，我们在 RL 训练中施加不同的长度惩罚和上下文窗口，因此会产生不同的推理输出 token 长度。为了整合这些不同的推理模式，我们采用由 `<think>` 和 `</think>` token 界定的专门响应格式。此外，对于“Think Max”模式，我们会在系统提示开头添加一条特定指令，以引导模型的推理过程，如表 3 所示。

生成式奖励模型。通常，易于验证的任务可以通过简单的基于规则的验证器或测试用例得到有效优化。相比之下，难以验证的任务传统上依赖基于人类反馈的强化学习（RLHF），这需要大量人工标注来训练标量奖励模型。然而，在 DeepSeek-V4 系列的后训练阶段，我们不再使用这些传统的标量奖励模型。相反，为了处理难以验证的任务，我们构建了由评分准则引导的 RL 数据，并采用生成式奖励模型（GRM）来评估策略轨迹。关键在于，我们直接对 GRM 本身施加 RL 优化。在这一范式下，actor 网络天然地充当 GRM，从而使模型能够在其标准生成能力之外，同时联合优化其评估（判别）能力。通过统一这两种角色，模型内部的推理能力自然地融入了评估过程，从而带来了高度稳健的打分表现。此外，这种方法仅需极少量多样化的人类标注便可取得更优性能，因为模型能够利用自身逻辑在复杂任务之间进行泛化。

表 2 | 三种推理模式的比较

推理模式	特征	典型使用场景	响应格式
Non-think	基于习惯或简单规则的快速、直觉式响应。	日常例行任务、紧急反应、低风险决策。	<code></think></code> 总结
Think High	有意识的逻辑分析，更慢但更准确。	复杂问题求解、规划、中等风险决策。	<code><think></code> 思考 token
Think Max	将推理能力推向极限。速度较慢但更强大。	探索模型推理能力的边界。	1. 开头加入一条特殊指令

表 3 | 为“Think Max”模式注入到系统提示中的指令。

注入的指令

推理强度：绝对最大化，不允许任何捷径。

你必须非常彻底地思考，并全面拆解问题以定位根因，同时对你的逻辑在所有潜在路径、边界情况和对抗场景下进行严格压力测试。

请明确写出你的完整思考过程，记录每一个中间步骤、考虑过的替代方案以及被否决的假设，确保没有任何一个假设未经检验。

工具调用 Schema 与特殊 token。与上一版本一致，我们使用专门的 `<think></think>` 标签来界定推理路径。在 DeepSeek-V4 系列中，我们引入了一种新的工具调用 schema，它使用特殊的“`|DSML|`” token，并采用基于 XML 的工具调用格式，如表 4 所示。我们的实验表明，XML 格式能够有效缓解转义失败并减少工具调用错误，为模型与工具的交互提供更稳健的接口。

交错思考。DeepSeek-V3.2 引入了一种上下文管理策略：在工具结果往返轮次之间保留推理轨迹，但在新用户消息到来时将其丢弃。虽然这种方法有效，但在复杂的智能体工作流中仍会造成不必要的 token 浪费——每一轮新的用户输入都会清空累积的推理内容，迫使模型从头重建其问题求解状态。借助 DeepSeek-V4 系列扩展后的 1M-token 上下文窗口，我们进一步完善了这一机制，以最大化交错思考在智能体环境中的效果：

表 4 | DeepSeek-V4 系列的工具调用 schema。

工具调用 Schema

```
## Tools
你可以使用一组工具来帮助回答用户的问题。你可以通过写出如下所示的 "<|DSML|tool_calls>" 代码块来调用工具：
<|DSML|tool_calls>
<|DSML|invoke name="$TOOL_NAME">
<|DSML|parameter name="$PARAMETER_NAME" string="true|false">$PARAMETER_VALUE </|DSML|parameter>
...
</|DSML|invoke>
<|DSML|invoke name="$TOOL_NAME2">
...
</|DSML|invoke>
</|DSML|tool_calls>
字符串参数应按原样指定，并设置 'string="true"'。对于所有其他类型（数字、布尔值、数组、对象），请以 JSON 格式。
否则，请在 </think> 之后直接输出工具调用或最终响应。
## Available Tool Schemas
{Tool Definition...}
你必须严格遵循上面定义的工具名称和参数 schema 来调用工具。
```

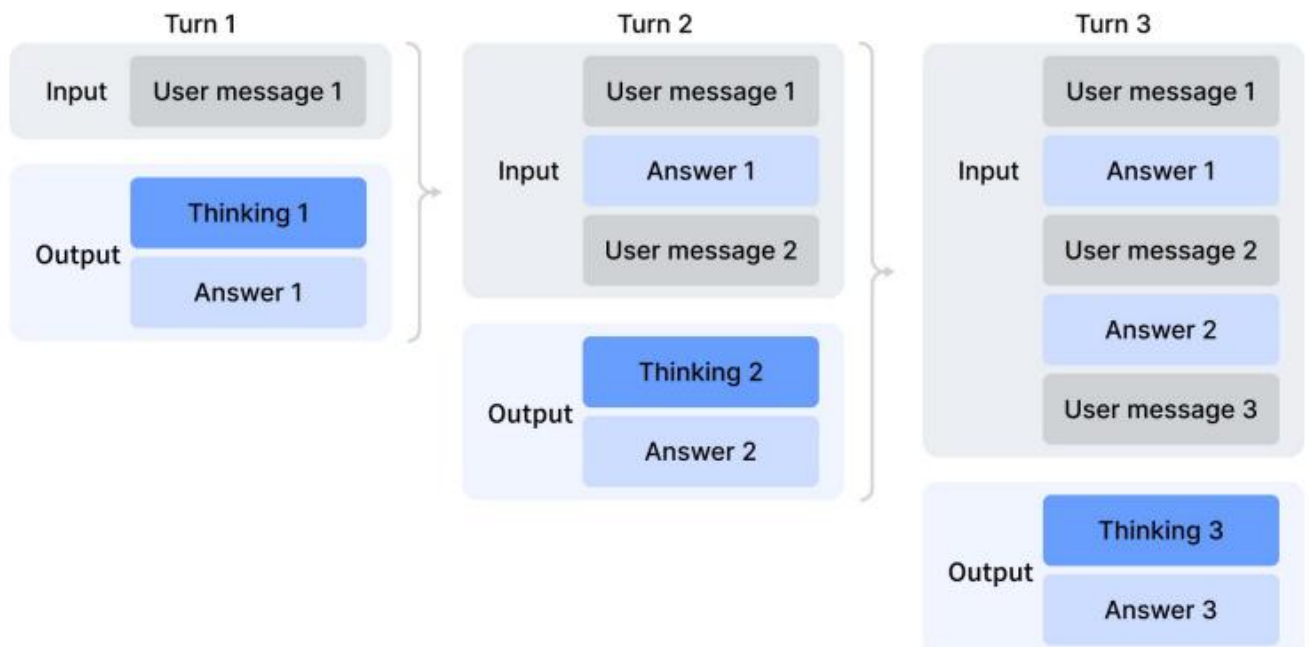
• 工具调用场景。如图 7(a) 所示，所有推理内容都会在整个对话过程中被完整保留。不同于 DeepSeek-V3.2 会在每个新的用户轮次到来时丢弃思考轨迹，DeepSeek-V4 系列会跨越所有轮次保留完整的推理历史，包括跨用户消息边界的部分。这使模型能够在长时程智能体任务中维持连贯且累积的思维链。

• 一般对话场景。如图 7(b) 所示，原有策略仍被保留：当新的用户消息到来时，上一轮的推理内容会被丢弃，从而在持久化推理轨迹收益有限的场景中保持上下文简洁。

与 DeepSeek-V3.2 一样，那些通过用户消息来模拟工具交互的智能体框架（例如 Terminus）可能不会触发工具调用上下文路径，因此也未必能从增强的推理持久化中受益。对于这类架构，我们仍然建议使用 non-think 模型。



a) 使用工具进行思考



b) 不使用工具进行思考

图 7 | DeepSeek-V4 系列的思考管理。

快速指令。在聊天机器人场景中，一系列辅助任务（例如判断是否要触发网页搜索、意图识别等）必须在生成响应之前执行。传统上，这些任务由一个单独的小模型处理，由于它无法复用已有的 KV cache，因此会带来冗余的预填充。为了解决这一限制，我们引入了 Quick Instruction。我们将一组专用特殊 token 直接附加到输入序列中，其中每个 token 对应一个特定的辅助任务。通过直接复用已计算好的 KV cache，这一机制完全避免了冗余预填充，并允许某些任务（如生成搜索查询以及判断权威性和领域）并行执行。因此，这种方法显著降低了用户感知到的首 token 时间（TTFT），并消除了维护和迭代额外小模型带来的工程开销。支持的 Quick Instruction token 总结见表 5。

5.1.2 在策略蒸馏

在通过专门微调和强化学习训练出多个领域特定专家之后，我们采用多教师在策略蒸馏（OPD）作为将专家能力合并进最终模型的主要技术。OPD 已成为一种有效的后训练范式，可高效地将领域专家的知识与能力迁移到单一统一模型中。这一过程通过让学生模型在其自身生成的轨迹上学习教师模型的输出分布来实现。形式化地，给定一组 N 个专家模型

表 5 | 用于辅助任务的 Quick Instruction 特殊 token。

特殊 Token	描述	格式
< action >	判断用户提示是否需要网页搜索，或者是否可以直接回答。	...< User > {prompt} < Assistant > <think> < a
< title >	在首次 assistant 响应后生成简洁的对话标题。	...< Assistant > {response} < end_of_sentence
< query >	为用户提示生成搜索查询。	...< User > {prompt} < query >
< authority >	对用户提示中对信息源权威性的需求进行分类。	...< User > {prompt} < authority >
< domain >	识别用户提示所属的领域。	...< User > {prompt} < domain >
< extracted_url >	判断用户提示中的每个 URL 是否应被抓取并读取。	...< User > {prompt} < extracted_url > {url} <

$\{\pi_{E_1}, \pi_{E_2}, \dots, \pi_{E_N}\}$ ，OPD 目标函数定义为：

$$\mathcal{L}_{\text{OPD}}(\theta) = \sum_{i=1}^N w_i \cdot D_{\text{KL}}(\pi_{\theta} \| \pi_{E_i}). \quad (29)$$

在这一表述中， w_i 表示分配给每个专家的权重，通常由该专家的相对重要性决定。计算反向 KL 损失 $D_{\text{KL}}(\pi_{\theta} \| \pi_{E_i})$ 需要从学生模型 π_{θ} 中采样训练轨迹，以保持策略学习。其底层逻辑是，统一策略 π_{θ} 会有选择地从与当前任务上下文相关的专门专家中学习（例如，在数学推理任务上对齐数学专家，在编程任务上对齐代码专家）。借助这一机制，物理上彼此独立的专家权重所蕴含的知识通过 logits 级对齐被整合进统一的参数空间，从而在实践中避免了传统权重合并或混合 RL 技术中经常出现的性能退化。在这一阶段，我们使用了十多个覆盖不同领域的教师模型来蒸馏一个单一学生模型。

在处理上述 OPD 目标时，以往工作通常会将全词表 KL 损失简化为每个 token 位置上的 token 级 KL 估计，

并通过将 $\text{sg}[\log \frac{\pi_{E_t}(y_t|x, y_{<t})}{\pi_{\theta}(y_t|x, y_{<t})}]$ (其中 sg 表示 stop gradient 操作) 作为策略损失计算中的逐 token advantage estimate, 来复用 RL 框架。尽管这种方法资源效率较高, 但会导致梯度估计方差很大, 并常常引发训练不稳定。因此, 我们在 OPD 中采用全词表 logit 蒸馏。在计算反向 KL 损失时保留完整的 logit 分布, 可以带来更稳定的梯度估计, 并确保对教师知识的忠实蒸馏。在下一小节中, 我们将介绍使全词表 OPD 能够大规模可行的工程工作。

5.2 RL 与 OPD 基础设施

我们的后训练基础设施建立在为 DeepSeek-V3.2 开发的可扩展框架之上。具体来说, 我们集成了第 3.5 节所描述的同分布训练栈, 以及前文引入的用于高效自回归采样的 rollout 引擎。在此基础上, 我们在本工作中引入了以下几项关键增强。这些设计使得涉及十多个不同教师模型的超长上下文 RL 与 OPD 合并任务可以高效执行, 从而显著加快模型发布的迭代周期。

5.2.1 FP4 量化集成

我们应用 FP4 (MXFP4) 量化来加速 rollouts 以及所有仅推理的前向过程, 包括教师模型和参考模型的前向过程, 从而降低内存流量和采样延迟。如第 3.4 节所述, 在 rollout 和推理阶段我们直接使用原生 FP4 权重。对于训练步骤, 则通过无损的 FP4-to-FP8 反量化步骤来模拟 FP4 量化, 从而无缝复用现有的 FP8 混合精度框架及其 FP32 主权重, 而无需修改反向传播流水线。

5.2.2 面向全词表 OPD 的高效教师调度

我们的框架支持全词表在策略蒸馏 (OPD), 可容纳实际上无上限数量的教师模型, 而每个教师模型都可能拥有万亿级参数。为实现这一点, 所有教师权重都会被卸载到集中式分布式存储中, 并在教师前向过程中按需加载, 同时采用类似 ZeRO 的参数分片, 以缓解 I/O 与 DRAM 压力。此外, 对于词表大小为 $|V| > 100k$ 的情形, 如果为所有教师模型直接物化 logits, 即使写入磁盘也是不可承受的。为此, 我们在前向过程中只将教师模型最后一层的隐藏状态缓存到集中式缓冲区中。在训练时, 再取回这些缓存状态, 并通过相应的 prediction head 模块在线重建完整 logits。该设计带来的重计算开销可以忽略不计, 同时彻底规避了显式 logits 物化带来的内存负担。为了缓解教师 prediction head 的 GPU 内存占用, 我们在数据分发时按教师索引对训练样本排序。这种安排确保每个不同的教师 head 在每个 mini-batch 中只需加载一次, 并且任意时刻设备内存中至多驻留一个教师 head。所有参数和隐藏状态的加载/卸载操作都在后台异步进行, 不会阻塞关键路径上的计算。最后, 教师与学生 logits 之间的精确 KL 散度由专门的 TileLang kernel 计算, 从而加速计算并抑制动态内存分配。

5.2.3 可抢占且容错的 Rollout 服务

为了最大化 GPU 资源利用率, 同时支持为高优先级任务快速调配硬件资源, 我们的 GPU 集群采用了集群级可抢占任务调度器, 任何运行中的任务都可能在任意时刻被抢占。此外, 在大规模 GPU 集群中, 硬件故障也十分常见。为此, 我们实现了一个面向 RL/OPD rollout 的可抢占且容错的大语言模型生成服务。

具体而言, 我们为每个生成请求实现了一个按 token 粒度记录的预写日志 (WAL)。每当某个请求生成出一个新 token, 我们就会立即将其追加到该请求的 WAL 中。在发生抢占时, 我们暂停推理引擎, 并保存未完成请求的 KV cache。

对于尚未完成的请求，在恢复时我们利用持久化保存的 WAL 和保存下来的 KV cache 继续解码。即使发生致命硬件错误，我们也可以利用 WAL 中持久化的 token 重新运行 prefill 阶段，以重建 KV cache。

需要强调的是，从头重新生成未完成请求在数学上是不正确的，因为这会引入长度偏差。由于较短的响应更有可能在中断中幸存，从头重新生成会使模型在发生中断时更倾向于产生更短的序列。如果推理栈在 batch 维度上不变且具有确定性，那么这一正确性问题也可以通过使用与采样器中伪随机数生成器一致的种子进行重新生成来解决。然而，这种方法仍然需要额外重新运行解码阶段，因此其效率远低于我们按 token 粒度的 WAL 方法。

5.2.4 面向百万 Token 上下文的 RL 框架扩展

我们针对百万 token 序列上的高效 RL 与 OPD 引入了定向优化。在 rollout 阶段，我们采用了第 5.2.3 节中详细介绍的可抢占且容错的 rollout 服务。对于推理和训练阶段，我们将 rollout 数据格式拆分为轻量级元数据和重量级逐 token 字段。在数据分发过程中，可以先加载整个 rollout 数据的元数据，以执行全局打乱和 packing 布局计算。重量级逐 token 字段则通过共享内存数据加载器加载，以消除节点内数据冗余，并在按 mini-batch 粒度消费后立即释放，从而显著降低 CPU 和 GPU 内存压力。设备端 mini-batch 的数量会根据工作负载动态确定，以在计算吞吐和 I/O 重叠之间实现高效权衡。

5.2.5 面向智能体 AI 的沙箱基础设施

为了满足后训练与评估期间智能体 AI 的多样化执行需求，我们构建了一个生产级沙箱平台 DeepSeek Elastic Compute (DSec)。DSec 由三个 Rust 组件构成——API 网关 (Apiserver)、每主机代理 (Edge) 以及集群监控器 (Watcher)——它们通过自定义 RPC 协议互联，并在 3FS 分布式文件系统之上水平扩展 (DeepSeek-AI, 2025)。在生产环境中，单个 DSec 集群可管理数十万个并发沙箱实例。

DSec 的设计源于四点观察：(1) 智能体工作负载高度异构，既包括轻量级函数调用，也包括完整的软件工程流水线，而且它们对操作系统和安全性的要求各不相同；(2) 环境镜像数量众多且体积庞大，但又必须能够快速加载并支持迭代定制；(3) 高密度部署要求高效利用 CPU 与内存；(4) 沙箱生命周期必须与 GPU 训练调度协同，包括抢占以及基于检查点的恢复。基于这些观察，我们在下文分别阐述 DSec 的四项核心设计。

统一接口背后的四种执行底座。DSec 提供单一的 Python SDK (libdsec)，用于抽象四种执行底座。Function Call 会将无状态调用分发到预热好的容器池中，从而消除冷启动开销。Container 完全兼容 Docker，并利用 EROFS (Gao et al., 2019) 的按需加载能力高效组装镜像。microVM 基于 Firecracker (Agache et al., 2020) 构建，为对安全敏感且高密度的部署增加了虚拟机级隔离。fullVM 基于 QEMU (Bellard, 2005) 构建，支持任意客户机操作系统。这四种执行底座共享统一的 API 接口——命令执行、文件传输和 TTY 访问——在它们之间切换只需修改一个参数。

通过分层存储实现快速镜像加载。DSec 通过分层按需加载机制，在快速启动与不断增长的大规模环境镜像库之间取得平衡。对于容器，基础镜像和文件系统提交会作为由 3FS 支撑的只读 EROFS 层存储，并直接挂载到 overlay lowerdirs 中。我们在挂载时使文件元数据可以直接从本地磁盘获取；与此同时，数据块则按需从 3FS 拉取。对于 microVM，DSec 使用 overlaybd (Li et al., 2020) 磁盘格式：只读基础层位于 3FS 上以供跨实例共享，而写入则进入本地 copy-on-write 层。此类快照可形成链式结构，从而支持高效版本管理和毫秒级恢复。

海量并发下的密度优化。为了让每个集群容纳数十万个沙箱，DSec 重点解决了两个资源瓶颈。首先，它缓解

了虚拟化环境中的重复 page-cache 占用，并通过内存回收来支持安全的超额分配。其次，它减轻了容器运行时中的 spinlock 争用，从而降低单沙箱的 CPU 开销，并显著提升单主机装箱密度。

轨迹日志与安全抢占恢复。DSec 为每个沙箱维护全局有序的轨迹日志，持久化记录每一次命令调用及其结果。该轨迹有三个用途：(1) 客户端快进——当训练任务被抢占时，沙箱资源仍然保留；恢复后，DSec 会重放此前已完成命令的缓存结果，从而加速任务恢复，并防止重新执行非幂等操作带来的错误；(2) 细粒度溯源——每次状态变化的来源及其对应结果都可追踪；(3) 确定性回放——任何历史会话都可以基于其轨迹被忠实复现。

5.3 标准基准评估

5.3.1 评估设置

知识与推理。知识与推理数据集包括 MMLU-Pro (Wang et al., 2024b)、GPQA (Rein et al., 2023)、Human Last Exam (Phan et al., 2025)、Simple-QA Verified (Haas et al., 2025)、Chinese-SimpleQA (He et al., 2024)、LiveCodeBench-v6 (Jain et al., 2024)、CodeForces (内部基准)、HMMT 2026 Feb、Apex (Balunović et al., 2025)、Apex Short-list (Balunović et al., 2025)、IMOAnswerBench (Luong et al., 2025) 以及 PutnamBench (Tsoukalaset al., 2024)。

对于代码任务，我们在 LiveCodeBench-v6 和一个内部 Codeforces 基准上评估 DeepSeek-V4 系列。对于 Codeforces，我们收集了 14 场 Codeforces Division 1 比赛，共包含 114 道题目 (2025 年 5 月至 2025 年 11 月)。Elo 评分的计算方式如下。对每场比赛，我们为每道题生成 32 个候选解。对每道题独立地，我们从这些解中无放回地抽样 10 个，并按随机顺序排列，形成提交序列。每次提交都由领域专家构建的测试集进行评判。对于成功解出的题目，其得分遵循 OpenAI (2025) 的惩罚规则：模型获得的是在人类参赛者中，解决同一题目且具有相同先前失败次数者所得分数的中位数。由此可得到每条采样提交序列的总比赛得分，再将其转换为比赛名次，并进一步依据标准 Codeforces 评级系统换算为估计 rating。比赛级期望 rating 定义为：对每题那 10 次提交的所有可能随机选择与排序下估计 rating 的期望。模型的总体 rating 则是这 14 场比赛中各比赛级期望 rating 的平均值。

对于推理与知识任务，我们分别将 Non-think、High 和 Max 模式的 temperature 设为 1.0，并将上下文窗口设为 8K、128K 和 384K tokens。对于数学任务 (如 HMMT、IMOAnswerBench、Apex 和 HLE)，我们使用如下模板进行评估：“{question}\n 请逐步推理，并将你的最终答案放在 \boxed{} 中。”对于数学任务上的 DeepSeek-V4-Pro-Max，我们使用如下模板来诱导更深层的推理：“请解决下面的问题。该问题可能要求你证明一个命题，也可能要求你给出一个答案。如果需要求出答案，你应当先得到该答案，并且你的最终解答还应当是对该答案有效性的严格证明。\n\n{question}”。

对于形式化数学任务，我们在 Lean v4.28.0-rc1 (Moura and Ullrich, 2021) 的智能体设置中进行评估，允许访问 Lean 编译器与语义 tactic 搜索引擎，并在最大推理强度下最多运行 500 次工具调用。此外，我们还评估了一条计算开销更大的流程：先生成候选自然语言解答，并通过自验证 (Shao et al., 2025) 进行筛选，然后将保留下来的解答作为指导提供给形式化智能体，用于证明对应的 Lean 陈述。这一设计利用非形式化推理提升探索能力，同时通过形式化验证保持严格正确性。只有当严格验证器 Comparator 在两种设置下都接受某个提交时，我们才将其记为正确。

对于 K2.6 和 GLM-5.1，我们留空了部分条目，因为它们的 API 过于繁忙，无法对我们的查询返回响应。

1M-Token 上下文。由于 DeepSeek-V4 系列支持 1M-token 上下文，我们选取 OpenAI MRCR (OpenAI, 2024b) 和 CorpusQA (Lu et al., 2026) 作为基准，在长上下文场景下评估模型性能。为了统一所有模型的配置，我们重新评估了 Claude Opus 4.6 和 Gemini 3.1Pro 在这些任务上的表现。我们没有评估 GPT-5.4，因为它的 API 对我们相当一部分查询都没有响应。

智能体。智能体数据集包括 Terminal Bench 2.0 (Merrill et al., 2026)、SWE-Verified (OpenAI,2024e)、SWE Multilingual (Yang et al., 2025)、SWE-Pro (Deng et al., 2025)、BrowseComp (Weiet al., 2025)、MCPAtlas (Bandi et al., 2026) 的公开评测集、GDPval-AA (AA, 2025;Patwardhan et al., 2025) 以及 Tool-Decathlon (Li et al., 2025)。

对于代码智能体任务 (SWE-Verified、Terminal-Bench、SWE-Pro、SWE Multilingual)，我们使用内部开发的评估框架来评估 DeepSeek-V4 系列。该框架提供一组最小工具——bash 工具和文件编辑工具。最大交互步数设为 500，最大上下文长度设为 512K tokens。对于 Terminal-Bench 2.0，我们注意到 GLM-5.1 所指出的环境相关问题。尽管如此，为保持一致性，我们仍报告在原始 Terminal-Bench 2.0 数据集上的性能。在 Terminal-Bench 2.0 Verified 子集上，DeepSeek-V4-Pro 取得了约 72.0 的得分。

对于搜索智能体任务 (BrowseComp、HLE w/ tool)，我们同样使用内部 harness，并提供 websearch 和 Python 工具，同时将最大交互步数设为 500，最大上下文长度设为 512K tokens。对于 BrowseComp，我们采用与 DeepSeek-V3.2 (DeepSeek-AI, 2025) 相同的 discard-all 上下文管理策略。

5.3.2 评估结果

表 6 | DeepSeek-V4-Pro-Max 与闭源/开源模型的比较。“Max”、“xHigh”和“High”表示推理强度。最佳结果以粗体标出，次优结果以下划线标出。

基准 (指标)		Opus-4.6 Max	GPT-5.4 xHigh	Gemini-3.1-Pro High	K2.6 Thinking	GLM-5.1
知识 & 推理	MMLU-Pro (EM)	89.1	87.5	91.0	87.1	86.0
	SimpleQA-Verified (Pass@1)	46.2	45.3	75.6	36.9	38.1
	Chinese-SimpleQA (Pass@1)	76.4	76.8	85.9	75.9	75.0
	GPQA Diamond (Pass@1)	91.3	93.0	94.3	90.5	86.2
	HLE (Pass@1)	40.0	39.8	44.4	36.4	34.7
	LiveCodeBench (Pass@1)	88.8	-	91.7	89.6	-
	Codeforces (Rating)	-	3168	3052	-	-
	HMMT 2026 Feb (Pass@1)	96.2	97.7	94.7	92.7	89.4
	IMOAnswerBench (Pass@1)	75.3	91.4	81.0	86.0	83.8
	Apex (Pass@1)	34.5	54.1	60.9	24.0	11.5
长上下文	Apex Shortlist (Pass@1)	85.9	78.1	89.1	75.5	72.4
	MRCR 1M (MMR)	92.9	-	76.3	-	-
	CorpusQA 1M (ACC)	71.7	-	53.8	-	-
	Terminal Bench 2.0 (Acc)	65.4	75.1	68.5	66.7	63.5
	SWE Verified (Resolved)	80.8	-	80.6	80.2	-
	SWE Pro (Resolved)	57.3	57.7	54.2	58.6	58.4
智能体	SWE Multilingual (Resolved)	77.5	-	-	76.7	73.3

BrowseComp (Pass@1)	83.7	82.7	85.9	83.2	79.3
HLE w/ tools (Pass@1)	53.1	52.0	51.6	54.0	50.4
GDPval-AA (Elo)	1619	1674	1314	1482	1535
MCPAtlas Public(Pass@1)	73.8	67.2	69.2	66.6	71.8
Toolathlon (Pass@1)	47.2	54.6	48.8	50.0	40.7

DeepSeek-V4-Pro-Max 与其他闭源/开源模型的比较见表 6。此外，我们还评估了 DeepSeek-V4-Flash 和 DeepSeek-V4-Pro 的不同模式，结果见表 7。

知识。在通用世界知识评估中，DeepSeek-V4-Pro-Max 作为 DeepSeek-V4-Pro 的最高推理强度模式，在开源大语言模型中创下了新的最先进水平。正如 SimpleQA-Verified 所展示的那样，DeepSeek-V4-Pro-Max 以 20 个绝对百分点的优势显著超过了现有所有开源基线。尽管取得了这些进展，它目前仍落后于领先的专有模型 Gemini-3.1-Pro。在教育知识与推理领域，DeepSeek-V4-Pro-Max 在 MMLU-Pro、GPQA 和 HLE 基准上略微优于 Kimi 和 GLM，但仍落后于领先的专有模型。总体而言，DeepSeek-V4-Pro-Max 在提升开源模型世界知识能力方面标志着一个重要里程碑。

此外，DeepSeek-V4-Flash 与 DeepSeek-V4-Pro 在知识类任务上存在显著性能差距；这是可以预期的，因为更大的参数规模有助于在预训练期间保留更多知识。值得注意的是，这两个模型在获得更高推理强度时，都在知识基准上呈现出更好的结果。

表 7 | DeepSeek-V4 系列不同规模与模式的比较。“Non-Think”、“High”和“Max”表示推理强度。

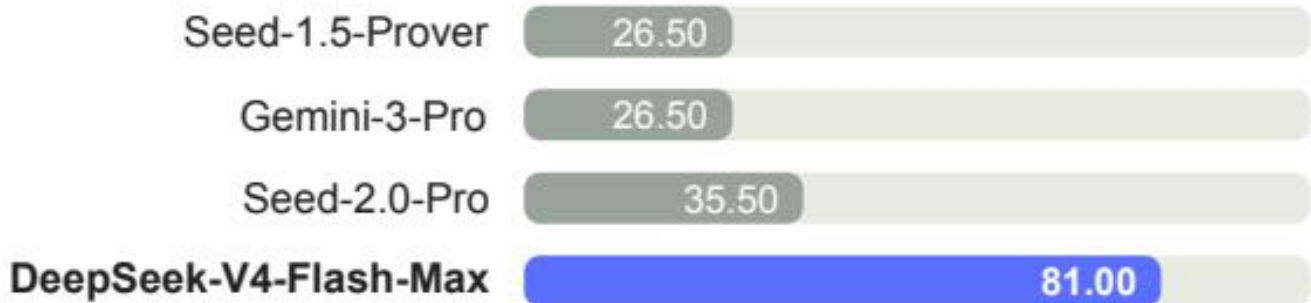
	基准 (指标)	DeepSeek-V4-Flash			DeepSeek-V4-Pro		
		Non-Think	High	Max	Non-Think	High	Max
知识 & 推理	MMLU-Pro (EM)	83.0	86.4	86.2	82.9	87.1	87.5
	SimpleQA-Verified (Pass@1)	23.1	28.9	34.1	45.0	46.2	57.9
	Chinese-SimpleQA (Pass@1)	71.5	73.2	78.9	75.8	77.7	84.4
	GPQA Diamond (Pass@1)	71.2	87.4	88.1	72.9	89.1	90.1
	HLE (Pass@1)	8.1	29.4	34.8	7.7	34.5	37.7
	LiveCodeBench (Pass@1-COT)	55.2	88.4	91.6	56.8	89.8	93.5
	Codeforces (Rating)	-	2816	3052	-	2919	3206
	HMMT 2026 Feb (Pass@1)	40.8	91.9	94.8	31.7	94.0	95.2
	IMOAnswerBench (Pass@1)	41.9	85.1	88.4	35.3	88.0	89.8
	Apex (Pass@1)	1.0	19.1	33.0	0.4	27.4	38.3
长上下文	Apex Shortlist (Pass@1)	9.3	72.1	85.7	9.2	85.5	90.2
	MRCCR 1M(MMR)	37.5	76.9	78.7	44.7	83.3	83.5
	CorpusQA 1M(ACC)	15.5	59.3	60.5	35.6	56.5	62.0
	Terminal Bench 2.0 (Acc)	49.1	56.6	56.9	59.1	63.3	67.9
	SWE Verified (Resolved)	73.7	78.6	79.0	73.6	79.4	80.6
	SWE Pro (Resolved)	49.1	52.3	52.6	52.1	54.4	55.4
智能体	SWE Multilingual (Resolved)	69.7	70.2	73.3	69.8	74.1	76.2

BrowseComp (Pass@1)	-	53.5	73.2	-	80.4	83.4
HLE w/ tools (Pass@1)	-	40.3	45.1	-	44.7	48.2
MCPAtlas Public (Pass@1)	64.0	67.4	69.0	69.4	74.2	73.6
GDPval-AA (Elo)	-	-	1395	-	-	1554
Toolathlon (Pass@1)	40.7	43.5	47.8	46.3	49.0	51.8

推理。DeepSeek-V4-Pro-Max 在各项推理基准上都优于以往所有开源模型，并在许多指标上追平了最先进的闭源模型；与此同时，规模更小的 DeepSeek-V4-Flash-Max 也在代码和数学推理任务上超过了此前最佳开源模型 K2.6-Thinking。同时，DeepSeek-V4-Pro 和 DeepSeek-V4-Flash 在编程竞赛中表现出色。根据我们的评估，它们的表现可与 GPT-5.4 相比肩，这是开源模型首次在该任务上追平闭源模型。在 Codeforces 排行榜上，DeepSeek-V4-Pro-Max 目前在人类参赛者中排名第 23。DeepSeek-V4 在形式化数学任务上也在智能体设置和高算力设置下表现强劲。在智能体设置下，它取得了最先进结果，如图 8 所示，超过了 Seed Prover (Chen et al., 2025) 等先前模型。在计算开销更大的流程下，其性能进一步提升，超过了 Aristotle(Achim et al., 2025) 等系统，并追平了这一设置下已知的最佳结果。

智能体。DeepSeek-V4 系列在评估中展现出强劲的智能体能力。对于代码智能体任务，DeepSeek-V4-Pro 取得了与 K2.6 和 GLM-5.1 可比的结果，尽管这些开源模型整体上仍落后于对应的闭源模型。DeepSeek-V4-Flash 在编码任务上的表现逊于 DeepSeek-V4-Pro，尤其是在 Terminal Bench 2.0 上。类似趋势也出现在其他智能体评估中。值得注意的是，DeepSeek-V4-Pro 在 MCPAtlas 和 Toolathlon 上表现良好——这两个评测集包含了广泛的工具和 MCP 服务——这表明我们的模型具有出色的泛化能力，而不仅仅是在内部框架上表现良好。

实用范式 Putnam-200 Pass@8，仅使用最少工具且采样受限。



前沿范式 Putnam-2025，采用形式化-非形式化混合推理并进行大规模算力扩展。



图 8 | 实用范式与前沿范式下的形式化推理。左图：Putnam-200 Pass@8 按照 Seed-Prover 提出的设置，从 PutnamBench (Tsoukalas et al., 2024) 中选取一个固定的随机子集进行评估；所有模型都在同一题目集上测试。我们遵循 Seed-Prover 协议，但将专有搜索工具替换为开源 LeanExplore (Asher,2025)，从而得到一个仅使用最少智能体工具且采样受限的轻量级设置。右图：Putnam-2025 探索了放大后的形式化-非形式化混合范式下数学推理的前沿，在该范式中，非形式化推理与形式化验证相结合，以暴露缺口并提升严谨性；DeepSeek-V4 达到了完美证明的 120/120。

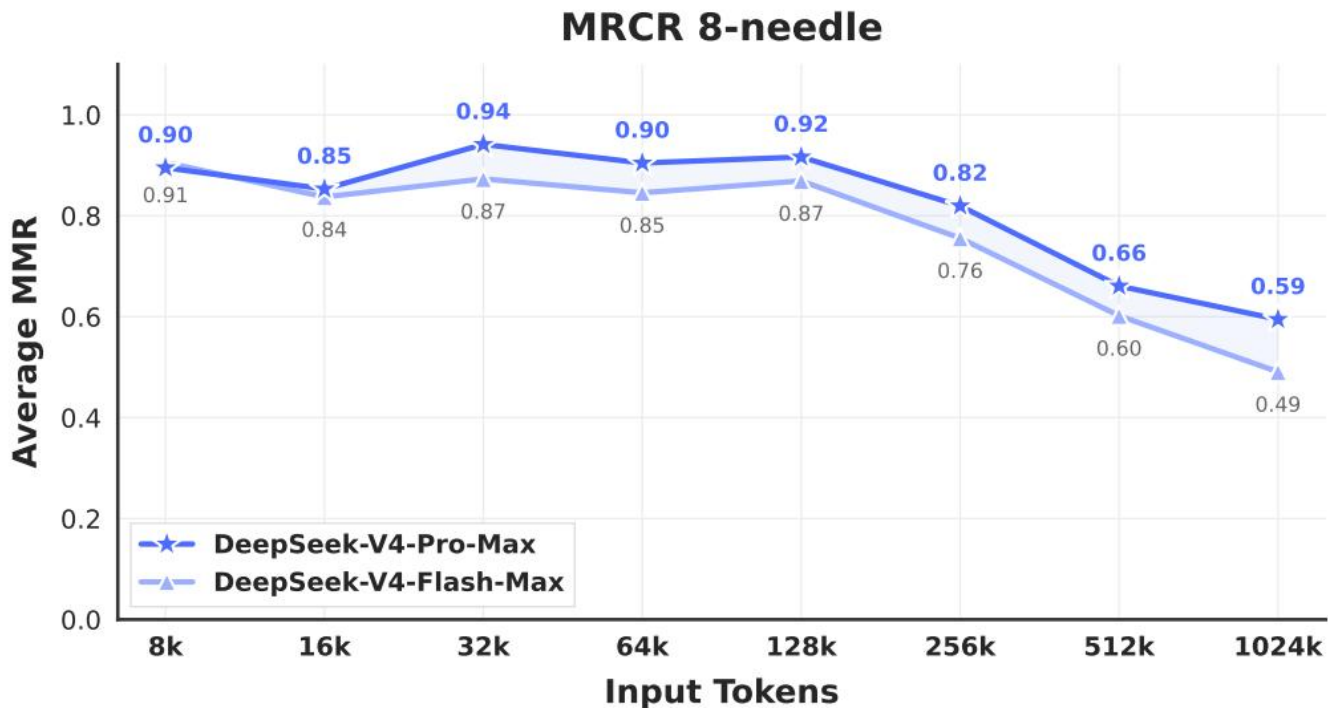
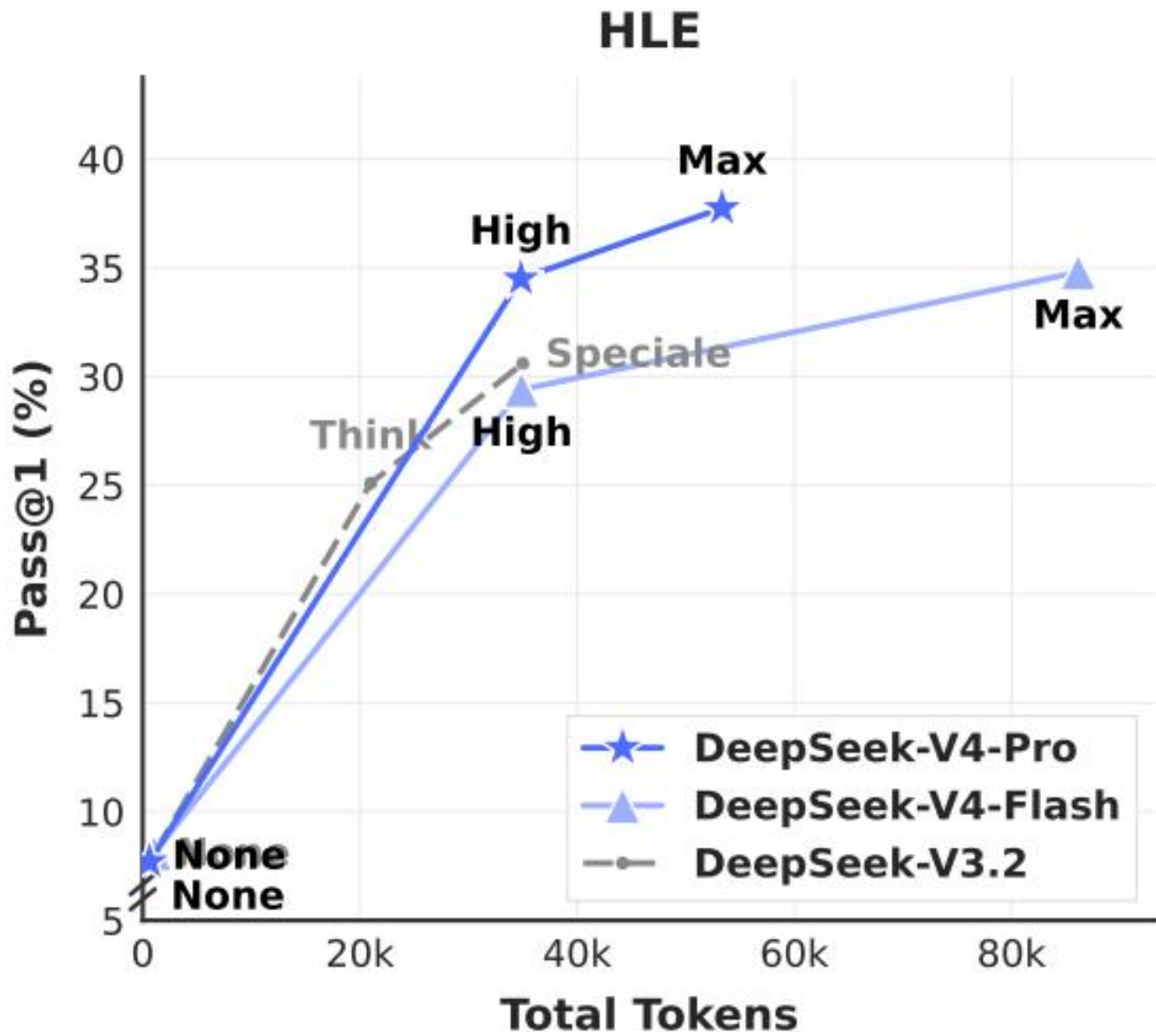


图 9 | DeepSeek-V4 系列在 MRCR 任务上的表现。

1M-Token 上下文。DeepSeek-V4-Pro 在衡量上下文内检索能力的 MRCR 任务上优于 Gemini-3.1-Pro，但仍落后于 Claude Opus 4.6。如图 9 所示，在 128K 上下文窗口内，检索性能保持高度稳定。尽管在超过 128K 之后性能下降开始变得可见，但与闭源和开源对手相比，该模型在 1M tokens 下的检索能力依然非常强劲。与 MRCR 不同，CorpusQA 更接近真实场景。评估结果也表明，DeepSeek-V4-Pro 优于 Gemini-3.1-Pro。

推理强度。如表 7 所示，Max 模式使用更长的上下文，并在 RL 中采用更小的长度惩罚，因此在最具挑战性

的任务上优于 High 模式。图 10 展示了 DeepSeek-V4-Pro、DeepSeek-V4-Flash 和 DeepSeek-V3.2 在代表性推理与智能体任务上的性能和成本对比。通过扩大测试时计算量，DeepSeek-V4 系列相较前代取得了显著提升。此外，在 HLE 这类推理任务上，DeepSeek-V4-Pro 表现出比



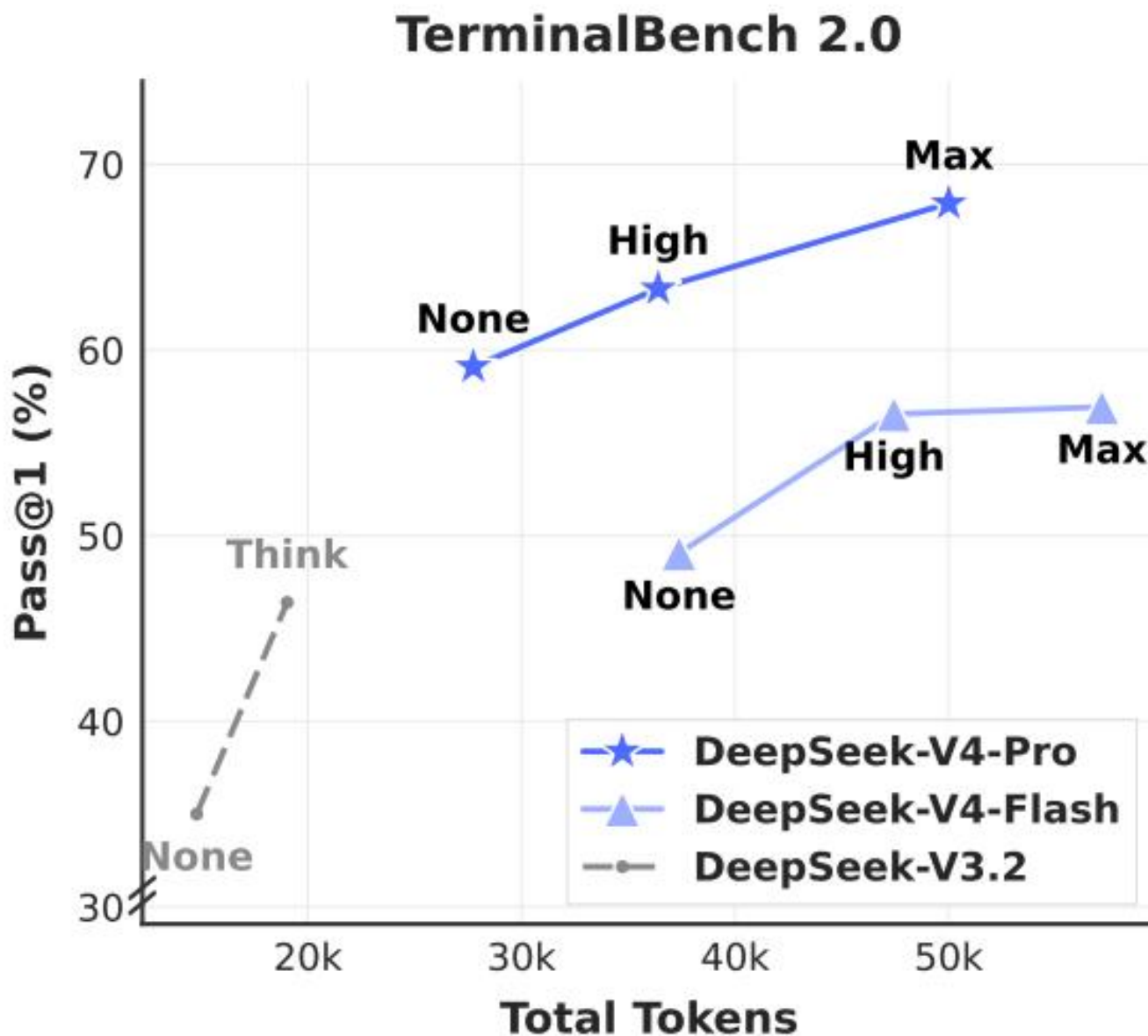


图 10 | 不同推理强度下 HLE 和 Terminal Bench 2.0 的表现。“None”表示 Non-think 模式，“Speciale”表示 DeepSeek-V3.2-Speciale 模型。

DeepSeek-V3.2 更高的 token 效率。

5.4 真实世界任务上的表现

标准化基准往往难以捕捉多样化真实世界任务的复杂性，从而在测试结果与实际用户体验之间形成落差。为了弥合这一差距，我们开发了专有的内部指标体系，相比传统基准更加优先关注真实使用模式。这种方法确保我们的优化能够转化为切实可感的收益。我们的评估框架专门针对 DeepSeek API 和 Chatbot 的主要使用场景，使模型性能与实际需求保持一致。

5.4.1 中文写作

中文写作是 DeepSeek 的主要使用场景之一。我们对功能性写作和创意写作进行了严格评估。表 12 展示了 DeepSeek-V4-Pro 与 Gemini-3.1-Pro 在功能性写作任务上的两两比较。这些任务由常见的日常写作请求构成，提示通常简洁直接。之所以选择 Gemini-3.1-Pro 作为基线，是因为它在我们的评估中是中文写作方面表现最好的外部模型。结果表明，DeepSeek-V4-Pro 以总体胜率 62.7% 对 34.1% 超过该基线；这主要是因为是在中文写作场景中，Gemini 有时会让其固有的风格偏好压过用户的明确要求。

表 13 展示了创意写作对比，其评估沿两个维度展开：指令遵循和写作质量。与 Gemini-3.1-Pro 相比，DeepSeek-V4-Pro 在指令遵循上的胜率为 60.0%，在写作质量上的胜率为 77.5%，表明它在指令遵循上有小幅提升，而在写作质量上则有显著增益。尽管 DeepSeek-V4-Pro 在整体用户案例分析中表现更优，但若将评估限制在最具挑战性的提示上——尤其是涉及高复杂度约束或多轮场景的提示——则 Claude Opus 4.5 仍然保持着相对于 DeepSeek-V4-Pro 的性能优势。如表 14 所示，Claude Opus 4.5 的胜率为 52.0%，而 DeepSeek-V4-Pro 为 45.9%。

5.4.2 搜索

搜索增强问答是 DeepSeek 聊天机器人的核心能力。在 DeepSeek web 与 app 中，“non-think”模式采用 Retrieval-Augmented Search (RAG)，而“thinking”模式则使用智能体式搜索。

检索增强搜索。我们在客观和主观两类 Q&A 任务上，对 DeepSeek-V4-Pro 与 DeepSeek-V3.2 进行了成对评估。如表 11 所示，DeepSeek-V4-Pro 以显著优势超过了 DeepSeek-V3.2，并在两类任务上都体现出一致领先。最显著的提升出现在单值搜索以及 planning & strategy 任务上，这表明 DeepSeek-V4-Pro 擅长从检索到的上下文中定位精确事实答案，并综合形成结构化方案。然而，DeepSeek-V3.2 在对比与推荐任务上仍然相对具有竞争力，这说明在需要基于搜索结果进行平衡、多视角推理的场景中，DeepSeek-V4-Pro 仍有改进空间。

智能体式搜索。与标准 RAG 不同，智能体式搜索使模型能够针对每个查询迭代调用搜索与抓取工具，从而显著提升整体搜索性能。对于 DeepSeek-Chat 中的 thinking 模式，我们优化了智能体式搜索功能，以在预定义的“thinking budget”内最大化响应准确性。如表 9 所示，智能体式搜索始终优于 RAG，尤其是在复杂任务上。此外，它的成本依然非常高效，智能体式搜索仅比标准 RAG 略贵一些（见表 10）。

5.4.3 白领任务

为了严格评估模型在复杂企业生产力场景中的实用性，我们构建了一套包含 30 个高级中文专业任务的综合评测集。这些工作流有意覆盖高层次认知需求，包括深入的信息分析、完整的文档生成和细致入微的文档编辑，并横跨 13 个关键行业（如金融、教育、法律和科技）。评估是在一个配备了基础工具（包括 Bash 和 web search）的内部智能体 harness 中完成的。

鉴于这些任务具有开放性，自动化指标通常难以捕捉高质量响应中的细微差别。因此，我们进行了人工评估，以比较 DeepSeek-V4-Pro-Max 与 Opus-4.6-Max 的表现。标注者在盲评条件下从四个维度对模型输出进行评估：

- 任务完成度：核心问题是否得到成功解决。
- 指令遵循：是否遵守特定约束与指令。
- 内容质量：事实准确性、逻辑连贯性与专业语气。

- 格式美观度：版式可读性和视觉呈现。

如图 11 所示，DeepSeek-V4-Pro-Max 在多样化的中文白领任务上优于 Opus-4.6-Max，取得了令人印象深刻的不败率 63%，并在分析、生成和编辑任务上都展现出稳定优势。图 12 所示的详细维度得分凸显了该模型在任务完成度和

内容质量方面的主要优势。具体来说，DeepSeek-V4-Pro-Max 会主动预判用户的隐含意图，经常提供补充洞见和自我验证步骤。它在长篇生成方面也表现突出，能够产出深入、连贯的叙述，而不是像 Opus-4.6-Max 那样经常依赖过于简单的项目符号列表。此外，该模型严格遵循正式的专业规范，例如标准化的中文层级编号。然而，在指令遵循方面，它偶尔会忽视特定格式约束，略微落后于 Opus。进一步地，该模型在将大量文本输入压缩为简洁摘要方面还不够擅长。最后，其格式美观度在演示文稿整体视觉设计方面仍有相当大的提升空间。图 13、14 和 15 展示了一些测试案例；由于某些输出过长，这里仅展示部分页面。



图 11 | 分析、生成、编辑任务及整体表现上的胜率比较。

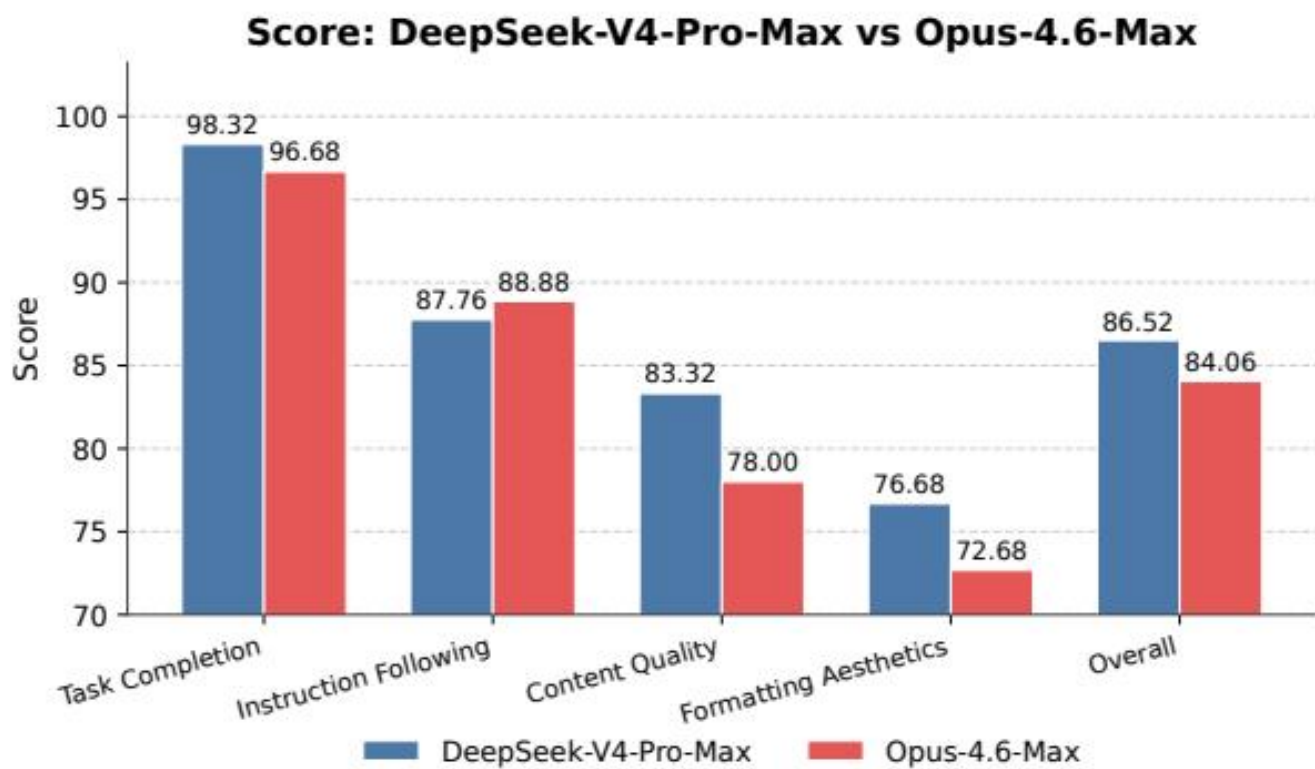


图 12 | 详细维度得分，包括任务完成度、内容质量、格式美观度和指令遵循。

SOCIAL AMPLIFICATION & UGC STRATEGY

构建「触点-参与-分享-激励-再参与」的完整社交传播闭环，让每个地铁站成为内容生产引擎。

传播效果预估（保守测算）

◎ 雜誌特刊

▲ 设计说明：本案为某公司设计，设计主题为“企业文化”，设计内容为公司VI设计，包括标志、标准字、标准色、辅助图形等。设计过程中，设计师首先进行了市场调研，了解了公司的行业背景、企业文化、竞争对手等情况。在此基础上，设计师结合公司的实际情况，提出了设计思路，并进行了多次方案汇报和修改。最终，设计师完成了标志、标准字、标准色、辅助图形等设计，并制作了VI设计手册，为公司VI系统的实施提供了指导。

② 剂量参与

地址：中国 北京市 西城区 西便门大街 54 号 邮编：100045 电话：010-66555777 传真：010-66555778

③ 同时激励

黃世龍以他與王基幹、鄧力、鄧浩林等關係密切，正合犯罪嫌疑人的身份，故應予撤銷。

④ 社交分享

内容来源: 知乎@陈鹤琴(华东师范大学教授)——《为什么孩子要读绘本》

⑤ 二次裂變

UGO对品牌出友好型→「FOMO效应」驱动品牌广告主站到了卡一循环狂型，声量持续被放大

单站日均互动参与 基于移动端站内跨行业互动活动参与1-2%与活动转化率	80-350人次
单站月均UGC产出 小红书+抖音+朋友圈合计，自然内容营销占比约40%	200-500条
单站次全网曝光量 含UGC自然传播+品牌官方内容+达人合作内容量级	300-800万次
全年累计UGC数量 四季路品牌站内容，Q1-Q4品牌官方+内容品牌传播	3,500-3,000万条
核心话题阅读量 *霸王茶姬北京首店 #CHANG科齿减脂季 双话题单篇累计	500-3,000万
互动转化率 从互动参与数/单帖单篇阅读量/站内容曝光量计算	8-15%

Page 18 / 25

ANNUAL MARKETING CALENDAR OVERVIEW

30站全年轮转逻辑 | 旗舰站复用策略 | 空白区域补充机制

站名	换乘线路	换乘方式	站名	换乘线路	换乘方式
南屏堡站	10、19	同台换乘	南屏大学站	10、19	同台换乘
奥林匹克公园站	8、19	同台换乘	工人体育场站	8、19	同台换乘
星火/朝霞站	5、19	同台换乘	团结湖站	5、19	同台换乘
四惠站	6、19	同台换乘	双井站	6、19	同台换乘
王府井站	7、19	同台换乘	巴沟站	7、19	同台换乘
国家图书馆站	9、19	同台换乘	宋家庄站(10号线)	9、19	同台换乘
榆门站	10、19	同台换乘	高家园站	10、19	同台换乘
珍珠大桥站	10、19	同台换乘	呼家楼站	10、19	同台换乘
三元桥站	10、19	同台换乘	姚家园站	10、19	同台换乘
四季青站	10、19	同台换乘	东坝站	10、19	同台换乘
东直门站	2、19	同台换乘	东直门站	2、19	同台换乘
西便门站	2、19	同台换乘	西便门站	2、19	同台换乘
李台站	2、19	同台换乘	李台站	2、19	同台换乘
金台遗址站	2、19	同台换乘	金台遗址站	2、19	同台换乘
海子站(南)	2、19	同台换乘	海子站(北)	2、19	同台换乘
海子站(北)	2、19	同台换乘	海子站(南)	2、19	同台换乘
宋家庄站(亦庄线)	2、19	同台换乘	宋家庄站(地铁)	2、19	同台换乘
七里庄站	2、19	同台换乘	七里庄站	2、19	同台换乘
安华桥站	2、19	同台换乘	安华桥站	2、19	同台换乘
慈溪站	2、19	同台换乘	慈溪站	2、19	同台换乘

此處僅供參考

- 展览形式：(2次/年)：商榷雕塑、园林艺术为主，主材料、漆画等——均为为景观或大型站点，在2个波段中出展空间主题。提供视觉、思想内容完全独特信息，确保“隔一站，完全不认识环境”。
- 年度展览单元应用：星火计划期：(208m²)——全年最大空问仅在画廊体现其价值和文化主题的展览一次，保持稀缺性与神秘感
- 展示频率使用原则：17日：根据展览主题与题材安排展览频次，避免过度消费与浪费者审美疲劳
- 单位联动应用：七位点、安海路、石庄、东家江沿线、湾子小学等以每年年度主题展览频次或重要节日邀请嘉宾未纳入主力合作，可作为机动补充完成年度品牌活动

Page 22 / 25

ANNUAL MARKETING CALENDAR OVERVIEW

四季四波段，12个月全覆盖，30站轮转激活，全年品牌声量无空窗

月度活动强度

1月 高热度 新春开屏·品牌宣传 3站	2月 高热度 元宵团圆·亲子互动 3站	3月 中热度 早春焕新·助力消费 3站	4月 中热度 清明踏青·城市漫步 3站
5月 高热度 五一黄金周·消费盛宴 3站	6月 高热度 端午飘香·非遗展示 3站	7月 高热度 暑假狂欢·文化体验 3站	8月 高热度 七夕浪漫·中秋预热 3站
9月 高热度 中秋团圆·金秋品鉴 3站	10月 中热度 国庆献礼·感恩回馈 3站	11月 中热度 暖冬关怀·社区关怀 3站	12月 高热度 岁末收官·跨年预热 3站

Page 21 / 25

QUARTERLY INVESTMENT BREAKDOWN & RESOURCE ALLOCATION

基于「标准配置档」500万基准方案的四季度资金与资源分配模型

季度预算拆解（基准方案-年度500万）

项目	Q1 春季启动	Q2 夏季预热	Q3 秋季爆发	Q4 冬季总结	全年 总计	占比
站点部署	10	35	38	42	145	29%
品牌搭建	48	52	55	60	215	43%
互动装置	12	15	15	13	55	11%
人员运营	8	10	10	12	40	8%
主题车棚	28	0	30	28	86	—
灯箱广告	10	12	12	15	49	—
创意设计	15	10	10	10	45	9%
合计(万元)	151	134	170	180	635	—

- 以上为《标准合同图》基础方案(合同总价暂按总造价10%比例保留的总造价2500万,不含增值税)
- Q4时房屋处于预售阶段, 预售合同年度房屋事件发生(例如:一森林花园公园竣工交房, 面积约25-35亩)
- 完成房屋交付验收后, 土地开发权, 房屋开发权或承租权(如Q4中收入), Q3-Q4房屋交付内容(如交付面积)
- 房屋开发权Q4时房屋交付时(如交付方式)交付, 达到标准(通常标准)约15-20%, 其中已交付房屋比例
- 主要交付内容(25亩)房屋交付时(如交付方式)交付, 达到标准(通常标准)约15-20%, 其中已交付房屋比例

Page 24 / 25

图 13 | 某项任务的示例输出：该任务要求为一个热门奶茶品牌与北京地铁撰写联合营销方案。

5.4.4 代码智能体

为了对我们的代码智能体能力进行基准评测，我们从真实的内部研发工作负载中整理任务。我们从 50+ 名内部工程师那里收集了 ~200 个具有挑战性的任务，覆盖功能开发、缺陷修复、重构和诊断，并横跨包括 PyTorch、CUDA、Rust 和 C++ 在内的多种技术栈。每个任务都附带其原始代码仓库、对应的执行环境以及人工标注的评分准则；经过严格的质量筛选后，最终保留 30 个任务作为评测集。如表 8 所示，DeepSeek-V4-Pro 显著优于 Claude Sonnet 4.5，并接近 Claude Opus 4.5 的水平。

表 8 | 研发代码基准对比（外部模型仅为评估目的纳入）。

模型	Haiku 4.5	Sonnet 4.5	DeepSeek-V4-Pro-Max	Opus 4.5	Opus 4.5 Thinking	Opus 4.6 Thinking
通过率 (%)	13	47	67	70	73	80

在一项针对 DeepSeek 开发者和研究员 ($N = 85$) 的调查中——这些受访者都在日常工作中使用过 DeepSeek-V4-Pro 进行智能体式编程——我们询问他们：与其他前沿模型相比，DeepSeek-V4-Pro 是否已准备好成为其默认且主要的代码模型。结果显示，52% 的受访者回答“是”，39% 倾向于“是”，而回答“否”的不足 9%。受访者认为 DeepSeek-V4-Pro 在大多数任务上都能给出令人满意的结果，但也指出它会出现琐碎错误、误解模糊提示，以及偶尔过度思考。

6 结论、局限性与未来方向

在这项工作中，我们发布了 DeepSeek-V4 系列的预览版本，目标是构建能够打破超长上下文处理效率瓶颈的下一代大语言模型。通过结合融合 CSA 与 HCA 的混合注意力架构，DeepSeek-V4 系列在长序列效率上实现了显著跃升。架构创新与大规模基础设施优化相结合，使模型能够高效地原生支持百万 token 上下文，并为未来的测试时扩展、长时程任务以及在线学习等新兴范式奠定了必要基础。评测结果表明，DeepSeek-V4-Pro-Max 作为 DeepSeek-V4-Pro 的最高推理强度模式，重新定义了开放模型的最先进水平。它在知识类基准上大幅超越以往开源模型，推理性能逼近前沿专有模型，并展现出具有竞争力的智能体能力。与此同时，DeepSeek-V4-Flash-Max 在保持极高成本效率架构的同时，达到了与领先闭源模型相当的推理性能。我们相信，DeepSeek-V4 系列开启了开放模型百万级上下文的新纪元，并为更高的效率、规模与智能铺平了道路。

为了追求极致的长上下文效率，DeepSeek-V4 系列采用了大胆的架构设计。为尽可能降低风险，我们保留了许多经过初步验证的组件和技巧；它们虽然有效，却也使整体架构相对复杂。在未来的迭代中，我们将开展更全面、更具原则性的研究，提炼出架构中最本质的设计，使其在不牺牲性能的前提下更加简洁优雅。与此同时，尽管 Anticipatory Routing 和 SwiGLU Clamping 已被证明能够有效缓解训练不稳定性，但其底层原理仍未得到充分理解。我们将积极研究训练稳定性的基础问题，并加强内部指标监控，以期形成一种更有原则、也更具预测性的稳定大规模训练方法。

此外，除 MoE 与稀疏注意力架构之外，我们还将积极探索新的模型稀疏化维度 — 例如更稀疏的嵌入模块 (Cheng et al., 2026) — 以在不损害能力的前提下进一步提升计算与内存效率。我们也将持续研究低延迟架构与系统技术，使长上下文部署与交互更加敏捷。进一步地，我们认识到长时程、多轮智能体任务的重要性与实际价值，并将继续在这一方向上迭代探索。我们也在推进将多模态能力融入模型。最后，我们致力于开发更好的数据整理与合成策略，以在日益广泛的场景与任务中持续提升模型的智能性、鲁棒性与实际可用性。

References

- AA. Gdpval-aa leaderboard, 2025. URL <https://artificialanalysis.ai/methodology/intelligence-benchmarking#gdpval-aa>.
- T. Achim, A. Best, A. Bietti, K. Der, M. Fédérico, S. Gukov, D. Halpern-Leistner, K. Henningsgard, Y. Kudryashov, A. Meiburg, et al. Aristotle: Imo-level automated theorem proving. arXiv preprint arXiv:2510.01346, 2025.
- A. Agache, M. Brooker, A. Florescu, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa. Firecracker: lightweight virtualization for serverless applications. In Proceedings of the 17th Usenix Conference on Networked Systems Design and Implementation, NSDI'20, page 419–434, USA, 2020. USENIX Association. ISBN 9781939133137.
- O. J. Aimuyo, B. Oh, and R. Singh. Flashmoe: Fast distributed moe in a single kernel. Advances in Neural Information Processing Systems, 2025. URL <https://neurips.cc/virtual/2025/poster/119124>.
- J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. arXiv preprint arXiv:2305.13245, 2023.
- J. Asher. LeanExplore: A search engine for Lean 4 declarations, 2025. URL <https://arxiv.org/abs/2506.11085>.
- Y. Bai, Y. Bao, G. Chen, J. Chen, N. Chen, R. Chen, Y. Chen, Y. Chen, Y. Chen, Z. Chen, J. Cui, H. Ding, M. Dong, A. Du, C. Du, D. Du, Y. Du, Y. Fan, Y. Feng, K. Fu, B. Gao, H. Gao, P. Gao, T. Gao, X. Gu, L. Guan, H. Guo, J. Guo, H. Hu, X. Hao, T. He, W. He, W. He, C. Hong, Y. Hu, Z. Hu, W. Huang, Z. Huang, Z. Huang, T. Jiang, Z. Jiang, X. Jin, Y. Kang, G. Lai, C. Li, F. Li, H. Li, M. Li, W. Li, Y. Li, Y. Li, Z. Li, Z. Li, H. Lin, X. Lin, Z. Lin, C. Liu, C. Liu, H. Liu, J. Liu, J. Liu, L. Liu, S. Liu, T. Y. Liu, T. Liu, W. Liu, Y. Liu, Y. Liu, Y. Liu, Y. Liu, Z. Liu, E. Lu, L. Lu, S. Ma, X. Ma, Y. Ma, S. Mao, J. Mei, X. Men, Y. Miao, S. Pan, Y. Peng, R. Qin, B. Qu, Z. Shang, L. Shi, S. Shi, F. Song, J. Su, Z. Su, X. Sun, F. Sung, H. Tang, J. Tao, Q. Teng, C. Wang, D. Wang, F. Wang, and H. Wang. Kimi K2: open agentic intelligence. CoRR, abs/2507.20534, 2025a. URL <https://doi.org/10.48550/arXiv.2507.20534>.
- Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, et al. Longbenchv2: Towards deeper understanding and reasoning on realistic long-context multitasks. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 3639–3664, 2025b.
- M. Balunović, J. Dekoninck, I. Petrov, N. Jovanović, and M. Vechev. Matharena: Evaluating llms on uncontaminated math competitions. Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmark, 2025.
- C. Bandi, B. Hertzberg, G. Boo, T. Polakam, J. Da, S. Hassaan, M. Sharma, A. Park, E. Hernandez, D. Rambado, et al. Mcp-atlas: A large-scale benchmark for tool-use competency with real mcp servers. arXiv preprint arXiv:2602.00933, 2026.
- F. Bellard. Qemu, a fast and portable dynamic translator. In Proceedings of the Annual Conference on USENIX Annual Technical Conference, ATEC '05, page 41, USA, 2005. USENIX Association.
- I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio. Neural combinatorial optimization with reinforcement learning, 2017. URL <https://openreview.net/forum?id=rJY3vK9eg>.

- J. Chen, W. Chen, J. Du, J. Hu, Z. Jiang, A. Jie, X. Jin, X. Jin, C. Li, W. Shi, Z. Wang, M. Wang, C. Wei, S. Wei, H. Xin, F. Yang, W. Gao, Z. Yuan, T. Zhan, Z. Zheng, T. Zhou, and T. H. Zhu. Seed-prover 1.5: Mastering undergraduate-level theorem proving via learning from experience, 2025. URL <https://arxiv.org/abs/2512.17260>.
- M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. CoRR, abs/2107.03374, 2021. URL <https://arxiv.org/abs/2107.03374>.
- T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy. TVM: An automated End-to-End optimizing compiler for deep learning. In 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18), pages 578–594, Carlsbad, CA, Oct. 2018. USENIX Association. ISBN 978-1-939133-08-3. URL <https://www.usenix.org/conference/osdi18/presentation/chen>.
- A. Cheng, A. Jacovi, A. Globerson, B. Golan, C. Kwong, C. Alberti, C. Tao, E. Ben-David, G. S. Tomar, L. Haas, et al. The facts leaderboard: A comprehensive benchmark for large language model factuality. arXiv preprint arXiv:2512.10791, 2025.
- X. Cheng, W. Zeng, D. Dai, Q. Chen, B. Wang, Z. Xie, K. Huang, X. Yu, Z. Hao, Y. Li, H. Zhang, H. Zhang, D. Zhao, and W. Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models. CoRR, abs/2601.07372, 2026. doi: 10.48550/ARXIV.2601.07372. URL <https://doi.org/10.48550/arXiv.2601.07372>.
- K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.
- D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K. Li, P. Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. CoRR, abs/2401.06066, 2024. URL <https://doi.org/10.48550/arXiv.2401.06066>.
- T. Dao, D. Haziza, F. Massa, and G. Sizov. Flash-decoding for long-context inference, 2023. URL <https://pytorch.org/blog/flash-decoding/>.
- L. De Moura and N. Bjørner. Z3: an efficient smt solver. In Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems, TACAS’08/ETAPS’08, page 337–340, Berlin, Heidelberg, 2008. Springer-Verlag. ISBN 3540787992.
- DeepSeek-AI. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. CoRR, abs/2406.11931, 2024. URL <https://doi.org/10.48550/arXiv.2406.11931>.
- DeepSeek-AI. Deepseek-v3 technical report. CoRR, abs/2412.19437, 2024. URL <https://doi.org/10.48550/arXiv.2412.19437>.
- DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. CoRR, abs/2405.04434, 2024. URL <https://doi.org/10.48550/arXiv.2405.04434>.

- DeepSeek-AI. Fire-flyer file system, 2025. URL <https://github.com/deepseek-ai/3FS>.
- DeepSeek-AI. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nat.*, 645(8081):633–638, 2025. URL <https://doi.org/10.1038/s41586-025-09422-z>.
- DeepSeek-AI. Deepseek-v3.2: Pushing the frontier of open large language models, 2025. URL <https://arxiv.org/abs/2512.02556>.
- X. Deng, J. Da, E. Pan, Y. Y. He, C. Ide, K. Garg, N. Lauffer, A. Park, N. Pasari, C. Rane, K. Sampath, M. Krishnan, S. Kundurthy, S. Hendryx, Z. Wang, V. Bharadwaj, J. Holm, R. Aluri, C. B. C. Zhang, N. Jacobson, B. Liu, and B. Kenstler. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks?, 2025. URL <https://arxiv.org/abs/2509.16941>.
- H. Ding, Z. Wang, G. Paolini, V. Kumar, A. Deoras, D. Roth, and S. Soatto. Fewer truncations improve language modeling. *arXiv preprint arXiv:2404.10830*, 2024.
- X. Dong, Y. Fu, S. Diao, W. Byeon, Z. CHEN, A. S. Mahabaleshwarkar, S.-Y. Liu, M. V. Keirsbilck, M.-H. Chen, Y. Suhara, Y. C. Lin, J. Kautz, and P. Molchanov. Hymba: A hybrid-head architecture for small language models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=A1ztozypga>.
- X. Du, Y. Yao, K. Ma, B. Wang, T. Zheng, K. Zhu, M. Liu, Y. Liang, X. Jin, Z. Wei, et al. Supergpqa: Scaling llm evaluation across 285 graduate disciplines. *arXiv preprint arXiv:2502.14739*, 2025.
- D. Dua, Y. Wang, P. Dasigi, G. Stanovsky, S. Singh, and M. Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In J. Burstein, C. Doran, and T. Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pages 2368–2378. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1246. URL <https://doi.org/10.18653/v1/n19-1246>.
- X. Gao, M. Dong, X. Miao, W. Du, C. Yu, and H. Chen. Erofs: a compression-friendly read-only file system for resource-scarce devices. In *Proceedings of the 2019 USENIX Conference on Usenix Annual Technical Conference, USENIX ATC '19*, page 149–162, USA, 2019. USENIX Association. ISBN 9781939133038.
- A. P. Gema, J. O. J. Leang, G. Hong, A. Devoto, A. C. M. Mancino, R. Saxena, X. He, Y. Zhao, X. Du, M. R. G. Madani, C. Barale, R. McHardy, J. Harris, J. Kaddour, E. van Krieken, and P. Minervini. Are we done with mmlu? *CoRR*, abs/2406.04127, 2024. URL <https://doi.org/10.48550/arXiv.2406.04127>.
- F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve. Better & faster large language models via multi-token prediction. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=pEWAcejiU2>.
- L. Haas, G. Yona, G. D’Antonio, S. Goldshtein, and D. Das. Simpleqa verified: A reliable factuality benchmark to measure parametric knowledge. *arXiv preprint arXiv:2509.07968*, 2025.
- Y. He, S. Li, J. Liu, Y. Tan, W. Wang, H. Huang, X. Bu, H. Guo, C. Hu, B. Zheng, et al. Chinese simpleqa: A chinese factuality evaluation for large language models. *arXiv preprint arXiv:2411.07140*, 2024.

- D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300, 2020.
- D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. arXiv preprint arXiv:2103.03874, 2021.
- Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, J. Lei, et al. C-Eval: A multi-level multi-discipline chinese evaluation suite for foundation models. arXiv preprint arXiv:2305.08322, 2023.
- D. Hupkes and N. Bogoychev. Multiloko: a multilingual local knowledge benchmark for llms spanning 31 languages. CoRR, abs/2504.10356, 2025. doi: 10.48550/ARXIV.2504.10356. URL <https://doi.org/10.48550/arXiv.2504.10356>.
- B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), June 2018.
- N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. arXiv preprint arXiv:2403.07974, 2024.
- K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein. Muon: An optimizer for hidden layers in neural networks. Cited on, page 10, 2024.
- M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In R. Barzilay and M.-Y. Kan, editors, Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL <https://aclanthology.org/P17-1147>.
- H. Li, Y. Yuan, R. Du, K. Ma, L. Liu, and W. Hsu. DADI: Block-Level image service for agile and elastic application deployment. In 2020 USENIX Annual Technical Conference (USENIX ATC 20), pages 727–740. USENIX Association, July 2020. ISBN 978-1-939133-14-4. URL <https://www.usenix.org/conference/atc20/presentation/li-huiba>.
- H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, and T. Baldwin. CMMLU: Measuring massive multitask language understanding in Chinese. arXiv preprint arXiv:2306.09212, 2023.
- J. Li, W. Zhao, J. Zhao, W. Zeng, H. Wu, X. Wang, R. Ge, Y. Cao, Y. Huang, W. Liu, et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. arXiv preprint arXiv:2510.25726, 2025.
- Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: speculative sampling requires rethinking feature uncertainty. In Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21–27, 2024. OpenReview.net, 2024. URL <https://openreview.net/forum?id=1NdN7eXyb4>.
- J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, E. Lu, J. Yan, Y. Chen, H. Zheng, Y. Liu, S. Liu, B. Yin, W. He, H. Zhu, Y. Wang, J. Wang, M. Dong, Z. Zhang, Y. Kang, H. Zhang, X. Xu, Y. Zhang, Y. Wu, X. Zhou, and Z. Yang. Muon is scalable for LLM training. CoRR, abs/2502.16982, 2025. URL <https://doi.org/10.48550/arXiv.2502.16982>.
- I. Loshchilov and F. Hutter. Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101, 2017.

- K. Lu and T. M. Lab. On-policy distillation. Thinking Machines Lab: Connectionism, 2025. doi:10.64434/tml.20251026. <https://thinkingmachines.ai/blog/on-policy-distillation>.
- Z. Lu, C. Li, Y. Shi, W. Shen, M. Yan, and F. Huang. Corpusqa: A 10 million token benchmark for corpus-level analysis and reasoning. arXiv preprint arXiv:2601.14952, 2026.
- T. Luong, D. Hwang, H. H. Nguyen, G. Ghiasi, Y. Chervonyi, I. Seo, J. Kim, G. Bingham, J. Lee, S. Mishra, A. Zhai, C. H. Hu, H. Michalewski, J. Kim, J. Ahn, J. Bae, X. Song, T. H. Trinh, Q. V. Le, and J. Jung. Towards robust mathematical reasoning. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, 2025. URL <https://aclanthology.org/2025.emnlp-main.1794/>.
- M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in commandline interfaces. arXiv preprint arXiv:2601.11868, 2026.
- MiniMax. Meet minimax-m2, 2025. URL <https://github.com/MiniMax-AI/MiniMax-M2>.
- L. d. Moura and S. Ullrich. The lean 4 theorem prover and programming language. In International Conference on Automated Deduction, pages 625–635. Springer, 2021.
- Y. Nesterov. A method of solving a convex programming problem with convergence rate $O(1/k^2)$. Soviet Mathematics Doklady, 27:372–376, 1983.
- NVIDIA Corporation. cublas documentation, 2024. URL <https://docs.nvidia.com/cuda/cublas/>. Version 12.4. Accessed: 2024-09-16.
- OpenAI. Multilingual massive multitask language understanding (mmmlu), 2024a. URL <https://huggingface.co/datasets/openai/MMMLU>.
- OpenAI. Openai mrcr: Long context multiple needle in a haystack benchmark, 2024b. URL <https://huggingface.co/datasets/openai/mrcr>.
- OpenAI. Learning to reason with llms, 2024c. URL <https://openai.com/index/learning-to-reason-with-llms>.
- OpenAI. Introducing SimpleQA, 2024d. URL <https://openai.com/index/introducing-simpleqa/>.
- OpenAI. Introducing SWE-bench verified we’re releasing a human-validated subset of swe-bench that more, 2024e. URL <https://openai.com/index/introducing-swe-bench-verified/>.
- OpenAI. gpt-oss-120b & gpt-oss-20b model card. CoRR, abs/2508.10925, 2025. doi: 10.48550/ARXIV.2508.10925. URL <https://doi.org/10.48550/arXiv.2508.10925>.
- M. Osama, D. Merrill, C. Cecka, M. Garland, and J. D. Owens. Stream-k: Work-centric parallel decomposition for dense matrix-matrix multiplication on the gpu. In Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, pages 429–431, 2023.
- T. Patwardhan, R. Dias, E. Proehl, G. Kim, M. Wang, O. Watkins, S. P. Fishman, M. Aljubei, P. Thacker, L. Fauconnet, et al. Gdpval: Evaluating ai model performance on real-world economically valuable tasks. arXiv preprint arXiv:2510.04374, 2025.

L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi, et al. Humanity’s last exam. arXiv preprint arXiv:2501.14249, 2025.

W. Qi, Y. Yan, Y. Gong, D. Liu, N. Duan, J. Chen, R. Zhang, and M. Zhou. Prophetnet: Predicting future n-gram for sequence-to-sequence pre-training. In T. Cohn, Y. He, and Y. Liu, editors, Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020, volume EMNLP 2020 of Findings of ACL, pages 2401–2410. Association for Computational Linguistics, 2020. URL <https://doi.org/10.18653/v1/2020.findings-emnlp.217>.

Qwen. Qwen3 technical report. CoRR, abs/2505.09388, 2025. doi: 10.48550/ARXIV.2505.09388. URL <https://doi.org/10.48550/arXiv.2505.09388>.

S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. Zero: Memory optimizations toward training trillion parameter models. In SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1–16. IEEE, 2020.

J. K. Reed, Z. DeVito, H. He, A. Ussery, and J. Ansel. Torch.fx: Practical program capture and transformation for deep learning in python, 2022. URL <https://arxiv.org/abs/2112.08429>.

D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. arXiv preprint arXiv:2311.12022, 2023.

G. T. M. Riviere, S. Pathak, P. G. Sessa, C. Hardin, S. Bhupatiraju, L. Hussenot, T. Mesnard, B. Shahriari, A. Ram’e, J. Ferret, P. Liu, P. D. Tafti, A. Friesen, M. Casbon, S. Ramos, R. Kumar, C. L. Lan, S. Jerome, A. Tsitsulin, N. Vieillard, P. Stańczyk, S. Girgin, N. Momchev, M. Hoffman, S. Thakoor, J.-B. Grill, B. Neyshabur, A. Walton, A. Severyn, A. Parrish, A. Ahmad, A. Hutchison, A. Abdagic, A. Carl, A. Shen, A. Brock, A. Coenen, A. Laforge, A. Paterson, B. Bastian, B. Piot, B. Wu, B. Royal, C. Chen, C. Kumar, C. Perry, C. A. Welty, C. A. Choquette-Choo, D. Sinopalnikov, D. Weinberger, D. Vijaykumar, D. Rogozinska, D. Herbison, E. Bandy, E. Wang, E. Noland, E. Moreira, E. Senter, E. Eltyshev, F. Visin, G. Rasskin, G. Wei, G. Cameron,

G. Martins, H. Hashemi, H. Klimczak-Plucińska, H. Batra, H. Dhand, I. Nardini, J. Mein, J. Zhou, J. Svensson, J. Stanway, J. Chan, J. Zhou, J. Carrasqueira, J. Iljazi, J. Becker, J. Fernandez, J. R. van Amersfoort, J. Gordon, J. Lipschultz, J. Newlan, J. Ji, K. Mohamed, K. Badola, K. Black, K. Millican, K. McDonell, K. Nguyen, K. Sodhia, K. Greene, L. L. Sjoesund, L. Usui, L. Sifre, L. Heuermann, L. Lago, L. McNealus, L. B. Soares, L. Kilpatrick, L. Dixon, L. B. Martins, M. Reid, M. Singh, M. Iverson, M. Gerner, M. Velloso, M. Wirth, M. Davidow, M. Miller, M. Rahtz, M. Watson, M. Risdal, M. Kazemi, M. Moynihan, M. Zhang, M. Kahng, M. Park, M. Rahman, M. Khatwani, N. Dao, N. Shad Bardoliwalla, N. Devanathan, N. Dumai, N. Chauhan, O. Wahltinez, P. Botarda, P. Barnes, P. Barham, P. Michel, P. Chong Jin, P. Georgiev, P. Culliton, P. Kuppala, R. Comanescu, R. Merhej, R. Jana, R. A. Rokni, R. Agarwal, R. Mullins, S. Saadat, S. M. M. Carthy, S. Perrin, S. M. R. Arnold, S. Bastian Krause, S. Dai, S. Garg, S. Sheth, S. Ronstrom, S. Chan, T. Jordan, T. Yu, T. Eccles, T. Hennigan, T. Kociský, T. Doshi, V. Jain, V. Yadav, V. Meshram, V. Dharmadhikari, W. Barkley, W. Wei, W. Ye, W. Han, W. Kwon, X. Xu, Z. Shen, Z. Gong, Z. Wei, V. Cotruta, P. Kirk, A. Rao, M. Giang, L. Peran, T. Warkentin, E. Collins, J. Barral, Z. Ghahramani, R. Hadsell, D. Sculley, J. Banks, A. Dragan, S. Petrov, O. Vinyals, J. Dean, D. Hassabis, K. Kavukcuoglu, C. Farabet, E. Buchatskaya, S. Borgeaud, N. Fiedel, A. Joulin, K. Kenealy, R. Dadashi, and A. Andreev. Gemma 2: Improving open language models at a practical

size. arXiv preprint arXiv:2408.00118, 2024.

S. Roller, S. Sukhbaatar, A. Szlam, and J. Weston. Hash layers for large sparse models. In M. Ranzato, A. Beygelzimer, Y. N. Dauphin, P. Liang, and J. W. Vaughan, editors, *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 17555–17566, 2021. URL <https://proceedings.neurips.cc/paper/2021/hash/92bf5e6240737e0326ea59846a83e076-Abstract.html>.

B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, S. Dusan, V. Elango, M. Golub, A. Heinecke, P. James-Roxby, D. Jani, G. Kolhe, M. Langhammer, A. Li, L. Melnick, M. Mesmakhosroshahi, A. Rodriguez, M. Schulte, R. Shafipour, L. Shao, M. Siu, P. Dubey, P. Micikevicius, M. Naumov, C. Verrilli, R. Wittig, D. Burger, and E. Chung. Microscaling data formats for deep learning, 2023.

K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi. Winogrande: An adversarial winograd schema challenge at scale, 2019.

Z. Shao, Y. Luo, C. Lu, Z. Z. Ren, J. Hu, T. Ye, Z. Gou, S. Ma, and X. Zhang. Deepseekmath-v2: Towards self-verifiable mathematical reasoning, 2025. URL <https://arxiv.org/abs/2511.22570>.

N. Shazeer. Fast transformer decoding: One write-head is all you need. CoRR, abs/1911.02150, 2019. URL <http://arxiv.org/abs/1911.02150>.

N. Shazeer. Glu variants improve transformer. arXiv preprint arXiv:2002.05202, 2020.

F. Shi, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, D. Das, and J. Wei. Language models are multilingual chain-of-thought reasoners. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. URL <https://openreview.net/forum?id=fR3wGCK-IXp>.

J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V. Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022.

G. Tsoukalas, J. Lee, J. Jennings, J. Xin, M. Ding, M. Jennings, A. Thakur, and S. Chaudhuri. Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition, 2024. URL <https://arxiv.org/abs/2407.11214>.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polo-Sukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

L. Wang, H. Gao, C. Zhao, X. Sun, and D. Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. CoRR, abs/2408.15664, 2024a. URL <https://doi.org/10.48550/arXiv.2408.15664>.

L. Wang, Y. Cheng, Y. Shi, Z. Mo, Z. Tang, W. Xie, T. Wu, L. Ma, Y. Xia, J. Xue, et al. Tilelang: Bridge programmability and performance in modern neural kernels. In *The Fourteenth International Conference on Learning Representations*, 2026.

- Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. CoRR, abs/2406.01574, 2024b. URL <https://doi.org/10.48550/arXiv.2406.01574>.
- J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus, and A. Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents. arXiv preprint arXiv:2504.12516, 2025.
- T. Wei, J. Luan, W. Liu, S. Dong, and B. Wang. Cmath: Can your language model pass chinese elementary school math test?, 2023.
- G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. In The Twelfth International Conference on Learning Representations, ICLR2024, Vienna, Austria, May 7-11, 2024. OpenReview.net, 2024. URL <https://openreview.net/forum?id=NG7sS51zVF>.
- Z. Xie, Y. Wei, H. Cao, C. Zhao, C. Deng, J. Li, D. Dai, H. Gao, J. Chang, K. Yu, L. Zhao, S. Zhou, Z. Xu, Z. Zhang, W. Zeng, S. Hu, Y. Wang, J. Yuan, L. Wang, and W. Liang. mhc: Manifold-constrained hyper-connections, 2026. URL <https://arxiv.org/abs/2512.24880>.
- L. Xu, H. Hu, X. Zhang, L. Li, C. Cao, Y. Li, Y. Xu, K. Sun, D. Yu, C. Yu, Y. Tian, Q. Dong, W. Liu, B. Shi, Y. Cui, J. Li, J. Zeng, R. Wang, W. Xie, Y. Li, Y. Patterson, Z. Tian, Y. Zhang, H. Zhou, S. Liu, Z. Zhao, Q. Zhao, C. Yue, X. Zhang, Z. Yang, K. Richardson, and Z. Lan. CLUE: A chi-nese language understanding evaluation benchmark. In D. Scott, N. Bel, and C. Zong, editors, Proceedings of the 28th International Conference on Computational Linguistics, COLING2020, Barcelona, Spain (Online), December 8-13, 2020, pages 4762–4772. International Committee on Computational Linguistics, 2020. doi: 10.18653/V1/2020.COLING-MAIN.419. URL <https://doi.org/10.18653/v1/2020.coling-main.419>.
- J. Yang, K. Lieret, C. E. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang. Swesmith: Scaling data for software engineering agents, 2025. URL <https://arxiv.org/abs/2504.21798>.
- R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In A. Korhonen, D. R. Traum, and L. Màrquez, editors, Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pages 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/p19-1472. URL <https://doi.org/10.18653/v1/p19-1472>.
- C. Zhang, K. Du, S. Liu, W. Kwon, X. Mo, Y. Wang, X. Liu, K. You, Z. Li, M. Long, J. Zhai, J. Gonzalez, and I. Stoica. Jenga: Effective memory management for serving llm with heterogeneity. In Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles, SOSP '25, page 446–461, New York, NY, USA, 2025a. Association for Computing Machinery. ISBN 9798400718700. doi: 10.1145/3731569.3764823. URL <https://doi.org/10.1145/3731569.3764823>.
- S. Zhang, N. Zheng, H. Lin, Z. Jiang, W. Bao, C. Jiang, Q. Hou, W. Cui, S. Zheng, L.-W. Chang, Q. Chen, and X. Liu. Comet: Fine-grained computation-communication overlapping for mixture-of-experts. 2025b. URL <https://arxiv.org/abs/2502.19811>.
- C. Zhao, L. Zhao, J. Li, Z. Xu, and C. Xu. Deepgemm: clean and efficient fp8 gemm kernels with fine-grained scaling.

<https://github.com/deepseek-ai/DeepGEMM>, 2025.

W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: A human-centric benchmark for evaluating foundation models. CoRR, abs/2304.06364, 2023. doi: 10.48550/arXiv.2304.06364. URL <https://doi.org/10.48550/arXiv.2304.06364>.

D. Zhu, H. Huang, Z. Huang, Y. Zeng, Y. Mao, B. Wu, Q. Min, and X. Zhou. Hyper-connections. In The Thirteenth International Conference on Learning Representations, ICLR2025, Singapore, April 24-28, 2025. OpenReview.net, 2025. URL <https://openreview.net/forum?id=9FqARW7dwB>.

X. Zhu, D. Cheng, H. Li, K. Zhang, E. Hua, X. Lv, N. Ding, Z. Lin, Z. Zheng, and B. Zhou. How to synthesize text data without model collapse? arXiv preprint arXiv:2412.14689, 2024.

T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul, S. Brunner, C. Gong, J. Hoang, A. R. Zebaze, X. Hong, W. Li, J. Kaddour, M. Xu, Z. Zhang, P. Yadav, and et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. In The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025. OpenReview.net, 2025. URL <https://openreview.net/forum?id=YrycTjllL0>.

附录

A 作者名单与致谢

A.1 Author List

Authors are listed alphabetically by their first name. Names marked with * denote individuals who have departed from our team.

Research & Engineering: Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenhao Xu, Chenze Shao, Chong Ruan, Conner Sun, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Donghao Li, Erhang Li, Fangyun Lin, Fangzhou Yuan, Feiyu Xia, Fucong Dai, Guangbo Hao, Guanting Chen, Guoai Cao, Guolai Meng, Guowei Li, Han Yu, Han Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoling Zhang, Haoming Luo, Haoran Wei, Haotian Yuan, Haowei Zhang, Haowen Luo, Haoyu Chen, Haozhe Ji, Honghui Ding, Hongxuan Tang, Huanqi Cao, Huazuo Gao, Hui Qu, Hui Zeng, J. Yang, J. Q. Zhu, Jia Yu, Jialiang Huang, Jiasheng Ye, Jiashi Li, Jiaxin Xu, Jiewen Hu, Jin Yan, Jingchang Chen, Jingli Zhou, Jingting Xiang, Jingyang Yuan, Jingyuan Cheng, Jinhua Zhu, Jiping Yu, Joseph Sun, Jun Ran, Junguang Jiang, Junjie Qiu, Junlong Li*, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Kexing Zhou, Kezhao Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Wang, Li Zhang, Liang Zhao, Lihua Guo, Lingxiao Luo, Linwang Ma, Litong Wang, Liyu Cai, Liyue Zhang, Longhao Chen, M.S. Di, M.Y. Xu, Max Mei, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Panpan Huang, Peixin Cong, Peiyi Wang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Qiwei Jiang, Rui Tian, Ruifan Xu, Ruijie Lu, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqian Chen, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, S.H. Liu, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaofei Cai,

Shaoheng Nie, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shun-feng Zhou, Shuting Pan, Shuying Yu, Songyang Zhou, Tao Ni, Tao Yun, Tian Jin, Tian Pei, Tian Ye, Tianle Lin, Tianran Ji, Tianyi Cui, Tianyuan Yue, Tingting Yu, Tun Wang, W. Zhang, Wangding Zeng, Weilin Zhao, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjing Yao, Wenjun Gao, Wenkai Yang, Wenlve Huang, Wentao Zhang, Wenting Ma, Xi Gao, Xiang He, Xiangwen Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingchen Liu, Xingkai Yu, Xingyou Li, Xinyu Yang, Xu Chen, Xuanyu Wang, Xuecheng Su, Xuheng Lin, Xuwei Fu, Y.C. Yan, Y.Q. Wang, Y.W. Ma, Yanfeng Luo, Yang Zhang, Yanhong Xu, Yanru Ma, Yanwen Huang, Yao Li, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Qian, Yi Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yijia Wu, Yiliang Xiong, Ying He, Ying Zhou, Yingjia Luo, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiang Zhang, Yixiao Chen, Yixuan Tan, Yixuan Wei, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo, Yongtong Wu, Yu Wu, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yuanhao Li, Yudian Wang, Yuhang Wu, Yuhao Meng, Yuheng Zou, YuKun Li, Yunfan Xiong, Yupeng Chen, Yuqian Cao, Yuqian Wang, Yushun Zhang, Yutong Lin, Yuxian Gu, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuxuan Zhou, Yuyang Zhou, Yuzhen Huang, Z.F. Wu, Zehao Wang, Zehua Zhao, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhixuan Chen, Zhiyu Wu, Zhizhou Ren, Zhuoshu Li, Zhuping Zhang, Zian Xu, Zihao Wang, Zihui Gu, Zijia Zhu, Zilin Li, Zipeng Zhang*, Ziwei Xie, Ziyi Gao, Zizheng Pan, Zongqing Yao.

Business & Compliance: Chenchen Ling, Chengyu Hou, Dongjie Ji, Fang Wei, Hengqing Zhang, Jia Luo, Jia Song, Jialu Cai, Jian Liang, Jiangting Zhou, Jieyu Yang, Jin Chen, Jingzi Zhou, Junmin Zheng, Leyi Xia, Linyan Zhu, Miaojun Wang, Mingming Li, Minmin Han, Ning Wang, Panpan

Wang, Peng Zhang, Ruyi Chen, Shangmian Sun, Shaoqing Wu, W.L. Xiao, Wei An, Wenqing Hou, Xianzu Wang, Xiaowen Sun, Xiaoxiang Wang, Xinyu Zhang, Xueyin Chen, Yao Xu, Yi Shao, Yiling Ma, Ying Tang, Yuehan Yang, Yuer Xu, Yukun Zha, Yuping Lin, Yuting Yan, Zekai Zhang, Zhe Ju, Zheren Gao, Zhongyu Wu, Zihua Qu, Ziyi Wan.

A.2 致谢

我们感谢 Dolly Deng 和其他测试人员，他们就 DeepSeek-V4 系列模型的能力提出了宝贵的建议和反馈。

B 评测细节

表 9 | DeepSeek-V4-Pro 的智能体搜索与检索增强搜索对比。

Difficulty	Category	#	Agent Win	RAG Win	Tie	Agent%	RAG%	Tie%
Easy	Objective Q&A (客观问答)	196	110	43	43	56.1	21.9	21.9
	Subjective Q&A (主观问答)	321	198	56	67	61.7	17.4	20.9
Hard	Objective Q&A (客观问答)	168	102	33	33	60.7	19.6	19.6
	Subjective Q&A (主观问答)	184	126	27	31	68.5	14.7	16.8
	Total (总计)	869	536	159	174	61.7	18.3	20.0

表 10 | DeepSeek-V4-Pro 的成本对比：智能体搜索 vs. 检索增强搜索 (均值)。在智能体搜索中，大多数工具调用是并行执行的。

Version	Tool Calls	Prefill (tokens)	Output (tokens)
V4 Agentic Search	16.2	13649	1526
V4 Retrieval Augmented Search	—	10453	1308

表 11 | DeepSeek-V4-Pro 与 DeepSeek-V3.2 在搜索问答任务上的对比评测。

Category	Subcategory	#	Internal Evaluation (内部综合评估)					
			V4 win	V3.2 win	tie	V4%	V3.2%	tie%
Objective Q&A (客观问答)	Single-value Search (单值信息查找)	95	36	10	49	37.9	10.5	51.6
	Entity Search (实体信息查找)	99	24	7	68	24.2	7.1	68.7
	Enumerative Search (枚举型信息查找)	95	19	8	68	20.0	8.4	71.6
	Subtotal (小计)	289	79	25	185	27.3	8.7	64.0
	Causal Analysis (原因分析)	100	28	5	67	28.0	5.0	67.0
	Comparison (对比)	96	28	20	48	29.2	20.8	50.0
	Advice Seeking (寻求建议)	92	23	8	61	25.0	8.7	66.3
Subjective Q&A (主观问答)	Recommendation (推荐)	95	26	19	50	27.4	20.0	52.6
	Planning & Strategy (攻略计划)	92	32	11	49	34.8	12.0	53.3
	Opinion & Evaluation (评价看法)	96	30	8	58	31.2	8.3	60.4
	Trend Analysis (趋势分析)	96	23	3	70	24.0	3.1	72.9
	Subtotal (小计)	667	190	74	403	28.5	11.1	60.4
TOTAL (总计)		956	269	99	588	28.1	10.4	61.5



图 14 | 需要比较纳斯达克两种定投策略的任务输出示例。

Oral Writing (口头文本)	Comment (评论)	44	34	8	2	77.27	18.18	4.55
	Subtotal (小计)	390	271	101	18	69.49	25.90	4.55
	Speech (发言稿)	226	135	85	6	59.73	37.61	2.00
	Narration Script (口播文案)	51	25	23	3	49.02	45.10	5.00
	Sales Script (话术)	31	22	6	3	70.97	19.35	9.00
	Dialogue (对话文本)	10	4	6	0	40.00	60.00	0.00
	Congratulatory (祝贺文本)	1	1	0	0	100.00	0.00	0.00
	Subtotal (小计)	319	187	120	12	58.62	37.62	3.00
Official Document (公文文本)	Administrative Doc (事务文书)	117	60	53	4	51.28	45.30	3.00
	Personal Doc (个人文书)	73	45	27	1	61.64	36.99	1.00
	Government Doc (行政公文)	34	19	14	1	55.88	41.18	2.00
	Speech (发言稿)	3	1	2	0	33.33	66.67	0.00
	Essay Writing (申论写作)	3	1	1	1	33.33	33.33	33.33
Academic Writing (学术文本)	Subtotal (小计)	230	126	97	7	54.78	42.17	3.00
	Research Paper (学术论文)	104	67	32	5	64.42	30.77	4.00
	Coursework (课程作业)	90	53	35	2	58.89	38.89	2.00
	Academic Support (学术辅助)	15	11	3	1	73.33	20.00	6.00
	Science Outreach (专业科普)	7	6	1	0	85.71	14.29	0.00
	Subtotal (小计)	216	137	71	8	63.43	32.87	3.00
	Total (总计)	3170	1986	1081	103	62.65	34.10	3.00

表 13 | DeepSeek-V4-Pro 与 Gemini-3.1-Pro 在中文创意写作上的对比分析。

Subcategory (文体)	#	Instruction Following(指令遵循)						Writing Quality (写作质量)				
		DS	Gem	Tie	DS%	Gem%	Tie%	DS	Gem	Tie	DS%	Gem%
Fiction (小说故事)	836	504	323	5	60.58	38.82	0.60	672	157	3	80.77	18.87
General Fiction (泛小说故事)	662	368	290	3	55.67	43.87	0.45	467	194	0	70.65	29.35
Fan Fiction (同人文)	410	253	150	3	62.32	36.95	0.74	338	67	1	83.25	16.50
General Fan Fic. (泛同人文)	202	111	90	1	54.95	44.55	0.50	161	40	1	79.70	19.80
Narrative (记叙文)	171	115	54	2	67.25	31.58	1.17	141	30	0	82.46	17.54
General Prose (泛散文)	124	83	40	1	66.94	32.26	0.81	88	36	0	70.97	29.03
Prose (散文)	112	74	38	0	66.07	33.93	0.00	92	20	0	82.14	17.86
Writing Style (文笔)	112	81	31	0	72.32	27.68	0.00	86	26	0	76.79	23.21
Classical Poetry (古诗文)	48	24	24	0	50.00	50.00	0.00	39	9	0	81.25	18.75
Modern Poetry (现代诗)	43	23	20	0	53.49	46.51	0.00	32	11	0	74.42	25.58
Lyrics (歌词)	30	8	22	0	26.67	73.33	0.00	16	14	0	53.33	46.67
Literary Appreciation (赏析)	27	20	7	0	74.07	25.93	0.00	18	9	0	66.67	33.33
General Argument. (泛议论文)	24	15	9	0	62.50	37.50	0.00	17	7	0	70.83	29.17
General Narrative (泛记叙文)	23	11	12	0	47.83	52.17	0.00	15	8	0	65.22	34.78

General Classical (泛古文诗歌)	9	5	4	0	55.56	44.44	0.00	5	4	0	55.56	44.44
Creative Writing (创意写作)	6	2	4	0	33.33	66.67	0.00	4	2	0	66.67	33.33
Argumentative (议论文)	5	5	0	0	100.00	0.00	0.00	5	0	0	100.00	0.00
General Mod. Poetry (泛现代诗)	2	1	1	0	50.00	50.00	0.00	2	0	0	100.00	0.00
Total (总计)	2837	1703	1119	15	60.03	39.44	0.53	2198	634	5	77.48	22.35

表 14 | DeepSeek-V4-Pro 与 Claude-Opus-4.5 在复杂指令遵循与多轮写作上的对比。

Category	#	Internal Evaluation (内部综合评估)					
		DS	Opus	Tie	DS%	Opus%	Tie%
Complex Inst. Following (复杂指令跟随)	49	23	26	0	46.9%	53.1%	0.0%
Multi-Turn Writing (多轮写作)	147	67	76	4	45.6%	51.7%	2.7%
Total (总计)	196	90	102	4	45.9%	52.0%	2.0%